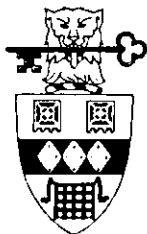


VECTOR

Special Issue: Error Trapping ... plus:

- * APL Award to British Airways
- * 16-page Educational Supplement
- * Reviews of APL2/PC and Dyalog for MS DOS
- * Mike Williams on Rule Induction
- * David Piper on Arrays of Functions
- * APL Printing from APL.68000



***The Journal of the
British APL Association***

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.6 No.1 July 1989

Contributions

All contributions to VECTOR may be sent to the Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (ideally with a photograph of the author and a brief biographical note). Unless otherwise specified, each item will be considered for publication as a personal statement by the author.

Please supply as much material as possible in machine-readable form, ideally on an IBM PC compatible (DSDD) diskette. APL code should be kept separate from the text, and can be accepted as camera-ready copy, or as APL*PLUS/PC workspaces or IBM .AIO files.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the editor of your intention to re-use material from VECTOR.

Membership Rates 1989-90

Category	Fee	Vectors	Passes
Non-voting Student	£5	1	1
UK Private	£10	1	1
Overseas Private	£18	1	1
(Supplement for Airmail)	£8		
UK Corporate Membership	£85	10+	5
Overseas Corporate	£140	10+	
Sustaining	£360	50+	5

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Treasurer on the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per full page, £100 for half-page or less (subject to layout constraints).

Deadlines for bookings and copy are given under the Quick-reference Diary

Advertisements should be booked with, and sent to, Alison Chatterton, whose address is given with the Index to Advertisers.

CONTENTS

EDITORIAL:	Peter Donnelly	Page 3
APL NEWS		
Quick Reference Diary		5
APL Course Diary		7
APL90 Information		8
British APL Association News	Graham Parkhouse	12
Special Achievement Award to BA		13
News from Sustaining Members	Alison Chatterton	18
The Education VECTOR	Alan Sykes	23
REVIEWS		
APL Product Guide	Cathy Dargue	38
The CIPS APL Toolkit	Jonathan Barman	51
Using IBM's APL2 on the PC	Anthony Camacho	57
Impressions: Dyalog APL for MS DOS	Jonathan Martin	61
GENERAL ARTICLES		
Printing APL Matrices	Jerry M. Brennan	64
Personal Construct Analysis in APL	Mike Williams	73
TECHNICAL SECTION: Error Trapping		
Hackers Corner	Adrian Smith	90
Error Trapping: Tutorial Introductions for ...		
APL*PLUS (mainframe and PC)	Dave Ziemann	91
APL.68000	David Eastwood	96
Dyalog APL	Pauline Brand	100
A Direct Definition Handler	Dave Ziemann	110
Semantic Class and Arrays of Functions	David Piper	118
APL Printing from APL.68000	Iain Hayward	127
British APL Association Software Library		
Order Form		134
Index to Advertisers		135

dyalog APL

for 386 PCs under MS-DOS

Dyalog APL for 386 PCs under MS-DOS is a complete implementation of Dyadic's popular APL for Unix systems, AND fully exploits the power of a DOS 386-PC.

JUST LOOK AT THE FEATURES . . .

- *An exciting new Screen Manager in which the contents of the screen are 'shared' with a nested system variable [ISM (Screen Map).*

Facilities include:

Pop-up windows by localisation of [ISM
Scrolling (vertical, horizontal, linked)
GENUINE numeric fields
Date fields
NESTED forms

- Fully integrated window-based Session Manager, Editor and Debugging Environment.
- Compatible with the Draft ISO standard for APL
- Huge Workspaces (16Mb and MORE!)
- Virtual Memory Support (using on-chip demand paging)
- Full 32-bit processing speed
- *APL2 features including:*
 - Nested Arrays
 - Defined Operators
 - Selective Specification
- *Commercial features including:*
 - [FMT
 - Component Files
 - Error Handling
 - "AP124-like" Screen Manager
- External Variables
- Interface to GSS*CGI for fast device-independent graphics
- C Interface

. . . THE BENEFITS SPEAK FOR THEMSELVES

Requirements: 386/486-based PC or PS/2, 640K DOS Memory plus 2Mb RAM, EGA or VGA (colour STRONGLY recommended), 80387 coprocessor STRONGLY recommended, DOS 3.3 or later.

For further information contact:

Dyadic Systems Limited

Park House, The High Street, Alton, Hampshire, GU34 1EN
Telephone: (0420) 87024 Fax: (0420) 89886 Telex: 85881

Editorial: Join the British APL Association

by Peter Donnelly

I am addressing this editorial to the APL89 delegates of APL89, particularly non-UK delegates, who will have received this issue of Vector free of charge. The reason we are giving it away is not entirely altruistic; we want you to join the British APL Association! For overseas members, Vector is the ONLY reason for joining the BAA, so what do you get?

For me, Vector is first of all 'a good read'. It is deliberately NOT a dry technical journal, but tries to be interesting, topical, and occasionally provocative. It contains something for everyone.

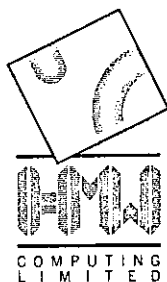
First is usually the correspondence columns. Vector's policy is to print any correspondence however contentious, as long as it is at least vaguely related to APL and is not actually libellous. Attacks on the BAA committee are happily printed alongside erudite arguments over the merits of various APL keyboards.

Next there is 'The Education Vector' edited by Dr. Alan Sykes of Swansea University and aimed fairly and squarely at new APL'ers and the growing community of I-APL users. Education vector contains valuable hints for APL novices and often surprises those of us who think we know it all!

Following this you get News from Sustaining Members, a useful update on what the vendors are up to (or at least what they would have you think they are up to) and a comprehensive APL Product Guide. Several overseas organisations already take advantage of free listings in the Product Guide, and we would like to see more. The BAA would also welcome overseas companies as sustaining members - the "News" section gives you an inexpensive and convenient way to keep a significant part of the APL market informed of your products and services.

Then there are the articles. These are split into "general" and "technical". General articles often reach (and occasionally surpass!) the standard expected of APL conference papers, but they tend to be presented in a less formal and more readable style. The technical section and "Hacker's Corner", is edited by Dave "show me your interpreter and I'll crash it in 5 minutes" Ziemann, and always contains informative snippets for the professional APL user.

Vector is published quarterly. The BAA intends to make it the leading APL journal for the English speaking world. You will find an application form for the BAA at the back of this issue, or you can contact a BAA representative at the conference. Join Today!

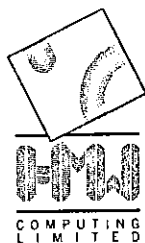


SOLUTIONS FOR THE CITY

We are a specialist firm of consultants with a prestigious list of clients including major City institutions.

For one of these clients we have recently developed a foreign exchange dealing system with the following features:- deal entry, deal enquiry, multi-currency position keeping, basket currency calculators, arbitrage calculators and ticket production.

If you are interested in foreign exchange dealing, treasury operations, portfolio management, commodity broking or similar areas we would be delighted to discuss how we could help. Call Stan Wilkinson, HMW COMPUTING LIMITED Hamilton House 1 Temple Avenue London EC4Y 0HA Telephone 01-353 4212



EXCITING POSITIONS AT THE LEADING EDGE

With top City institutions and leading Blue chip companies amongst our client list we have a lot to offer experienced APL programmers.

Call Stan Wilkinson to discuss your future.

HMW COMPUTING LIMITED
Hamilton House 1 Temple Avenue
London EC4Y 0HA Telephone 01-353 4212

Quick Reference Diary

Date	Venue	Event
29 September 1989	London	Design of large APL systems
27 October	London/Heathrow	APL at British Airways Demonstrations and Discourse by the APLers at BA.
24 November	London	BAA Meeting
12 January 1990	London	BAA Meeting
16 February	London	BAA Meeting
23 March	London	BAA Meeting
20 April	London	BAA Meeting
8th June	London	BAA Annual General Meeting

British APL Association meetings are to be held at the Royal Over-Seas League, Park Place, near Green Park Tube station. Meetings begin at 2.00pm and close at 5.30pm.

Topics for Forthcoming BAA Meetings

by Peter Branson

- Design of large APL systems. Considerations of workspace structure; function modularity; data structures; file systems; end-user screens etc. (Illustrated with practical applications.)
- APL at British Airways. A review of some of their major applications using APL. (This meeting will be held at the BA headquarters at Heathrow Airport, where a full demonstration of the systems is possible.)

Dates for Future Issues of VECTOR

	Vol.6 No.2	Vol.6 No.3	Vol.6 No.4
Copy date	1st Sept 89	30th Nov 89	1st March 90
Ad booking	10th Sept 89	8th Dec 89	15th March 90
Ad Copy	21st Sept 89	23rd Dec 89	25th March 90
Distribution	October 89	January 90	April 90

Would you like to receive
VECTOR regularly?

**THEN JOIN THE BRITISH
APL ASSOCIATION TODAY!**

It's easy - just complete the application
form in the back of this VECTOR and
send it with your remittance to
Alison Chatterton at the address
at the foot of the form.

APL

APL Training Courses

(Prices quoted are per course, unless otherwise stated)

Dates shown cover the period from July to December 1989. For confirmation of dates and/or further details please contact the vendor directly.

Level	Company	Days	Dates
Beginners	Cocking & Drury	3	4 July, 18 July
	MicroAPL	1	24 Aug, 19 Oct
	Mercia	3	17 July, 12 Sept
APL*PLUS/PC	MicroAPL	1	20 July, 14 Sept, 9 Nov
Intermediate	MicroAPL	1	6 July, 31 Aug, 26 Oct
Advanced	MicroAPL	1	13 July, 31 Aug, 14 Sept, 26 Oct
System Design	Cocking & Drury	4	
	Mercia	3	3 August
Statgraphics	Cocking & Drury	2	11 July
	Mercia	1	2 August

APL 90
APL For The Future

The International Conference on APL
13 - 17 August 1990
Hotel Marienlyst, Helsingør, Denmark

CALL FOR PAPERS

Organized by
The APL Special Interest Group of Danish Data Association
Co-sponsored by
The Association of Computing Machinery, Special Interest Group on APL,
ACM/SIGAPL

APL90 in Denmark

The APL90 conference will take place from 13th to 17th August 1990 at the Marienlyst Hotel in Elsinore (Helsingør), near Copenhagen.

This seaside hotel offers an excellent environment for APL90, the second APL conference to be arranged in Denmark. We are looking forward to a conference with many valuable papers on the progress of APL and its use in numerous applications areas.

We plan to introduce at this conference the concept of poster sessions, in which the speakers are given a small booth to present a poster with an eyecatching presentation of their work, and with the opportunity to discuss it with by-passers and to hand out written material. We believe that this will allow us to present even more interesting papers at APL90 than we otherwise would have been able to find room for.

We encourage you to submit a paper to the conference. You may choose between submitting a regular paper and submitting a proposal for a poster session. The schedule for submission is as follows:

Please send your abstract before the end of September 1989 to the APL90 address shown at the back of this folder, or to one of the members of the program committee.

Then send a complete draft before the end of November 1989. The submitted papers will be reviewed, and an answer will be sent to all submitters before the end of January 1990.

Both abstracts and complete drafts may be sent by mail, telefax, PC-readable diskette, or by any of the available mailboxes.

For accepted papers we expect to receive a camera-ready copy before the end of April 1990. Detailed instructions on the preparation of the paper will be mailed with the acceptance. We intend to mail the conference proceedings to all participants before the conference.

Plans for the conference include a Software Exchange as well as an exhibition, showing many of the new offerings in APL systems.

A call for participation will be distributed early 1990. Please note that a wide range of accommodation has been reserved already, from four star hotels to family rooms and a youth hostel. Also, a program for the families will be offered during the conference.

APL90 Program Statement:

In line with tradition, APL90 will be a forum for exchange of experience and ideas within the world of APL. 'For the future' is the main theme of the conference, which will illustrate, through the presentations and discussions, why APL not only has been but will remain an excellent tool for Information Management:

Whenever new technologies emerged, APL was there to greet them. Think of SQL/DB2 interfacing and vector processing, to name a few. By its open and dynamic structure, APL is particularly suited for the development and implementation of new ideas and techniques. We encourage papers illustrating how such frontline concepts are delivered through the use of APL. Any bright ideas on Image Processing, Neural Networks ...?

Effective Information Management and Management Information will be increasingly important for the competitive ability of any organization in the future. We invite papers showing how APL can combine the creation and use of corporate-wide information systems with end-user computing by professionals. We especially welcome papers which demonstrate how this can be done in an efficient, structured and well documented way.

We trust that APL90 will provide APL implementors and philosophers alike with the incentive and opportunity to present their current work and to discuss ideas and plans for the future. However, as much program development in other languages now takes place in environments that rival or surpass APL's once unique environment, we hope that users will take this opportunity to put demand on the implementors for modernizing APL's look and feel. We encourage papers that guide the implementors in the right direction, so that our language can remain competitive in the world of windows, CASE, object oriented programming systems and distributed databases.

APL was originally developed for educational purposes and has recently started to find its way back into schools due to its availability on small computers. It is an attractive educational tool because it enables teachers to use data with its natural structure, rather than in a traditional, scalar fashion. We encourage papers which focus on the advantages of this use of APL.

APL implementations continue to evolve, with extensions to the language itself, as well as interfaces to - and from - other environments; recently, encouraging signs have been noted with respect to standardization of 'enhanced' APL dialects. We would very much like to see papers presenting problems in and, hopefully, proposing solutions to these areas.

APL90 Addresses:**APL90 Conference:**

c/o Kim S. Andreasen
Snerlevej3,
DK-3000 Helsingør, Denmark

Mailbox:	IPSA STSC:	APL90
	IBM VNET:	GJERLOV at CPHVM1
	EARN/BITNET:	APL90 at DKUCCC11
	BIX:	kandreasen

Organising Committee:

ChairmanGitte Christensen, I.P. Sharp, Denmark.
IPSA Mailbox: GITTE

TreasuryKim S. Andreasen, Beta-Plan ApS,
IPSA mailbox: KIMA

MarketingIvan Jensen, Novo-Nordisk A/S, Denmark.
IPSA mailbox: LJENSEN

Program Committee:

ChairmanPer Gjerlov, IBM A/S, Denmark.
IBM VNET: GJERLOV at CPHVM1,
IPSA mailbox: PER

Conference Bureau:

DIS Congress Service,
Linde Alle 48,
DK-2720 Vanløse, Denmark
Attn: Nils Ole Kondrup

**Official carrier**

Ask your nearest SAS office for
assistance with your travel arrangements.



Renaissance Data Systems
Enlightenment Thru Information Processing
ALL APL BOOKS IN PRINT
Catalog Excerpts June 1, 1989



If we do not carry it (in English, at least), we dare you to find it elsewhere!
Specializing in books on:
APL AS A TOOL OF THOUGHT!

ACCOUNTING STRUCTURED IN APL , Ijiri, 1984, 491p.	\$10.00
The basic principles of accounting with numerous APL expressions to model them.	
ABSTRACT ALGEBRA - A COMPUTATIONAL APPROACH , Simms, 1984, 491p.	\$55.95
An introductory course in the basic concepts of algebra and fundamental algebraic algorithms. A methodology for investigating computational questions. Includes lemmas, theorems, and corollaries.	
COMPUTING IN STATISTICAL SCIENCE THRU APL , Anscombe, 1981, 426p.	\$39.00
Lots of statistics and APL. His workspaces are available from Yale University. A classic.	
MATHEMATICAL EXPERIMENTS ON THE COMPUTER , Grenander, 1982, 525p.	\$59.50
Case studies using APL in a number of fields, including statistics, linear algebra, geometry, asymptotics, neural networks, invariant curves. Detailed description and analysis of APL functions in all topics discussed.	
PROBABILITY IN APL , Alvord, 1984, 142p.	\$19.50
A delightful, yet serious development of 31 functions for fun and learning with roll, deal, binomial distributions, combinations, permutations, geometric distributions, and the World Series. Friendly examples. No previous APL required.	
APL - THE LANGUAGE AND ITS ACTUARIAL APPLICATIONS , de Kerf, Goovaerts, Stiers, 1987, 223p.	\$22.00
Introduction to APL. Loss reserves, credibility, probability, numerical analysis, forecasting, with APL functions.	
COMPUTER GRAPHICS FOR DESIGNERS AND ARTISTS , Kerlow and Rosebush, 1986, 298p.	\$44.95
A well organized, beautifully done reference book. Step-by-step explanations of terms and techniques from bits and bytes to 3-dimensional imaging. Many explanatory illustrations, most in color. No APL, but, says Rosebush, APL thinking and data structures are implicit throughout.	
APL2 AT A GLANCE , Brown, Pakin, Polivka, 1988, 444 p.	\$29.00
Solid, unimpeachable, introduction to APL2. Clear illustrations, modern exercises in each chapter. Gets you started.	
THE APL IDIOM LIST , Perlis and Rugaber, 1977, 50p.	\$ 6.50
Funded by Mobil. Written at Yale. Very terse, but includes nuggets of gold.	
APL - ADVANCED TECHNIQUES AND UTILITIES , Berquist, 1986, 450p.	\$44.95
Good discussion of alternative approaches to a wide range of programming tasks - efficiency, searching, dates, workspace documentation, file design, boolean techniques, financial utilities. Many idioms.	
Diskette for above	\$15.00
Contains all functions described in text. Formatted for APL*PLUS/PC, uploadable to Sharp APL, VS/APL, APL2.	
APL AS A TOOL OF THOUGHT, PROFESSIONAL DEVELOPMENT SEMINARS, NY/SIGAPL	
They cover a wide range of topics in business and education. 1987 - logic, insurance, statistics, A. I., accounting, fractals, linked lists. Other years: Teachers toolbox, computer science, biology, graphics, engineering, data bases, and more.	
1984, 1985, 1986, 1987, 120 to 150 pages each (1983, 47p. \$10.00)	each \$15.00
All five as a package	\$55.00

Name: _____	Date: _____	Please make check or money order payable in U.S. and send with this form to: Renaissance Data Systems P. O. Box 20023 Park West Finance Station New York, New York 10025-1510 U.S.A. (212)864-3078	
Street: _____			
City: _____	Prov./State: _____		
Country: _____	Postal Code: _____		
Telephone Day: _____	Evening: _____		
Title _____	Price _____	Quantity _____	Total _____
Title _____	Price _____	Quantity _____	Total _____
Title _____	Price _____	Quantity _____	Total _____
Subtotal: _____			
Postage and handling - U.S. and Canada (\$2.50 + \$2.50 each item over \$25.00): _____			
- International (surface mail add 15%, airmail add 50%): _____			
NYC Residents please add 8 1/4% sales tax, NY State, 4%: _____			
Please send complete CATALOG of 64 Titles: ☐ Total: _____			

British APL Association News

from Graham Parkhouse

We are very proud of Vector and are giving this issue to every delegate at APL89 in New York. While we are a national special interest group, everyone enthusiastic about APL knows that it is a language that bridges cultural barriers, both national and professional. Overseas membership of the British APL Association (BAA) brings with it four airmailed copies of Vector per year for about \$45.

Last week I was invited to a meeting of the British Airways APL Association (BAAA?) at Heathrow Airport for the presentation of the BAA's Outstanding Achievement Award. The demonstration of AIMS, the information management system which is the subject of the award, was impressive.

I enjoyed Maurice Jordan's talk which followed the presentation. He covered some of the things he had been up to recently, such as function point analysis and 'assertive comments', comments written in a slightly enhanced APL on characteristics of the data being manipulated. I gathered that these assertive comments could be evaluated for truth at the debugging stage, then were left in as comments, but not evaluated. Maurice said more time was spent debugging the assertive comments than the rest of the code, which might at first sight make them seem self-defeating, but to the contrary were worth every bit of trouble they caused because of the rigorous thought needed to get them right. I am looking forward to hearing more about assertive comments because I suspect Maurice is onto a significant development.

The British APL Association Special Achievement Award for 1989

The Special Achievement Award for 1989 has been given to Delia Hickson and Graeme Davison on behalf of their project teams at British Airways. The award was presented at a special meeting of the British Airways APL User Group on 6th June.



The award was given for a suite of APL executive information systems which are in daily use at British Airways. The BAA considers that these APL systems are exceptional in terms of their acceptance by Senior Board Members and for the importance of the information they provide. Within British Airways, these systems have achieved an enormous respect for APL and what it can do.

Delia and Graeme have generously donated the £500 prize to the Dhaka Orphanage Fund; a charity founded by a British Airways stewardess to build a new orphanage with facilities for 750 boys in Bangladesh. The BAA is delighted to see the money go to such a worthy cause.

Following the award presentation by Peter Donnelly, the audience were treated to a demonstration of the applications; AIMS (Airline Information Management System) and a suite of Forecasting programs.

A Demonstration of AIMS

The systems are designed to provide the top brass in British Airways with the sort of information they need to make executive decisions.

First we were shown the various facilities provided. These included charts showing bookings and forecasts broken down in every conceivable manner, actual results, operational statistics (eg the number of delays caused by catering hiccups), competitive information, ... in all a most impressive amount of data presented in a pleasing and informative manner. It was easy to understand how useful it must all be to the Airline's Senior Management.

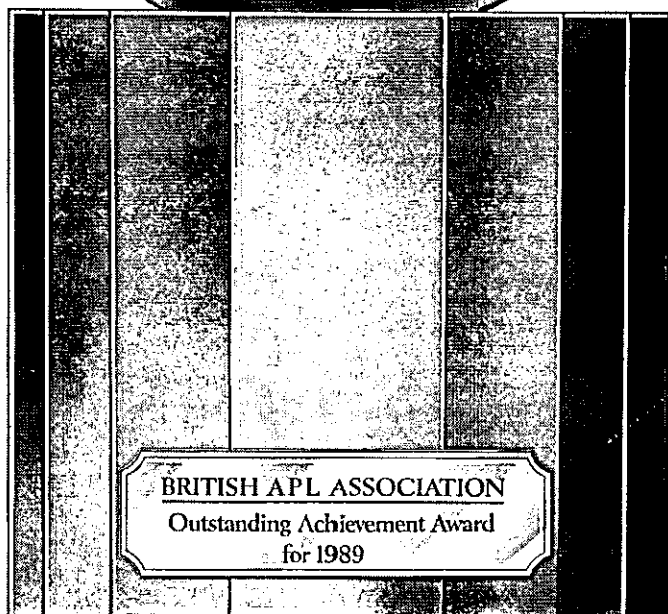
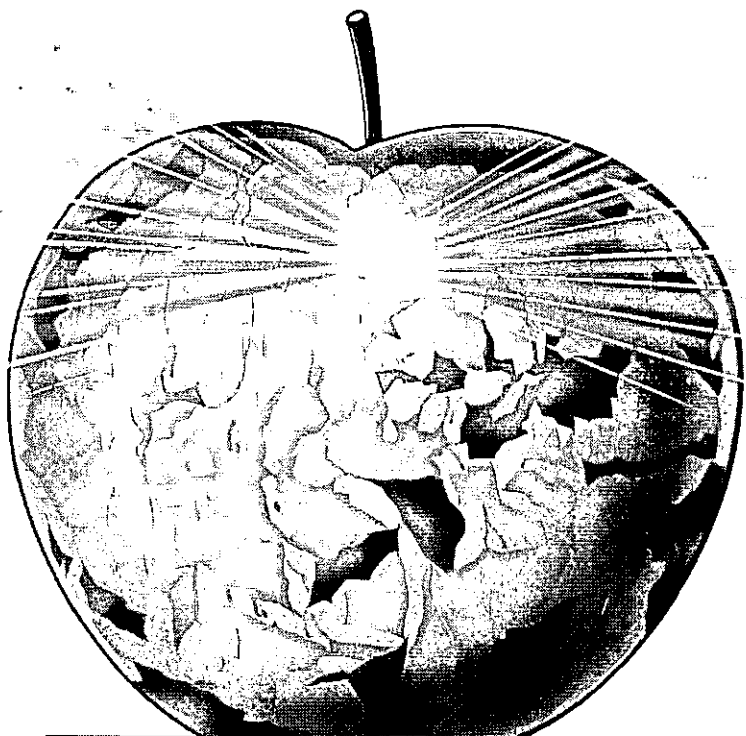
As evidence of the system's success, Keith Rapley (Senior Manager, Corporate Business Centre) described how they were used during two major crises during 1986, the American bombing of Libya and the Chernobyl disaster. Together these events caused an enormous upheaval in the world travel industry which jeopardised the survival of many of the major international airlines.

At the start of the crisis, bookings fell dramatically and cancellations rose, as tourists (particularly American tourists) decided that it would be safer to stay at home. The AIMS and Forecasting systems, which were normally updated weekly, were now consulted by anxious BA executives on a daily basis. The modifications that were necessary to meet this new demand for information were, naturally, completed in only a few days.

The result was that BA's executives had access to timely, accurate information from a system they had grown to trust which was presented in a (graphical) form they could readily understand. This enabled them to detect a recovery in the confidence of the travelling public long before their competitors. At the same time that other airlines were reducing capacity, BA launched a bold promotional campaign in North America designed to reinforce the recovery in confidence. As a result, BA came out of the crisis more or less unscathed (some other airlines suffered greatly) and was able to continue with its privatisation in February 1987.

A British Airways Director has said that 'Without a doubt, British Airways weathered the storm better than any other airline' and 'without (these systems) we'd have been lost since I'm sure that we'd have reacted like other airlines ... AIMS has been indispensable to running our business and to its privatisation'.

Keith Rapley concluded his talk by emphasising the importance of APL to British Airways and said that 'APL has already played a major role in the airline and will continue to do so for the next decade'. He went on to say 'Some of our junior members of staff occasionally question the use of APL ... I can assure you however that the management is committed to it and recognises APL's strategic importance in the development of present and future information systems for the airline'.



USE YOUR APL SKILLS WITH AN AWARD WINNING TEAM

British Airways has just won the APL Association Achievement Award for 1989 — a recognition of our contribution to the development across APL for use in worldwide telecommunications networks.

So what better time than now for you to be part of our innovative IT team at Heathrow?

By using advanced technology to collect, process and analyse complex marketing and financial data, you will make a real contribution to the commercial success of British Airways. This involves developing your APL skills on projects ranging from decision support systems for our international sales force to Executive Information Systems for our Company Directors.

ANALYST PROGRAMMER

£14-20K package

With at least 4 years' commercial computing experience, you will ideally have a mathematical or Operational Research background and be familiar with decision support systems. A knowledge of VM/CMS, C, PL/1, SQL or modular programming would be an advantage. Excellent problem-solving skills are essential. Ref: 181/TN/1599.

PROGRAMMER

£10-17K package

With at least 1 year's commercial computing experience, you will ideally have a mathematical or Operational Research background and a knowledge of VM/CMS, C, or PL/1. Ref: 182/TN/1599.

For both posts you will need to be self-motivated and able to work well on your own or as part of a team. There are outstanding opportunities to advance into team leadership, business analysis, or a specialised technical area.

The rewards, like the challenges, are substantial. Apart from a performance related salary, you will receive a profit share, holiday bonus, holiday travel opportunities, outstanding sports and social facilities — and the kind of career development opportunities that only a world leading IT operation can provide.

Interested? Then please write with a full c.v., quoting the appropriate reference number, to: Selection & Assessment, British Airways Plc, "Meadowbank", PO Box 59, Hounslow TW5 9QX.



News from Sustaining Members

Compiled by Alison Chatterton

HMW Computing Limited

HMW Computing Limited continue to develop "4XTRA" our front end Foreign Exchange Dealing and Position Keeping System. The current upgrades will make our product confirm to IBM's SAA, Common User Access Panel Design and Interaction and allow full support of mice and touch pads. Interest in "4XTRA" continues to grow and we have many UK and International Banks showing interest. We are currently installing the product in a Major UK Clearing Bank.

Our employment service for the APL community 'HMW EMPLOYMENT LTD.' continues to supply people with jobs. Currently we have many vacancies for the less experienced APL technicians. To find out what we can offer, please call Stan Wilkinson on 01-353-4212. If you would prefer to phone out of office hours Stan's home number is 01-286-7068.

APL People Limited

For those of you reading Vector for the very first time, let me introduce myself. My name is Jill Moss and I run a specialist employment agency for the APL Community.

If you are looking for skilled APL personnel, either on contract or for permanent vacancies, or are interested in finding out what opportunities are currently available - this is the place to come!

We are now in our eighth year of operation and have successfully placed people throughout the UK and Europe, including France, Belgium, Sweden and Luxembourg. Our next aim is to conquer the States! (We've already placed someone in Canada).



We also supply a consultancy service in APL, using our own highly-skilled professionals and supplementing their expertise with the use of top-quality freelance people. Our people offer a wide range of business skills, in addition to APL. For larger projects, we can supply a whole team of people, complete with Project Manager, to tackle the design, programming and implementation of major systems from start to finish.

Dyadic Systems Ltd

Dyadic is pleased to announce the launch at APL89 of Dyalog APL for 386 PCs under DOS. The product includes the new Dyalog APL screen manager using `□SM` (Screen Map) and `□SR` (Screen Read), which is proving extremely popular with users. The screen manager is the subject of a paper given at the Conference by John Scholes, head of the Dyalog APL development team.

In addition Dyalog APL for DOS contains the first release of a new development environment which Dyadic expects to be equally popular. APL has always offered an interactive environment in which programs can be developed and debugged very rapidly. However, the programmer interface has not kept abreast of modern developments, and even BASIC now provides a more interactive method of programming. The Dyalog APL development environment exploits the workstation environment of a bit-mapped colour screen, mouse and keyboard to provide a more convenient and powerful means of writing applications.

All APLs have you type an expression, then press a special key (usually ENTER or RETURN) to execute it. In Dyalog APL this idea is extended by adding keys that mean EDIT and TRACE (the actual keystrokes mapped to these functions are user-definable). Furthermore the EXECUTE, EDIT and TRACE keys can be mapped to buttons to give you a 'point and click' mouse interface.

If you type FOO and press <TRACE>, the function FOO will appear in a pop-up window on your screen, and your cursor will point to the first executable line which will be highlighted. If you now press <EXECUTE> the line will be executed and the cursor will move on to the next line. If instead you press <TRACE>, the line would be executed with trace on any defined functions and operators referenced on that line. For example, if the statement traced in FOO said `Z+AVERAGE Y`, a new window containing the function AVERAGE would pop-up overlapping the FOO window. You could then single-step through AVERAGE or trace deeper into your application as you wish. Execution can be restarted at any arbitrary line by simply moving the cursor to that line and pressing <EXECUTE> or <TRACE>.

The function on top of the stack (the APL stack AND the window stack) is always available for editing. Other functions and variables can be edited by pointing at their name and pressing <EDIT>. At any time you can have the SESSION window, the stack in several overlapping windows, and one or more edit windows, active. You can move between windows with a single keystroke or by moving the mouse, and data can be cut from one window and pasted into another.

By default, the debugger will activate automatically when an APL event occurs. The event message (e.g. DOMAIN ERROR) will be displayed as usual, but the stack will appear

with the top function in an editable window, and the cursor already placed on the line at which execution halted.

The interactive nature of APL and facilities like `⌈STOP` and `⌈TRACE` were originally crucial factors in establishing APL as a tool for rapid application development. To some extent, APL has fallen behind other software in this respect. The new Dyalog APL development environment is intended to redress the balance and bring APL once more to the forefront of software development technology.

I.P.Sharp Associates

Version 20 of Sharp APL is due to be released this autumn. The enhancements and new features in this version fall into 3 categories:

- Language extensions include new primitives for enclosed arrays (raze, permissive disclose) and data searching (in, nubsieve).
- APL system extensions include task specific names for 'continue' workspaces, access control for file libraries, file (as opposed to component) information, and an alternative character control system for non-APL terminals.
- Changes to the APL environment include full dumps of workspace libraries with Sharp APL up and running, and transaction recording for improved user statistics. In response to customer requests, a dedicated OPR1 terminal is no longer required to operate Sharp APL.

The APLX interpreter is now available for the Siemens BS2000 environment. This is based on version 19 of Sharp APL and includes Auxiliary processors for BS2000 commands, file and syslist processing, and full screen management. It replaces the first generation APL1 product.

Cocking and Drury Ltd

Release 2 of the APL*PLUS II system for 386 machines is due for international launch this August. The new facilities include the `⌈NA` interface to non-APL programs, a numeric full-screen editor, easier migration and a very useful 'point and edit' feature which lets you open an edit session for the name nearest to the cursor, anywhere on the screen.

The `⌈NA` facility can be used to call programs written in Microsoft C and Borland Turbo C in real mode, or MetaWare High C in the 32-bit protected mode. APL arrays can be

passed into and returned from C programs. The numeric editor is a very welcome facility. Each edit cell is a single number in the numeric matrix. Blocks of numbers can be tagged, moved, imported and exported.

For some time we have had an internal 2-day nested arrays course for mainframe and micro-based APL*PLUS systems. We will develop this into a product if there is sufficient interest from users of these systems.

Readers may be interested to hear that we have successfully migrated two internal Symphony applications into Oracle PC, with interfaces to APL*PLUS PC.

Cocking and Drury are pleased to announce the launch of our new FOCUS consultancy service. We are committed as a company to working with our clients in developing superior business support systems, and this new service gives recognition to the place that FOCUS plays in the portfolio of business support tools at our clients' disposal.

MicroAPL Ltd

MicroAPL is continuing to place an increased emphasis on consultancy activities and we still looking for suitably qualified programmers for this area of our business. APL work undertaken in recent months includes Securities analysis, local government contracts and real time management information systems. Non-APL work has included PC network consultancy and installation as well as the provision of systems written in C, Assembly Language and work with some of the modern database software for micros.

Our core business of APL software sales is running at a higher level than in this period last year. Of the various implementations of APL.68000, the Mac version (both for the Mac and Mac II systems) is now the biggest seller having overtaken the Atari version in the last few months. In addition we have supplied various Unix-based versions of APL.68000 for systems such as Wicat and NCR.

Following our release of APL.68000 version 7.20 for the Apple Macintosh version, we have upgraded a number of other APL.68000 implementations to version 7.20. Users of our Mirage-based machines such as the Spectrum and the Aurora will be pleased to hear that version 7.20 is available for their machines - with the most significant feature being the inclusion of the EDIT full screen editor into APL.68000 as a system function (and command). We have also started shipping version 7.20 for the Wicat MCS and Unix versions and will shortly be announcing a release date for the Commodore Amiga. Apple Mac users should note that we have made some minor improvements to version 7 and version 7.30 is now available for the Mac. This is available at no cost for users of version 7. Apply to MicroAPL or its distributors for release details.

MicroAPL will be exhibiting at APL 89 in conjunction with our US distributors, the Spencer Organisation of New Jersey, and we look forward to meeting some of our many US customers in person at the conference. Richard Nabavi will be presenting a paper on his work with Windowing interfaces for APL and David Eastwood will be discussing some of the real time work that our consultancy operations have been involved with.

MicroAPL for ...

APL on the widest range of micros

STSC's APL*PLUS Release 8

£425

STSC's APL*PLUS II PC

£1395

APL.68000

APL.68000 for the Mac, ST, Amiga

£86.91

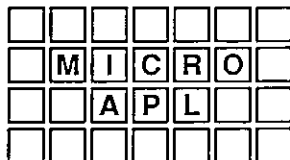
APL.68000 for the Mac II, SE30

£260.82

APL.68000 runs on many multi-user computers, including:

Sun, NCR, Wicat, HP200/300, Spectrum, Aurora, Stride £1200 to £2000

All prices quoted exclude VAT



MicroAPL Ltd.
South Bank Technopark
LONDON, SE1 6LN
Telephone: 01 922 8866

Access and Barclaycard
Orders accepted

VECTOR

THE EDUCATION VECTOR

JULY 1989

Edited by Alan Sykes

Reprinted from VECTOR Vol.6 No.1. VECTOR is the Quarterly Journal of the British APL Association; for more information about the British APL Association, contact the Secretary: Graham Parkhouse, Dept. of Mech. Eng., University of Surrey, GUILDFORD.

Contents

Editorial	Alan Sykes	24
APL Art	Alan Sykes	25
Let a child loose in APL!!	Richard Smith	27
The Latest I-APL News	Anthony Camacho	29
Numbers and Bases	Alan Sykes	31

Dr Alan Sykes,
Dept of Management Science and Statistics,
University College of Swansea,
Singleton Park, SWANSEA SA2 8PP

The Education Vector

by Alan Sykes

Welcome to Education Vector, the educational supplement to the quarterly journal produced by the British APL Association which is also mailed to all members on the (British) I-APL mailing list. For this edition we also welcome to our readership all attendees at APL '89, the international APL conference in New York in August.

I am glad to report that feedback to Education Vector is increasing and I welcome all contributions, comments and indeed criticisms. I thank those readers who (rightly) criticised some of my coding in past issues and I shall make every attempt to rectify my ways in future! (My use of APL tends to be very much one of getting the job done as soon as possible, and despite definite attempts to improve code presented in these columns, clearly I still haven't achieved the standards of professional APLers.)

In Education Vector (January 1989) I presented the question of how to turn a matrix of marks into grades. The solution I had in mind was duly presented by a number of APLers. This involved indexing the vector 'FEDCBA' according to the number of elements of the vector 70 60 50 40 35 0 that a mark was greater than or equal to. Thus the expression:

```
'FEDCBA' [⊖IO+1++/MARKMATRIX°.≥70 60 50 40 35 0]
```

provides the matrix of grades. (The ⊖IO is included to give complete generality to the expression whatever index origin the user is using - see this edition's article by me.)

I did however receive another solution which I had not anticipated and which is really preferable. Thanks to Mike Kingston (of IBM!) who pointed out that if we construct a vector of grades for each possible mark then all we need do is index this vector using MARKMATRIX. The easiest way to do this is to use / to replicate the required values: the vector of 100 grades is given by:

```
35 5 10 10 10 30/'FEDCBA'
```

Hence the alternative solution, with virtually no computation, is:

```
(35 5 10 10 10 30/'FEDCBA')[⊖IO+MARKMATRIX]
```

In this issue of Education Vector, the theme is 'using APL is fun'. Richard Smith (now aged 5) shows us just how much fun he has had at the APL keyboard. In 'APL ART' I throw open the challenge to readers to use their creative talents to produce pictures using APL characters. Finally in 'Numbers and Bases' we show how APL can be used in an investigative project on numbers and bases. The great beauty of such investigations is that everybody has something to offer - please share your gems of APL fun with us all by submitting comments, notes, or articles for inclusion in future editions. I hope that you enjoy the columns of this edition as much as I have enjoyed compiling them.

Puzzle Corner

QuestionWhat do you get if you multiply 6 by 9?

Answer42

Explain (ideally with an appropriate APL expression) in the light of 'Numbers and Bases' which you will find on page 31 of this Education Vector.

APL ART

APLers are often teased concerning their use of strange symbols. Curiously enough, even mathematicians are sometimes guilty of this! Whilst teaching students who use APL in their statistics practical I have sometimes caught them playing with the symbols to see just what shapes can be created from them. All good fun though at the time they had better things to do!

Recently I noticed that my wife Claire (who is both a mathematician and an APLer) was doodling with the APL characters, and I began to wonder just how many works of apart there are out there. So here is Claire's contribution to start the ball rolling. The program is called LEAR and it defines global variables OWL CAT and BOAT. So there are no prizes for guessing just what LEAR does. As you can see the results are quite good. (I have to say that!) But of course you, the reader can do better?

```

V RES←LEAR;O;C;B
[1] O←'  Λ ∨ ⊙ π '
[2] OWL← 6 5 pO[1 2 1 2 1 1 4 3 4 1 ,(15p5), 1 5 1 5]
[3] C←'  Λ O=1/ _ '
[4] CAT← 7 5 pC[1 2 1 2 1 1 3 1 3 1 1 4 5 4 1 ,(15p6), 7 7 7 7 6]
[5] B←'  _ \ / ~ '
[6] BOAT← 3 20 p(20 1 1 16 1 1 20)/B
[7] RES←((7 8 ↑CAT), 7 12 ↑ ~7 ~9 ↑OWL)↑BOAT
V

```

```

  Λ  Λ
  ○  ○
=1=
/////
/////
/////
____/

```

```

  Λ  Λ
  ⊙ ∨ ⊙
  π π π π π
  π π π π π
  π π π π π
  π  π

```

```

\
~~~~~
/

```

My contribution to APLART looks forward to the summer and the next international conference APL '89 to be held in Manhattan, New York. Since I have never been to the Big Apple before I am really excited at the thought of seeing the Empire State Building and other sites. So below is a function, cryptically called ESB, which requires a left argument equal to an integer, and a right argument requiring two positive integers. The result, for suitable choices, is something not dissimilar to part of the Manhattan skyline.

```

      ∇ R←A ESB N
[1]  R←'█'[1+(⌊N[2]⌋)◦.≤A×{(⌊N[1]⌋)-⌈N[1]⌉÷2}
      ∇

```

```

3 ESB 11 22

```

```

      █
      █
      █
    █ █
    █ █
    █ █
  █ █ █
  █ █ █
  █ █ █
█ █ █ █
█ █ █ █
█ █ █ █
█ █ █ █ █
█ █ █ █ █
█ █ █ █ █
█ █ █ █ █ █
█ █ █ █ █ █
█ █ █ █ █ █
█ █ █ █ █ █
█ █ █ █ █ █
█ █ █ █ █ █
█ █ █ █ █ █
█ █ █ █ █ █
█ █ █ █ █ █
█ █ █ █ █ █
█ █ █ █ █ █
█ █ █ █ █ █

```

If you are a newcomer to APL, then you may be a bit bemused as to how this function works. The best way to proceed is to assign say $N \leftarrow 11 \ 22$ and $A \leftarrow 3$. Then try looking at bits of the code. (Note the use of $|$ which takes the absolute value of its right argument.)

So may I exercise my privilege as editor of Education Vector and offer a prize for the most innovative APLART submission using a maximum of 10 lines of code? Please submit your contributions to the editor at the address at the foot of the front page.

Let a Child Loose in APL!!

by Richard Smith (when aged 3)

The following APL doodles are all the work of one Richard Smith, at the ages shown. They were done on APL*PLUS/PC, but with the I-APL keyboard layout to make it easier for him to identify the keys. (Reverse is much easier to hit if you don't need the Alt-combination).

They were snapped by a little function called ZAP which simply took a screen copy into a variable named (for example) ZZ041083 (1st birthday in origin-0); Richard was unsupervised for all the examples shown, except for father occasionally sneaking up on the keyboard to ZAP anything interesting.

Incidentally we don't have any APL labels on the keys, but we do have a Hercules+ card installed, so the APL symbols appear correctly on the screen.

----- Age: 2 years 8 months -----

```

      .
      ^
      .
SYNTAX ERROR
      .
      ^
      C
CAT    E
ELEPHANT
      8
      N→
SYNTAX ERROR
      N→
      ^
      N
NUT    H
HANNAH
      M, M
MUMMUM

```

----- Age: 2 years 8 months -----

```

4
10    4+6
      4+1
5      8+3
11     9×8×9
648    7+6
13     5×4
20     6+1
7      5×8
40     9×0
0      9×7
63     7×7

```

----- Age: 2 years 11 months -----

8/8
8 8 8 8 8 8 8
9/9
9 9 9 9 9 9 9
0/0

7/7
7 7 7 7 7 7
6/6
6 6 6 6 6
5/5
5 5 5 5
4/4
4 4 4 4
3/3
3 3 3
2/2
2 2
1/1
1

----- Age: 3 years 1 month -----

14x112
14 28 42 56 70 84 98 112 126 140 154 168
4x112
4 8 12 16 20 24 28 32 36 40 44 48
3x112
3 6 9 12 15 18 21 24 27 30 33 36
2x112
2 4 6 8 10 12 14 16 18 20 22 24
1x112
1 2 3 4 5 6 7 8 9 10 11 12
0x112
0 0 0 0 0 0 0 0 0 0 0
5x112
5 10 15 20 25 30 35 40 45 50 55 60

----- Age: 3 years 4 months -----

ΦUΦΦE
PU
TNAHPELE

ΦU Φ ΦE
ALLERBMU
EYE
ΦMΦΦUΦΦY
NAM
PU
THCAY

ΦM Φ ΦU Φ ΦY
PAM
REDNU
TRUHGoy

----- Age: 4 years 3 months -----

20 ρN
NO NO NO NO NO NO NO

20ρN
NET NET NET NET NET

20ρN
NIGHTSHIRT NIGHTSHIR

20ρN
NOSE NOSE NOSE NOSE

20ρN
NINCOMPOOP NINCOMPOO
21ρC
CLOUD CLOUD CLOUD CLO
22ρC
CASTLE CASTLE CASTLE C

34ρC
CRICKET CRICKET CRICKET CRICKET CR
55ρC

CLOUD CLOUD CLOUD CLOUD CLOUD CLOUD CLOUD CLOUD CLOUD C

The I-APL News

by Anthony Camacho

Problems

We are a volunteer organisation and do not expect to be able to match the service standards of the major vendors - but our customers sometimes expect us to! We do not spend money on publicity, relying on what we can get by sending letters and press releases to the magazines. When somebody gets a good letter in a major magazine then I get a flood of letters and spend a few evenings replying to them. When this happens the porter correspondence stops for a few days and APL gets tried by a lot more people.

For example David Piper's letter (p233 Personal Computer World May 89) produced about two dozen queries and, together with the details I send out, will have induced perhaps a quarter of them to try out I-APL. If you write letters to magazines or could do so please do more of it. The press coverage of APL has been better than ever in the U.K. for the last year or so. Let's keep it that way.

We caused ourselves considerable extra trouble by issuing the BBC version with two bugs in it. The second bug came to light the day after all the letters about how to cure the first bug had gone out and so I had to do almost the same again. The current versions have no known bugs. Everyone who we know bought a faulty version has been written to and anyone who has a faulty version may get it exchanged for the cost of the postage.

There are two mistakes we know about in the manual:

IAPL/E on the Archimedes does not send the APL font to an EPSON FX printer.

IAPL B is necessary on a BBC B or Master to get APL output (using EPSON grafrax compatible graphic output).

Progress

We are making steady sales and people are still working on ports to other machines. Ed Shaw and Rolf Levenbach are working together on the Apple II and Drs Ian Clark and Edwin Ashworth may do a Mac. Emmanuel Roche is working on the Amstrad word processors and Geoff Robinson on the Transputer (on a Bucon board which is a PC add-on).

We are still looking for porters. In particular we would like to find a porter or a group who would collaborate to produce a port for the Commodore CBM64. This is the most important of the machines common in schools that is not progressing as we would like.

We have received camera-ready copy for the second edition of 'An Encyclopedia of APL' by Garry Helzer. By the time you read this it will be in the hands of the printer. We have as good as sold out of the first edition of 1000 and as the second edition is laser printed and will be a better-looking product, easier to read, we think it worth doing a larger print run so that we can hold the price. Who, in 1986, would have thought I-APL would last long enough for us to be boasting that we have never had a price increase!

Among the many teachers who have bought a copy of I-APL, most of them BBC or Archimedes users, there are a few that have become enthusiasts for APL overnight as it were. We would very much like to help them. Last year the British APL Association said it was going to ask people who would be willing to help a teacher with APL to register and to make a list publicly available. I-APL is very keen on the idea. If you too are keen, and especially if you would be one of the volunteers, please let someone on the BAA committee know.

At the risk of being boring I repeat my appeal for teachers who use APL to write down anything they feel would be useful in school. If it is short we can put it in the Education Vector; if it is long we can publish it as a pamphlet. Either way other teachers trying APL for the first time may be grateful for the help.

Numbers and Bases

by Alan Sykes

Introduction

The vast majority of numeric procedures on a computer are done using numbers to the base 10. There are however occasions where one needs to represent numbers to different bases. APL offers facilities to change numbers from base 10 to other bases and vice versa. Although I have used APL for about six years, only recently have I become aware of how useful these facilities are. So this article attempts to explore numbers and bases and presents one or two unexpected applications.

Decode and Encode

Somewhere(!) on your APL keyboard you will find the two symbols `⍎` and `⍉`, called decode and encode respectively. Basically decode is used to decode a vector representing the digits of a representation to its conventional representation in base 10. Encode does the reverse operation. Here are a few examples to introduce them to you. First of all let's use encode.

Typing `⍉ 16 16 53` returns the answer `3 5`, corresponding to the fact that 53 to the base 16 equals $(3 \times 16) + 5$. (I picked on this number quite randomly without realizing that the digits become reversed - more about that later.) There are a few slightly odd facts about encode - first note that `⍉ 16 53` would return only the digit 5 - the other digit being lost. Alternatively one can type `⍉ 0 16 53` and that would have returned the vector `3 5`. As another example, `⍉ 16 16 500` returns the vector `1 15 4`, since 500 equals $(1 \times 16 \times 16) + (15 \times 16) + 4$, whereas `⍉ 0 16 500` gives `31 4`. So with encode you have to be a little careful how many numbers occur in the left argument.

Encode may also be used to convert numbers to multiple base systems. For example seconds can be converted into days, hours, minutes and seconds. Since 1 day 2 hours 21 minutes and 3 seconds equals 94863 seconds, the expression `⍉ 0 24 60 60 94863` gives the vector `1 2 21 3`. Here the use of 0 as the first value of the left argument may be used if no other unit of days (e.g. years) is required.

Encode can be used with a vector right argument - it returns a matrix with as many columns as elements in the vector, such that each column gives the base representation required. For example `⍉ 16 16 116` gives the 2 by 16 matrix:

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	0

If you require the usual hexadecimal representation for such numbers then index the vector '0123456789ABCDEF' on 1 plus the above matrix. If you prefer, you can transpose the resulting character matrix as follows:

```
Q'0123456789ABCDEF'[1+16 16T116]
01
02
03
04
05
06
07
08
09
0A
0B
0C
0D
0E
0F
10
```

So much for encode - experimentation is all you need to consolidate and that's where APL is on its own - it's so easy to experiment without actually writing programs.

What about the reverse procedure - decode. All you need to do is try similar examples but with arguments reversed. For example 16 13 5 gives the answer 53 as expected. The left argument of decode must be either one element or a vector of the same length as the right argument. Thus for the time example, we must type something like 0 24 60 6011 2 21 3 to get the value 94863. (Instead of 0 here, we could have put anything.)

If you wish to use decode with more than one set of values, then recalling that encode on a vector gave the results required in columns, it is not surprising that the sets of values required for decoding must be in columns.

For example, if we make a two column matrix of the values 1 2 21 3 using M+Q2 401 2 21 3 then 0 24 60 601M returns the two-element vector 94853 94863.

Encode and Decode have a number of surprising uses in areas which seem divorced from numbers and bases. We discuss some of these in the following sections. If there are applications which you have found useful which I do not mention - please write and tell me.

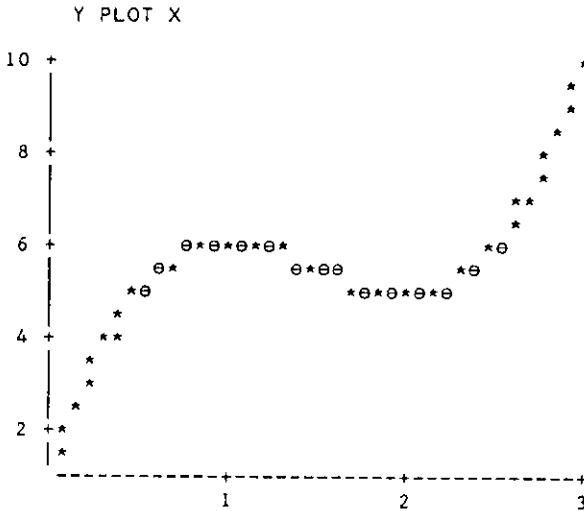
Evaluating Polynomials

Suppose that you wish to evaluate a quadratic or cubic expression in a variable X . Let's assume that values of $(2 \times X^3) + (-9 \times X^2) + (12 \times X) + 1$ are required. Then the expression `0 12 -9 12 1` evaluates the cubic at the point X equal to 0 giving the answer 1. Replacing 0 by 1, 2 or 3 gives values 6, 5 and 10 respectively.

If you wish to perform this operation for more than one X value at the same time, then the values must be supplied to the decode function as a one-column matrix. Take X values from .05 to 3 in intervals of length .05 by assigning `X←.05×160`. Now convert X into a 60 by 1 matrix and use with 1, assigning the values to Y :

```
Y←(60 1ρX)12 -9 12 1
```

Using a suitable PLOT function, we can plot the resulting curve.



... and the cubic (with turning points at 1 and 2) is sketched.

Searching For Values in a Matrix

APL provides the ability to locate a particular value in a vector using the iota symbol `⍲`. For example `1 10 3⍲10` gives the answer 2 as 10 is the second element of `1 10 3`. (Note that when the vector does not contain the target value, the value returned is one more than the length of the vector - for example `1 10 3⍲11` returns 4.

However iota cannot be used when the left argument is a matrix. So the way to proceed is to ravel the matrix into a vector, locate the position of the element as above, and then re-interpret the result in terms of rows and columns. Thus, suppose:

```

M+3 3p1 2 3 1 2 10 1 2 3
M
1 2 3
1 2 10
1 2 3

```

Using encode, we can find that 10 occurs in the second row and the third column as follows:

```

1 1+3 3T-1+(,M)110
1 3

```

Generating All Possible Outcomes

This is an application useful in elementary probability. Suppose that you toss a coin four times and require to describe the set of all possible outcomes of this experiment. Denoting 'heads' by 1 and 'tails' by 0, the set of sixteen possibilities is represented by the columns of the matrix resulting from:

```

M+2 2 2 2T0,115
M
0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1
0 0 0 0 1 1 1 0 0 0 0 1 1 1 1 1
0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1
0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1

```

(I should point out at this stage that this is one of the formulas that is simplified by changing the 'index origin' to zero, which can be done with the expression $\square 10+0$. Then the equivalent expression is $2\ 2\ 2\ 2T116$.)

Looking at the columns of the matrix, the first column represents the outcome that all tosses are tails, the second column where all are tails except the fourth toss, and so on. If we wanted to know how many outcomes produced two heads, then we have to sum the columns of the matrix ($+/\#M$) and equate to 2.

```

2=+/\#M
0 0 0 1 0 1 1 0 0 1 1 0 1 0 0 0

```

Hence:

```

+ / 2=+/\#M
6

```

or even better

```
      +/0 1 2 3 4°. = +/M
1 4 6 4 1
```

The values returned are of course the binomial coefficients representing the number of ways of choosing r objects ($r=0\ 1\ 2\ 3\ 4$) out of 4 without replacement. These can be generated directly from the keyboard:

```
      0 1 2 3 4!4
1 4 6 4 1
```

which is one line in Pascal's triangle which can also be generated as follows:

```
      0 1 2 3 4 5 6 7 8°. !0 1 2 3 4 5 6 7 8

1  1  1  1  1  1  1  1  1  1
0  1  2  3  4  5  6  7  8
0  0  1  3  6 10 15 21 28
0  0  0  1  4 10 20 35 56
0  0  0  0  1  5 15 35 70
0  0  0  0  0  1  6 21 56
0  0  0  0  0  0  1  7 28
0  0  0  0  0  0  0  1  8
0  0  0  0  0  0  0  0  1
```

(Of course I could write $x°. !x+0, 18$, which would be more efficient but less obvious.) There are all sorts of investigations that can be done with constructions of this type and the interested reader is referred to Linda Alvord's excellent book entitled 'Probability in APL'.

Cartesian Product

This is just the mathematician's name for what happens when one creates a new set of objects from two (or more) sets, by taking all possible pairs of objects, one from one set, and one from another. It plays a fundamental role in advanced Mathematics, and as we shall see, has its uses in computing as well. Suppose that the first set consists of the numbers 1 2 and the second set has the numbers 1 2 3. Then the possible pairs are (1 1) (1 2) (1 3) (2 1) (2 2) (2 3). These objects fall out neatly as columns of a matrix using encode as in the previous section.

```
      1+2 3T0 1 2 3 4 5
1 1 1 2 2 2
1 2 3 1 2 3
```

So we can write a cartesian product function CP as follows:

```

      V R+CP N; IO
[1]   IO+0
[2]   R+1+NTLx/N
      V
```

(I have made the formula simple by changing the index origin to zero. However, since I personally prefer to always work with index origin 1, I have made IO a local variable - so work carried out at the keyboard external to CP can still uses index origin 1.)

As an illustration, let's take the cartesian product of three sets having 2, 3 and 4 elements respectively.

```

      CP 2 3 4
1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
1 1 1 1 2 2 2 2 3 3 3 3 1 1 1 1 2 2 2 2 3 3 3
1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4
```

Applications of the Cartesian Product Function - Two Way Tables

I have recently made use of CP in a variety of different contexts, from designing experiments to constructing two-way tables and, to bring the discussion full circle, investigating properties of numbers to different bases! In the rest of the article I demonstrate some of these applications.

First of all it's back to the theme of my first two articles - that of counting. We have looked at deriving frequency tables of individual data vectors. What happens when we have two related variables and we wish to derive a two-way table? A simple example is sufficient to illustrate the argument.

Say X+1 2 1 1 2 1 and Y+1 3 3 2 2 1.

In a practical situation X might be the coded vector corresponding to individuals' heights and Y the coded vector for their weights. We wish to derive a two-way frequency table which tells us how many people have height coded to 1 and weight coded to 1 etc.

The two-way frequency table (a 2 by 3 table) corresponding to X and Y can be generated easily as follows. First of all, we need the cartesian product CP 2 3 equal to:

```

1 1 1 2 2 2
1 2 3 1 2 3
```

If we create a 6 by 2 matrix with X and Y as its columns:

```

      M←X,[1.5]Y
      M
1 1
2 3
1 3
1 2
2 2
1 1

```

Then we use something that looks like matrix multiplication but isn't. In APL we call it an inner product.

```

      M∧.=CP 2 3
1 0 0 0 0 0
0 0 0 0 0 1
0 0 1 0 0 0
0 1 0 0 0 0
0 0 0 0 1 0
1 0 0 0 0 0

```

The expression $\wedge. =$ ('and dot equals') works like matrix multiplication so that each row of M is paired with a column of CP 2 3, then 'equals' is applied to all the column and row pairs, followed by a reduction using 'and'. Thus a 1 appears in column i and row j of the resulting matrix if row i of M equals column j of CP 2 3. Hence the columns of the result count the frequency of occurrence of each possible pair of X and Y values.

All that is necessary now is to sum the columns of the binary matrix, and then re-shape the result as a 2 by 3 table:

```

      2 3p+∇M∧.=CP 2 3
2 1 1
0 1 1

```

With a little more coding, this approach can be made to produce a fully documented two-way table with headings - any offers? It also easily extends to producing three-way tables!

A Mathematical Investigation

My second example of using CP returns back to numbers to different bases and to my chance choice of 53 which when converted to base 16 results in the digits 3 5. So the question arises of how often this reversal of digits occurs when changing bases. I shall limit myself to considering only digits which are non-zero. How can we check systematically whether there are any more two digit numbers that behave like this. And what about three digit numbers?

If we denote the number to the base sixteen by ab , then a and b must be integers from 1 to 9 satisfying the equation $16a + b = 10b + a$ or $15a - 9b = 0$. To find all possible numbers, use CP 9 9 to generate all possible pairs of digits, evaluate $15a - 9b$, and search for zero values. The expression:

```
15 ^9+.xCP 9 9

6 ^3 ^12 ^21 ^30 ^39 ^48 ^57 ^66 21 12 3 ^6 ^15 ^24 ^33 ^42 ^51 36
27 18 9 0 ^9 ^18 ^27 ^36 51 42 33 24 15 6 ^3 ^12 ^21 66 57 48
39 30 21 12 3 ^6 81 72 63 54 45 36 27 18 9 96 87 78 69 60 51
42 33 24 111 102 93 84 75 66 57 48 39 126 117 108 99 90 81 72
63 54
```

yields a vector with only one zero result which occurs in the 23rd place. The 23rd column of CP 9 9 contains the digits 3 5 and hence we have solved the two-digit problem.

What about the three-digit problem? This can be tackled in a similar way by looking for integer solutions to an equation, and supplying all possible combinations of a, b , and c using CP 9 9 9. This time the equation is $255a + 6b - 99c = 0$. Hence the solutions, if any, are given by:

```
Y+CP 9 9 9
Y[(0=255 6 ^99+.xY)/19*3]

1
7
3
```

Thus the only three digit number with the desired property is 371 (base 10) or 173 (base 16). It is interesting that $371 = 7 \times 53$ but this does not help to find the four-digit number. Not to spoil your fun, let me throw open a challenge for readers to show me how they derived the four digit number with the required property of being, shall we say 'reversible'. Incidentally, despite my mathematics being somewhat rusty, I can prove that there are no 'reversible' numbers with five or more non-zero digits.

Such is mathematics, that once started on an investigation, you can't stop. All sorts of questions spring to mind. Have I destroyed possible patterns by excluding zero? What happens with different bases and so on? So here's my final challenge to the reader for this issue. For what bases (between 10 and 100 inclusive) are there two digit numbers whose digits are reversed when converted to base 16? The answer according to me is a good illustration of the innate stupidity of questions such as the following:

'What number comes next? 1,4,7,11,14,17 ...'

(Answer 21? No, 41 of course. Why 41? Because it is the next number which can be drawn using only straight lines!)

APL Product Guide

Compiled by Alison Chatterton

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry - see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- * Complete APL Systems (Hardware & Software)
- * APL Timesharing Services
- * APL Interpreters
- * APL Visual Display Units
- * APL character set printers
- * APL-based packages
- * APL Consultancy
- * APL Training Courses
- * Other services
- * Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'.

All contributions to the APL Product Guide should be sent to Alison Chatterton, at the address on the inside back cover.

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Dyadic	APL Coprocessor	3,500+	32-bit coprocessor board for IBM PC. NS32000 cpu with FPP, up to 4Mb RAM, 16Mb virtual memory. Software includes Unix V.2, Dyalog APL, graphics support, DOS interface. Provides multi-user Unix/DOS environment.
	IBM 6150	15,000+	Multi-user Dyalog APL system with Fast 32-bit RISC processor, FPP, up to 8Mb RAM, 210Mb Disk, 16 users. Interface to SQL, graphics and APL support for standard IBM peripherals.
	Altos 3068	25,000+	Multi-user Dyalog APL system with MC68020 cpu & MC68881 FPP. Also features a LAN which supports IBM PCs as Dyalog APL terminals.
	Sun 3	15,000+	Multi-user Dyalog APL systems which can be configured as a network of workstations and/or a traditional time-sharing cpu. With its 25MHz 68020 cpu, the Sun 3/200 is the fastest APL microcomputer on the market.
HMW	Powertools	poa	PCs, APL*PLUS/PC and Powertools supplied with bespoke turnkey systems.
Inner Product M.B.T.	IBM PC	2,000-6,000	IBM PCs supplied for turnkey applications
	MBT Series 10	poa	UNIX/68010 based multi-user APL system
	TORCH	poa	68000/Z80 multiprocessor
MetaTechnics	-	poa	Details on application -- IBM PC compatible
MicroAPL	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
	Spectrum	7,000+	Expandable multi-user APL computer using Motorola 68000. Std. configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
	Spectrum/386	2,000+	80386-based PC to run APL*PLUS II. Fully expandable.
	Atari 1040ST	799	1 Mb Mono/Colour System, includes 1 Mb disc drive & mains transformer built into Console.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES (£)	DETAILS
Mercia	APL*PLUS	poa	STSC's Mainframe Service - MAILBOX etc.
I.P. Sharp	SHARP APL	poa	International Network application systems and public databases.
Uniware	APL*PLUS	call	STSC's mainframe service

APL INTERPRETERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL Software	Dyalog APL	1,000-8,000	See Dyadic Systems entry
Cocking/Drury	APL*PLUS/PC Ref 8	475	STSC's full featured APL for IBM PC, PC/AT and PS/2. Now with virtual workspace, quad-MF, GSS graphics, Network and VGA support. Includes session ring editor.
	Run-time	poa	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL*PLUS II runtime	2,000	includes 1st year maintenance
	site licence	poa	
	APL*PLUS UNX	poa	STSC's 2nd gen hi-performance APL for IBM PC/RT, Sun, DEC, HP and major Unix computers and workstations.
	APL*PLUS VMS	poa	STSC's 2nd generation APL for DEC VAX computers running under VMS. Comparable in feature to APL*PLUS II system, APL*PLUS Unix, and APL*PLUS mainframe.

	APL*PLUS mainframe	poa	STSC's 2nd gen APL interpreter. Enhances VS APL with component files, nested arrays, formatting, user command processor, fastfns and other productivity features. For VM/CMS, MVS/TSO: offers simple upgrade from VS APL.
Dyadic	Dyalog APL	795-10,000	2nd gen. APL for UNIX systems, e.g. IBM 6150, Sun, Vax, NCR, HP9000, AT&T, Altos, Apollo, Whitechapel, Sperry, etc.
Gen. Software	APL*MYRIADE	poa	Runs on Texas Instruments TI990 range.
HMW	APL*PLUS/PC	poa	STSC's powerful APL Interpreter
IBM UK	IBM PC APL2	348	APL2 for the IBM PC. Program 5799-PGG/1, PRPQ number RJ-0411. From all IBM dealers.
	Mainframe APL2	poa	APL2 release 3
	APL2 Appl. Envt	poa	
Inner Product	VIZ:APL	250-350	8-bit Zilog Z-80 CP/M
	APL*PLUS/PC	600	See under Cocking & Drury
M.B.T.	Dyalog APL	poa	See Dyadic Systems entry
	MBTAPL	poa	Enhanced Dyalog APL for MBT hardware.
	VIZ:APL	poa	Customized for TORCH hardware
MetaTechnics	APL*PLUS Rel 6	475	Discount on quantity.
MicroAPL	APL68000	1,000-2,000	Full implementation with component files, error trapping etc. for SPECTRUM, HP300, SUN, NCR etc.
	QL/APL (keyword)	87	Full keyword APL for QL with many extra features.
	QL/APL (APL chars)	87	VSAPL compatible APL for QL with many extra features.
	APL68000 Apple Macintosh	87	
	APL68000 Apple Mac II	260	
	APL68000 Amiga	87	Full APL interpreters with support for windows, mouse, graphics etc.
	APL68000 for Atari ST	87	
	APL68000 for IBM-PC	995	
	APL*PLUS/PC - REL 8	425	
	APL*PLUS II	1,395	
Portable	PortAPL	\$195	IBM PC Software
		\$275	Mackintosh
		\$2,995	DEC VAX
I.P. Sharp	APL*PLUS/PC	480	STSC APL
	Upgrade V7 to V8	100	
	Upgrade any to V8	200	
	Sharp APL/PCX V1	1,650	For IBM XT/AT 370
	Sharp APL/PC V2	40	shareware for IBM PC/XT
	Sharp APL	poa	for IBM mainframes
Uniware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers.
	PortaAPL	280	PORTABLE SOFTWARE's APL for APPLE MACINTOSH.
		2,995	PORTABLE SOFTWARE's APL for the DEC VAX.

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	Lynwood j300	1,560	Monochrome ANSI 3.64 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.
	Lynwood j500	2,295	Colour ANSI 3.64 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.
	IBM 3163	791	Low-cost Monochrome APL vdu. Supports downloaded Dyalog APL font.
	IBM 3164	1,093	Low-cost Colour APL vdu. Supports downloaded Dyalog APL font.

Farnell	Tandberg TDV 2221	995	Ergonomic design APL terminal, 50-19200 baud, 1588if5 anti-reex screen, low profile keyboard
	Tandberg TDV 2271	1,195	Combined APL/ANSI ergonomic terminal as above.
Gen. Software	Mellordata	400	Second-hand Elite 3045A
M.B.T.	various		Contact MBT for details
Meta Technics	IBM EGA compatible	299	Emulates EGA & Hercules, Half Card
MicroAPL	Insight VDT-1	795	Inexpensive APL VDU
	Insight GDT-1	1,450	With monochrome graphics
	Concept201	1,295	APL VDU with 8 page memory
	Concept 201G	1,650	Graphics VDU
Shandell	HDS2010	1,215	ANSI3.64 DEC VT52/100/220 compatible. 15" tilt/swivel screen, low profile keyboard & page memory, windows, viewports, 80/132 columns, full overstrike, 2 or 3 comms, ports, 55 PF keys, NVM storage.
	HDS2010G/GX	1495+	As above plus Tektronix 4014, Retrographics VT640/D0640 and Visual 500 compatible. 1024 x 390 or 1024 x 780 resolution.
Tektronix	41148	13,500+	19" D.V.S.T.:Graphics: 3120 x 4096 displayable; Intelligent: up to 800K memory; APL keyboard (option 4E)
	4125	21,550+	19" 2D colour graphics; Workstation (1280 x 1024);Intelligent: up to 800K memory; APL keyboard (mod AP)
	4128	26,822+	As 4125 plus 3D wireframe

APL PRINTERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Datatrade	Datasouth DS180+	1,295	180 cps matrix printer with 4K buffer, 9 x 7 dot matrix and APL option.
	Datasouth DS220	1,695	Letter quality; graphics capability, APL option (both available with IBM Twinex or Coax interface).
Dyadic	IBM 4201 Proprinter	poa	100, 200, 40(nlq) cps, matrix printer, with graphics. Supports downloaded Dyalog APL font.
	Toshiba P351	poa	24 pin high-quality matrix printer 100 cps letter quality, 192 cps draft.
Inner Product	Epson FX80	500	Soft char. set, 160 cps, 80 column
	Anadex 9620	1,150	200 cps., 132 col., tractor feed
	Siemens PT88	620	180 cps., 80 col., silent
	TGC Starwriter	1,180	40 cps., letter quality
M.B.T.	Facit 4565	poa	40 cps letter-quality
	Facit 4510/11/12	poa	Matrix printers
MetaTechnics	Quen-data	295	Low-cost APL Daisy-wheel printer
MicroAPL	Datasouth DS180+	1,195	See Datatrade entry
	Philips GP480	2,137	Matrix printer with letter & draft quality and APL (480 cps).
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL-385	APL-385	50(PC),125(mf)	including ...
	FSM-385		Screen development
	DRAW-385		Screen design
	DB-385		Relational W.S.
	GEN-385		Miscellaneous Utilities
APL Software Ltd (mainframe)	AFM/AP	11,035	Interprocess Software for VM/CMS & MVS/TSO.
	- Keyed Access	2,650	Component File Management System (VSAPL/APL2)
	- Interactive Link	1,325	

	- Mail Exchange	2,650	
	CALL/AP	4,030	Non-APL program execution (VSAPL/APL2)
	APLPRINT	2,205	Output to high speed line printer or 328x devices (VSAPL/APL2)
	ENHANCED FORMAT	2,205	Extends Format operator to full "Quad-FMT" status (VSAPL/APL2)
	ISP	750	Input and Output Stack Processors for manipulating terminal I/O
	OSP	2,205	with facilities for Error Trapping (VSAPL)
	DISPLAY CAPTURE	poa	Allows terminal output to be collected and held for retrieval by an APL function (APL2)
	UCF	poa	User Communication Facility for data transfer between users (APL2)
	RDS	poa	Relation Data Base System
	PANEL	poa	Fullscreen management system
	PFS	poa	Program File System - APL Systems development aid
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beta-plan	BETA-FONT	poa	Multiple font PC character generator. Dealers required for non-Scandinavian countries.
APL IMPETUS	Impetus	poa	Hierarchical Planning System
Butel	Merlin	5,000	Mainframe APL spreadsheet runs under VM/CMS, TSO, VSPC
	Merlin/PC	poa	Version for APL*PLUS/PC
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad- functions & nested arrays for IBM VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	First APL compiler. Available with APL*PLUS enhancements and Sharefile under VM/CMS & MVS/TSO
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Convert non-APL files to APL files
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Powerful Information Centre spreadsheet incorporating data exchange between APL and FOCUS, IFPS, SAS, APL/DI, ADRSII, LOTUS123, VISICALC, MULTIPLAN, DIF files
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(microcomputer)	STATGRAPHICS Rel 2.6	645	Integrated Statistics and graphics on IBM PC, AT, PS/2 and compatibles
	Update 2 to 2.6	140	
	Update 1 to 2.6	330	
	APL*PLUS PC Tools	325	Utilities including: RAM disk, full screen data entry, menu input, report generation, file documentor, exception handling and games.
	IRMA Module	120	327x IRMA support.
	Fin & Stat. Library	350	Financial & statistical routines
	SPREADSHEET MGR	195	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII, LOTUS
E&S	PROTOPAK		Packages for prototyping management information systems - consisting of: PC & mainframe modules

	RMS		Relational databases.
	AMS		Multi-dimensional arrays
	RAMS		Combined RMS & AMS
	BMS		Dynamic nancial modelling & forecasting
	FMS		Full-screen handler for APL*PLUS/PC. (AP124-based)
	CMS		Communications package.
	SOS	poa	Scheduled ordering and stock control.
FASTCODE	FASTCODE, MONITOR	\$239	Assembler written monitor for APL applications in APL*PLUS/PC
Gen. Software	PROPS	500+	Spreadsheet system for Product and/or Project Planning.
H.M.W.	INPUT	poa	Matrix manipulation package for data entry & report generation
	POWERTOOLS	poa	For faster primitive screen handling, forms validation (SAA / CUA conforming). Includes support for mice, touch-pads etc.
	PRINTPAK	poa	Block printing for VM/CMS
	VIEWPAK	poa	AP124 Protocol emulator for IBM/PC
	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
Inner Product	Viewcom	150	Control Viewdata from APL
	APL/dBASE II	150	Interface APL with dBase II
	APL/dBASE III	150	Interface APL with dBase III
	APL/LOTUS	150	Interface APL with Lotus
	APL/WORDSTAR	150	Interface APL with Wordstar
	APL/MULTIPLAN	150	Interface APL with spreadsheet
	CEMAS	3,500	EEC monetary and agrimonetary analysis.
M.B.T.	RHOMBUS	poa	Integrated Ofsu3suce System
	HASLEMERE	poa	Hotel Accounting System
Mercia	STATGRAPHICS 3.0	685	Integrated stat. graphic system for PCs.
	Upgrade 2.1 to 3.0	395	
	Upgrade 2.6 to 3.0	175	
	MICROSPAN	250	Comprehensive APL tutor
	LOGOL	poa	Logistics management system for PC, Forecasting, Inventory Control, Scheduling, Distribution, etc
MetaTechnics	MetaScreen	99	Full-screen handler for APL*PLUS/PC, based on VSAPL AP124
	MetaPack	495	Comprehensive utilities package for APL*PLUS/PC. Includes: MetaScreen, MetaWS, Browse, Toolbox, Numeric Editor.
	APL-IEEE488	99	Controls IEEE488/GPIB Bus from APL*PLUS/PC.
	PLOT/PC	99	2D & 3D Graphics package. Includes interactive diagram Editors.
	Browse	99	Scrolling of DOS files, large APL variables.
	ADAPTA DLS	poa	Production & purchasing scheduling for process manufacturing.
	ADAPTA MSP	poa	Job-shop loading & scheduling for multi-stage production.
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product

	MicroSPAN/PC	250	For APL*PLUS/PC product
	PC TOOLS Vol 1	295	
	STATGRAPHICS Rel 1	495	
	STATGRAPHICS Rel 2	535	
Parallax	ExecuCalc	\$5,000	Mainframe-based electronic spreadsheet for VM/CMS & MVS/TSO with links to micro products.
	ExecuPlot	\$5,000	Mainframe-based colour graphics with micro links.
I.P. Sharp	GLOBAL LIMITS	poa	Exposure management for banks
	IPSA/CONNECT	poa	Mainframe to micro link
	MAILBOX	poa	Electronic Mail
	MAILBOX/PC V 2	65	Full screen front end to IPSA mailbox
	upgrade to V 2	15	
	NEWSFLASH	poa	Real time message exchange
	VIEWPOINT	poa	4GL - Info centre product
Sugar Mill	Stat 1	\$129.95	Statistical toolbox, menu driven
Uniware (mainframe)	STSC's ENHANCEMENTS	10,715	Quad-functions & nested arrays for IBM VSAPL under VM/CMS and MVS/TSO
	STSC's SHAREFILE	10,715	Component files for IBM VSAPL under VM/CMS and MVS/TSO and for IBM APL2
	PROGRAMMER TOOLS & UTILITIES	5,715	
	FILEPRINT	5,715	
	FILESORT	5,715	
	FILECONVERT	5,715	
	FILEMANAGER (EMMA)	5,715	STSC's database package.
	APL*PLUS COMPILER	21,430	First APL compiler. Complements APL*PLUS enhancements and Sharefile under VM/CMS and MVS/TSO.
	EXECUCALC	3,995	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL (VM or TSO).
(microcomputer)	STATGRAPHICS	725	Statistics & Graphics for PCs.
	STATGRAPHICS FCA	140	An add-on module to STATGRAPHICS: Factorial Correspondence Analysis.
	APL*PLUS/PC TOOLS - VOL1	325	Incl. 327 x iRMA support, RAM disk, full screen data entry, menu input, report generation, games.
	- VOL2	125	Incl. File documentor, screen editor, exception handling.
	SPREADSHEET MNGR	250	APL spreadsheet with built-in ASCII, LOTUS and SYMPHONY interfaces.
	APL*PLUS/PC FIN. & STAT.LIBRARY	350	Collection of financial and statistical utilities.
	POCKET APL	140	Smaller version of APL*PLUS/PC.
	UNIASM	275	Collection of assembler routines for APL*PLUS/PC users.
	UNITAB	240	APL*PLUS/PC spreadsheet-like data entry and validation system.
	The APL DEBUGGER	105	First released APL*PLUS/PC debugger.
	OVERLAYS	250	Fast assembler routines to handle overlays in APL*PLUS/PC.
	R:BRIDGE	380	Interface between APL*PLUS/PC & R:BASE 5000.
	DMA	380	A version of EMMA (APL database manager) for APL*PLUS/PC users.
	APL2C	295	Interface between APL*PLUS/PC and DATALIGHT C language.
	ADAPTA/DLS	33,333	Production & purchasing scheduling for process manufacturing.

Warwick University	BATS	500	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to 35 for academic institutions.
	FAB	free	Training program for the above.

APL CONSULTANCY

(prices quoted are per day unless otherwise marked)

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL Consulting	Consultancy	180-550	Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation.
APL People	Consultancy	from 120	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL68000, C/Unix, TSO/MVS, VM/CMS, graphics. Additional skills available: OR, statistics, financial, commerce, retail, engineering.
APL Software Technology	Consultancy	poa	Technical & business systems, micros, networking & communications a speciality.
Buckland Management Systems	Consultancy	poa	Business and Technical systems in commerce and industry. Designing, programming and implementing applications of O.R. and statistics.
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	150-300	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	175-250	Junior consultant
		250-350	Consultant
		300-450	Senior consultant
		350-600	Principal Consultant
		400-750	Managing consultant
Peter Cyriax	Consultancy	100-150	Junior Consultant
		120-200	Consultant
		160-300	Senior Consultant
Delphi	Consultancy	poa	Specialising in management reporting systems and APL on microcomputers.
Dyadic	Consultancy	poa	APL system design, consultancy, programming & training for Dyalog APL, VSAPL, APL*PLUS, IPSA APL etc.
E & S	Consultancy	150-250	System prototyping: all types of information system.
FASTCODE	Consultancy	poa	Specialise in improving performance of APL applications on micros & mainframes.
Gen. Software	Consultancy	100+	System design consultancy, programming. HMW specialize in banking and prototyping work.
H.M.W.	Consultancy	poa	
Inner Product	Consultancy	200	On-site micro-mainframe APL, PC/DOS & Assembler
Lloyd Savage	Consultancy	poa	Decision support, particularly specialising in Sales & Marketing systems.
M.B.T.	Consultancy	poa	APL*PLUS & VSAPL consultancy.
Mercia	Consultancy	poa	
MicroAPL	Consultancy	poa	Technical & applications consultancy.
M.T.I.	Consultancy	poa	Specialise in Maintenance and development of existing APL systems
Parallax	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	200	Specialising in Banking, Financial & Planning Systems.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
I.P. Sharp	Consultancy	poa	Consultancy & support service world-wide.
Uniware	Consultancy	call	Junior to managing consultancy in APL.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
I.P. Sharp	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APL Consulting	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
	Tim Perry	Jubilee House, Chapel Rd, HOUNSLOW TW3 1XT. Tel: 01-577 3541
APL Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
APL People	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
APL Software	David Alis	Jubilee House, Chapel Rd, HOUNSLOW TW3 1XT Tel: 01-577 3541
APL Software Technology	John Hagger	14 Rosewood Avenue, Alveston, Bristol BS12 2PP Tel: 0454415737
	Per Hultin	46 Vicarage Road, South Benfleet, Essex SS7 1PB Tel: 0374550501
Beta-plan APS	Kim Andreasen	Stengrade 75, DK-3000 Helsingør, Denmark. Tel: 45 2 21 48 48
Buckland Management Systems	John Buckland	Westwood, 9 Grove Road, Camberley, Surrey, GU15 2DN Tel: 0276 684327
Butel Technology Ltd.	Mike Munro	Butel House, 3 Great West Rd., London W4 5QJ Tel: 01-995-1433
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. Tel: St. Albans (0727) 60130
Paul Chapman		18, Trevelyan Road, London, SW17 9LN Tel: 01-767 4254
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE16 Tel: 01436 9481
Datatrade Ltd.	Tony Checkseid	38 Billing Road, Northampton, NN1 5DQ. Tel: 0604-22289
Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 03677-384
Dyadic Systems Ltd.	Peter Donnelly	Park House, The High Street, Alton, Hampshire. Tel: 0420-87024
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-24741
Farnell International Instruments Ltd.	R. Fairbairn	Jubilee House, Sandbeck Way, Wetherby, W. Yorks. Tel: 0937-61961
	Roger Attard	Davenport House, Bowers Way, Harpenden, Herts. Tel: 05827-69071
FASTCODE	Andrew Dickey	P.O. Box 281, Croton-on-Hudson, New York 10520, U.S.A. Tel: (914) 271-3200
General Software Ltd.	M.E. Martin	22 Russell Road, Northolt, Middx. UB5 4QS. Tel: 01-864 9537
H.M.W. Computing Ltd.	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, LONDON EC4Y 0HA Tel: 01-353 4212 Telex: 926604 HAMHSEG Fax: 01-353 3325
IBM UK Ltd	Nat.Enq. Centre	414 Chiswick High Rd, London W4 5TF Tel: 01-747 0747
Inner Product Ltd.	Dominic Murphy	Eagle House, 73 Clapham Common Southside, London SW4 9DG. Tel: 01-673 3354
Lloyd Savage Ltd	Philip Johnson	Cambridge House, Oxford Road, Uxbridge, Middx, UB8 2UD. Tel: 0895-59826
Mercia Software Ltd.	Gareth Brentnall	Aston Science Park, Love Lane, Birmingham B7 4BJ. Tel: 021-359 5095

MetaTechnics Systems	John Stenbridge	Unit 216, 62 Tritton Road, London, SE21 8DE Tel: 01-670 7959
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 01-922 8866
M.T.I.	Ray Cannon	7 Pine Wood, Sunbury-on-Thames, Middx. TW16 6SH Tel: 09327 80848
Parallax Systems Inc.	Kevin Weaver	60 West 9th Street, New York, New York 10011, U.S.A. Tel: 212-475-4001
Peter Cyriax Systems	Peter Cyriax	213 Goldhurst Terrace, London NW6 3ER Tel: 01-624 7013 (Answerphone) 0860-377963 (Mobile)
Portable Software	Richard Smith	60 Aberdeen Ave, Cambridge, Mass. U.S.A. 02138 Tel: 617-547-2918
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd Camberley, Surrey, GU15 1LB Tel: 0276-20789
The Rochester Group	Robert Pullman	164 Pinnacle Rd., Rochester NY 14620 Tel: 716-461-3169
Shandell Systems Ltd.	Maurice Shanahan	12 High Street, Chalfont St. Giles, Bucks HP8 4QA. Tel: 02407-2027
I.P.S.A. Ltd.	Mike Butler	7th Floor B Block, Coventry Point, Market Way, Coventry CV1 1EA Tel: 0203 256562
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kika Place, Kailua, Hawaii 96734 Tel: (808) 261-7536
Tektronix UK Ltd.	Paul Morgan	Fourth Avenue, Globe Park, Marlow, Bucks SL7 1YD. Tel: 06284-6000
Uniware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France Tel: (1) 45-27-20-61 Telex: 648348F UNIWARE
Warwick Univ.	Dr Andy Pole	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel: 0203-523066 x2158

Reviews Section

introduction by Adrian Smith

Thanks to Jonathan Barman, Anthony Camacho and Jonathan Martin for completing three excellent reviews to a very tight schedule.

We have a much fuller technical review of the IBM APL2/PC product which will be carried in the next VECTOR. Unfortunately the twin constraints of space and time conspired to keep it out of this issue. Just for fun, here is a selection of the infamous 'Smith Benchmarks' as first seen in VECTOR Vol.1 No.4 page 55:

BENCHMARK -----	APL*PLUS/PC V.8 -----	APL2/PC 32-BIT -----
□WA	220K	3.6M
+ / V + V	60	<50
+ / V * 2	350	160
V + . x , M	220	110
+ / S = ' T '	550	490
X + S € ' F A T '	550	1260
X + B ∨ B ^ ~ B	770	<50
FN 1000	5200	6810

The tests were run on a PS/2 Model-70 (16 MHz) with 4Mbyte of memory and a maths co-processor fitted.

dyalog APL

C INTERFACE

```
#include "apl.h"
#include "support.h"

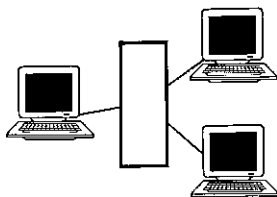
main()
{
    ARRAY *rslt, *larg, *rarg;
    *getarray(), *fre
    *idn_d(), *d_idn
    int incode;

    startup(2);
    define("IDN_TO_DATE", RSI
    define("DATE_TO_IDN", RSI

    larg = rarg = rslt = 0;
    setjmp(eirbuff);
    while(1)
    {
        larg = freearray;
        rarg = freearray;
```

UNIX INTERFACE

MULTI-USER



CHOICE OF HARDWARE

IBM Ps/2 Model 80
Compaq 386
Altos 3068
Apollo
AT&T 3B2
Bull SPS7, SPS9
Dec Vax
Hewlett-Packard HP9000
IBM 6150
Masscomp
NCR Tower 32
Pyramid
Sun
Unisys Series 5000

FILE SYSTEMS

EDITORS

```
[0] Z←DISPLAY Z;
[1] ⚡IO←0 ⚡SZ←*
[2] Z←' ';(⚡FMT
[3] NSS:→(1 ⚡DZ
[4] NOR:Z←(1⚡ρZ)ρ
[5] →(1=DZ)/SMP
[6] Z←' ',(QTV 1
[7] SMP:QB←(2⚡0 '
[8] FMT:Z←(FMT Z
[9] BOX:R C←1+ρZ
[10] QTV←1+1/2 SZ
```

FULL-SCREEN INPUT

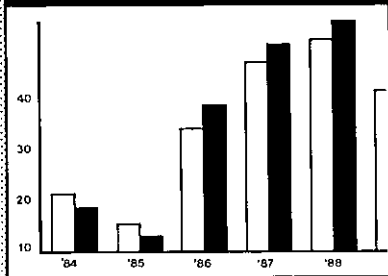
Share Name

Number of Shares

Unit Cost

Buyer

GRAPHICS INTERFACE



SQL INTERFACE

EMPNO	ENAME	JOB	MGR
7329	SMITH	CLERK	7902
7499	ALLEN	SALESMAN	7698
7521	WARD	SALESMAN	7698
7566	JONES	MANAGER	7839
7654	MARTIN	SALESMAN	7698
7698	BLAKE	MANAGER	7830
7782	CLARK	MANAGER	7839
7788	SCOTT	ANALYST	7566
7839	KING	PRESIDENT	
7844	TURNER	SALESMAN	7698
7876	ADAMS	CLERK	7788

DEVICE SUPPORT

NESTED ARRAYS

DISPLAY REPORT

	North	South	East
Cakes	14	76	46
Buns	22	5	68

...gives you all this – and APL too!

For further information, contact
Dyadic Systems Limited
Park House, The High Street, Alton, Hampshire, GU34 1EN
Telephone: (0420) 87024. Telex: 858811

The APL Toolkit, A Collection of APL Functions (Second Edition)
Canadian Information Processing Society, APL SIG.
Edited by Richard Levine

reviewed by Jonathan Barman

The second edition is an enormous improvement over the first. It now contains a substantial number of APL functions which every APL programmer needs to have available when building systems. Newcomers to APL will find the book invaluable. Those who have been programming for some time will already have created most of the functions, but may find some useful items to add to their tool chest.

About 150 functions are included, most of them very short utilities, but there are some longer ones which are more general purpose. The functions are grouped into sensible classifications, and each function has an excellent explanation of how it should be used. The functions and explanations are in separate sections, each with its own index. Those who want to use the functions will probably want to read the explanations without looking at the code, and will approve of this arrangement into separate sections. It was, however, most aggravating to have to keep flipping between the sections to read the code and the explanation together while reviewing the Toolkit.

The quality of the functions is patchy, which indicates the wide range of sources for the code. The introduction states that 'Optimal efficiency was not a requirement for initial inclusion in the collection, although many of the functions are efficient.' On the contrary, a few of the functions are efficient but many of them are very inefficient. No indication is given in the description of the relative efficiency of the functions, nor is there an indication that there may be an alternative way of carrying out the same task. For example, the function `FIRST` contains $Y \leftarrow (X \uparrow X) = 1 \rho X$ and $Y \leftarrow v / < \backslash X \wedge . = \emptyset X$ for the vector and matrix case respectively, but there is no indication that these algorithms work well for small arrays but are very inefficient for large arrays. The alternative sorting algorithm suitable for large arrays is not mentioned. The methods for writing efficient APL code have been published, in particular Roy Sykes's Whizbangs and Gary Berquist's Advanced Techniques and Utilities, so there should be little excuse for missing out this aspect.

There are 24 categories of functions, covering catenation, date manipulation, formatting, graphics, input, output, programming, searching, sorting, text editing and timing. Each category has a brief description of its contents in the introduction, and there is a listing by category of the header and first comment line of each function. The indexes for the function section and the explanations section are both by category and alphabetically by function name.

The coverage is quite good, with date manipulation, searching and sorting being well represented. The most notable omission is the formatting of numbers. The 'formatting' category supplies functions to help with writing reports, left/right justification, centering, column headings etc., but nothing for adding the usual decorations to numbers such as commas or brackets round negative numbers - usually a substantial number of functions are required for VSAPL users who do not have `□FMT` or the extended formatting capabilities in APL2. VSAPL comes with a formatting workspace, so perhaps it was assumed that everyone has access to suitable numeric formatting functions.

A set of skeleton output functions is provided which are designed to be used in a consistent way to output reports for an application. There is a good explanation (in an appendix) of how sets of device-dependent functions can be bolted on as required, and suggestions are made for various extensions which would be necessary to make them work properly. Comments are included such as 'CLOSE PRINTER AS REQUIRED (INSERT CODE HERE)', but the sections relating to files are not well commented, and people who are not familiar with shared variables could find this aspect quite difficult to write. It would have been useful to have had a worked example in a particular flavour of APL.

The original collection of functions for the Toolkit was made in the early 80's, when the majority of APL was carried out with typewriter style terminals on a timesharing bureau. The input, text editing, and graphics functions have been included in the second edition with little change, and are still geared for typewriter terminals. It was quite a nostalgic experience to see them again, and to remember what it was like without today's full-screen capabilities. Utilities for full-screen work are entirely absent, understandably perhaps as each of the flavours of APL provide such different facilities that it is difficult to provide a coherent set of useful functions, particularly ISO standard-conforming functions.

A few APL idioms and boolean techniques are included in an appendix. The reader is referred to the Finnapl Idiom Library and to Robert Smith's Boolean Functions and Techniques, which presumably provided the source for all the examples given.

There is a workspace associated with the Toolkit, but it is not being distributed by the CIPS APL SIG. It is not clear what is in the workspace, or whether the explanations are included as well as the functions. Also the flavour of APL, or the medium on which it is available, is not stated. Perhaps I-APL should get into the act!

The manual is spiral-bound in flimsy covers, and will get rather tatty after a little use. The function explanations are all in APL upper-case script, which is not very easy on the eye. The introduction is in a pleasant proportionally spaced font, but there appeared to be some difficulty mixing APL with ordinary text: a right arrow came out as a splodge.

Conclusion

The stated aim of the Toolkit is to provide a set of short functions - no more than a page in length - which provide the building blocks for APL systems, together with some 'fun' functions which illustrate APL techniques. On the whole this aim has succeeded, but more attention should have been paid to efficiency. Many more functions could have been included, but it is quite difficult to get people to come forward with public domain items and the CIPS APL group has really done very well.

This will be a useful book for the majority of APL programmers, and a buy can definitely be recommended.

CIPS APL SIG - Toronto
Attention: Toolkit Editor,
P.O. Box 384, Adelaide St Station,
Toronto, Ontario, Canada.

Price \$25.00 US (including postage) to addresses outside Canada

Appendix: A Short Review of Each Section

Catenation - 2 functions

Two functions for catenating different sized arrays along the first and second dimensions of a matrix. The padding is always out in the same direction as a positive take, and there are no functions given for padding in the opposite direction.

Computation - 8 functions

Three useful rounding functions, including distributive rounding. A 'THRU' function (TO in English) takes an additional argument of the step value, which must be supplied. A simpler function could have been included, so that one could specify indices such as '4 TO 8, 12 TO 14'. The SUBTOTAL function exhibits an interesting way of merging subtotals into the matrix of values. The remaining functions are not particularly useful.

Dates - 15 functions

A fairly comprehensive set of functions, but they need converting into the day, month, year format used in the UK. No functions are given for weekdays, nor for manipulating the months and years excluding the days.

Delete characters - 8 functions

The normal set of functions for deleting leading, trailing and surplus blanks, plus some for deleting specified characters.

Delete elements - 2 functions

The duplicate deletion function is inefficient on large arrays, with no mention of the alternative sorting algorithm.

Expansion - 2 functions

A useful pair of functions for generating expansion vectors. Inefficient on large arrays.

Formatting - 15 functions.

This section could have been split into three: report formatting, text formatting and function formatting.

Report formatting functions include the normal ones for left justifying, right justifying and centering text, and a couple of functions for generating column headings.

Text formatting functions include ones to split long lines of text and for displaying long thin matrices in columns across the page, though not ones for splitting the matrix into pages as well. The `REFORMAT` function is extraordinarily inefficient and could only be used on quite short paragraphs.

There are two function formatting functions, and one to sort the local variables on the header line.

There are no functions to help in the formatting of numbers.

Graphics - 4 functions

Simple functions designed for typewriter terminals.

Input - 1 function

Simple quad-quote input, for typewriter terminals.

Library-utility - 14 functions.

The functions used to produce the manual, and to help to search for and maintain the standard comments included in each function.

Output - 8 functions

A skeleton set of functions to standardise the output of reports to a printer, terminal or file. Useful as a basis, but a fair amount of work would be needed to turn them into a proper working set of utilities.

Plotting - 7 functions.

Simple functions designed for typewriter terminals.

Programming - 4 functions.

A gluing function (Δ) and an IF function. No UNLESS function.

Programming tools - 14 functions.

Functions for setting stop and trace bits (using SA and TA), for listing functions, for extracting global names, for analysing APL statements, and for data types and sizes of variables. The function for extracting global names is written by Roger Hui and is excellent. The functions for data types and sizes are VSAPL based. There is no relabel function.

Reshape - 11 functions.

The most useful functions are for turning delimited vectors into matrices (ΔBOX) and back again ($UNBOX$). The functions are unusual in that the pad character can be specified. A generalised delimited vector to array function is also available. The remaining functions carry out various simple replications of arrays, but there is no function to replace the primitive replicate which is available in all the more recent versions of APL.

Searching - 12 functions.

Contains functions for string searching, flagging unique elements in a vector or matrix, membership applied to matrices, and a couple of longer functions for picking out rows in a matrix depending on a specification vector - useful for displaying selected functions or variables in a large workspace. The string search functions do not use any of Roy Sykes's techniques for speeding up the search, but would be quite adequate for normal use. The $FIRST$ function for flagging the unique elements in a vector or matrix needs to be improved so that it can be used on large arrays, as does the $\Delta ROWMEM$ function which applies membership to rows of a matrix.

Sorting - 3 functions

Two simple functions for applying upgrade on character matrices, with a vector collating sequence. There is also a rather complex function for actually sorting a character vector or matrix. The advanced techniques given in Roy Sykes's Whizbangs are not used.

Substitution - 3 functions.

One text string can be substituted by another in a character vector. The method of generating compression and expansion vectors is rather inefficient.

Text-editing - 2 functions.

Only relevant for typewriter-style terminals.

Time - 2 functions.

Format the time of day.

Timing - 5 functions.

Simple functions for reporting CPU time.

Translation - 6 functions.

The most useful function is F 1 which converts characters to numbers, mapping invalid strings to zero. The remainder are hex and roman numeral conversion functions.

Uncategorised - 2 functions.

One function gives the union of two matrices. It is not at all clear what the other function is for.

Validation - 3 functions

The function for validating numeric input, by Gerald Bamberger, is essential for VSAPL users, and is efficient. The other two functions are rather specialised ones for validating name and integer specifications.

Last minute additions - 1 function

Dyadic iota for matrices. Uses the inner product technique, with no mention of the alternative sorting algorithm.

Experience of Using IBM's APL2/PC

by Anthony Camacho

Why I Bought It

I bought APL2/PC because of its free run time packager, hoping to be able to do some of the work which I was being asked to write in compiled BASIC in my favourite language instead. The end user, I thought, would not be able to tell the difference between one .EXE file and another. Furthermore I would be able to tell him I was doing his job with software supplied by IBM.

Of course there were some vicissitudes; the nature and resolution of these is described below. However I have to say that in general, my best hopes were realised: this product deserves success. I am plainly going to use it a lot.

The Maths Co-Processor

I use an Amstrad portable, which has the advantage that I can take it to the client, and use it to demonstrate things on, even if he hasn't a free machine for me. I can also transfer files on the spot, even if I have not foreseen the need and don't have the file on a disk of the client's format. APL2/PC would not run on it. 1 + 1 gave an answer consisting partly of letters u with grave accents.

I sought advice and was told that some timing characteristics of the processor were not PC compatible: the co-processor emulator will not run properly with this incompatibility. I had to buy a co-processor emulator for my Amstrad. Then it worked and I have had no compatibility problem since. Incidentally I understand that even 1 + 1 uses the maths co-processor and that the strong advice given in the manual to use a co-processor is because almost any program will run as much as four times faster with one.

Reading in Short Files

The jobs on hand each involved reading in several ASCII files of about 20K, and merging selected data from each of them into one or two output files. With BASIC this was difficult because there is insufficient array space to hold the files in memory. In APL2 the workspace available can easily exceed 400K; it depends on which auxiliary processors you load and on how much resident software you hold in memory.

To read the files in there is an auxiliary processor and a FILES workspace containing nice functions to do everything you want without having to read all about the AP. I soon

established that READFILEV would read a file into a vector where each element was one line of the file. This form is marvellously convenient for subsequent handling.

A system function 4 `⌈AT <namelist>` returns, for each object in the namelist, the number of bytes of memory it occupies and the number of bytes of data. I soon discovered that the nesting overhead on vectors of short vectors was high. 40K of characters (I was reading Financial Times share data) can use up 100K of memory.

The Shrinking Workspace

I encountered the first serious problem one Friday evening when there was no-one to phone for help. I had been trying out various file reading options and I got an unexpected `WS FULL`. `⌈WA` returned 67K. Where had my 400K of workspace gone?

I read the manual. I sometimes have to write manuals and count myself something of a connoisseur. This one is excellent. Every explanation I sought was in there - but not always easy to find. For example it was when reading about the packager that I discovered that APL2's internal tables can be cleared by:

<code>)RESET</code>	<code>⌈</code> Clears the stack (essential for OUT)
<code>)OUT WNAME</code>	<code>⌈</code> Writes WS to .ATF file
<code>)CLEAR</code>	<code>⌈</code> Clear WS
<code>)IN WNAME</code>	<code>⌈</code> Copies WS in from .ATF file
<code>)WSID WNAME</code>	<code>⌈</code> Was CLEAR WS
<code>)SAVE</code>	<code>⌈</code> Cleared tables in saved version

When I did this, I got back my 400K. I now do this routinely, using the .ATF and .APL versions of the workspace to back each other up. On I-APL I alternately save MYWS and MYWSX.

When I tried to repeat the fault I couldn't. It took some time to establish that as I was trying out file reading options, the workspace was mysteriously shrinking. Each time I read one of my small files in, I lost some space ... until eventually something failed with a `WS FULL`.

Next Monday I rang Dave Selby. Apparently there is at present no garbage collection process to restore to the workspace the space used by shared variables. Once a variable has been shared and retracted, the space it used is lost. In the FILES workspace such processes as READFILEV share the variables, read the file and then retract the sharing, so each time a file is read or written workspace is lost. Dave Selby told me of a simple work-around. Share the variable in the top level function, use it for all file reading and writing

operations and only retract it at the end. Setting the control variable to 10 closes the current file. This way the space is only lost once. It was necessary to amend some of the functions in the supplied FILES workspace. After that all was well. How pleasant to discover that they are all unlocked!

No Errors Found in the Interpreter

About the APL interpreter no news is good news. I haven't tried to find a bug and haven't found one by accident. The implementation does not support the diamond for multi-instruction lines, and the standard full screen editor doesn't allow function lines to exceed 80 columns. As I like to write functions which fit easily on a screen, this suits me fine. I haven't discovered how to edit a 200 character line. Maybe it is impossible.

I have not yet used the editors supplied in the EDIT workspace. I believe that they do allow you to edit 160 column lines. Under pressure to get the job done I stayed with the editor I know and have found the del editor perfectly adequate. One thing only I miss - being able to call back the last line with ")<enter>". I have quite often lost a long line I was working on by displaying a variable that was longer than expected. The default session manager does not allow you to scroll back to see items which have left the screen.

The Packager

The packager is a separate program that takes an APL saved workspace, incorporates or omits the co-processor emulator and such auxiliary processors as you specify, and produces a .EXE program. If you forget to set the `PLX` the .EXE run is very short! Any untrapped error or attempt to return to APL command mode in a packed program immediately returns you to DOS. The packaging process is interactive and fast. On one client's PS/2 model 70 it takes no more time than to enter the details required. Incidentally there is no `XLOAD` to load a workspace without executing the `PLX`, so I got into the habit of pressing CTRL+Break immediately after entering the load command. This prevents the `PLX` executing and doesn't leave anything on the state indicator. On the PS/2 model 70 I could only rarely do this. The disk is read so fast that nine times out of ten the program would load and begin executing before I could get interrupt pressed. That hard disk is really fast!

I have used the graphics auxiliary processor with success to do experimental map distortions. I have used the printing and DOS auxiliary processors too. DOS has occasionally puzzled me because I've somehow got it into a state where the variable is reported as shared, but commands assigned to it get no reaction. The cure is to reload the workspace. I have promised to seek and provide to Dave Selby a reproducible instance of this - it may be something I am doing wrong. I haven't yet had time to keep my promise.

It is not really fair to write about it before he has had a chance to help, but Vector deadlines have to be taken seriously nowadays.

I Have a Long Way to Go with APL2/PC

I have now delivered two jobs using APL2/PC and am delighted with it. As you see I have scarcely scratched the surface of the 27 workspaces or 17 auxiliary processors. I haven't even yet used the AP124 screen manager. I will.

For anyone who wants to deliver working programs to customers fast (where the customer doesn't care how you do it) this is a quite splendid product. For me it was worth its price and the cost of the 8087 just to get away from BASIC on the jobs I've done. I hope to do many more with it.

Some Additional Remarks

by Adrian Smith

I too have a copy of this excellent little APL, and I too have liked almost everything about it. Some random thoughts to soak up this spare half page:

- if anyone would like a Hercules + Ramfont for APL2/PC I would be delighted to pass mine on. Look out for it on the software library soon.
- AP124 works rather well. As a pressure test I pulled my GDDM-based editor off the mainframe and took it back to its 1982 status (i.e. to AP124). It runs fine; what's more you can do useful PC-ish things like pop-up help, fancy background colours, and character attributes. I much prefer it to QWIN.
- this beast deserves a better session manager! I flirted briefly with AP120, but found that it simply hobbles the PC and makes it feel like a mainframe! Like Anthony, I have retreated to using the 'raw' session instead.

Apparently it is shortly going to cost me £330 to upgrade my APL*PLUS/PC from version 5 to version 8. For £348 I could have free runtime, nested arrays, defined operators, and a 3.5Meg workspace. Clearly there is some effort involved in making the transition, but just at the moment I feel like giving it a damn good try!

Dyalog APL for 386 DOS PC's

reviewed by Jonathan Martin

The Reviewer

Let me put my cards on the table first. I've been an APLer for over 6 years, most of which was spent using APL*Plus under VM/CMS. However, for the last two years, I have been using Dyalog APL, both on an PC-AT with a go-faster Unix board in it and, for the last twelve months, on an IBM RT-6150 (all to myself!). Consequently, I was a little unsure of "going back" to good old DOS and its single-tasking environment.

My testing was done on a Compaq Deskpro 386 with 4Meg of memory, running at 16MHz. I had hoped to fully test it on an IBM PS/2 model 80, running at 25MHz as well but time caught up with me.

The Product

As I'm sure many of you are aware, Dyalog APL has been around for a number of years now but, until the release of this version, it was available solely for the Unix/Xenix world. I'm not sure exactly how many Unix machines it is available on - lets just say Dyadic are experienced at porting it... It is a true second generation APL with all the features you would expect - nested arrays, defined operators, error trapping, file system etc... but a lot more to distinguish it from the other APLs around.

The Review

The hardest part about doing this review is that I am used to using Dyalog - I spend most of my day working with it. Constantly, during this review, my feelings were that I was just using Dyalog APL. Now that may sound stupid. Yes, of course I was using Dyalog, there wouldn't have been much point in the review otherwise. What I mean is that it is just like the other versions of Dyalog I have used.

It was quick - that was something that you notice immediately. Starting up APL and loading a workspace takes three to four seconds. As far as benchmarks go, we have one at British Airways that is a solution to the Eight Queens problem. It is very typical of the work we do - lots of boolean matrices being thrown around, data being expanded to a common form and then compressed down to get the results out - Queens on Dyalog under DOS ran only about 15% slower than it does on my (old) 6150.

A major enhancement to Dyalog that, while not brand new (it has just appeared in the current release of the 6150 version), is certainly worth talking about. It is `QSM`, the new screen manager. Under this version, they have introduced something called Screen Memory Mapping which, while I do not claim to understand how it does what it does,

transforms $\square$ SM from being just a superb method of managing and updating the screen to a superb and extremely fast facility. If you have only ever used an AP124/6 type interface and have longed for those flashy PC-style front-ends, then want no more. $\square$ SM gives you all that and more and in such an intuitively APL-like way.

Useful things like pop-up windows become trivial and selection bars and "buttons" need only a few minutes coding. You can also do a hundred and one other things with it. Try your hand at Othello, again written using $\square$ SM. Simple animation was one thing that was just about possible using the 6150 version. It now becomes much more viable at this speed

Two tests that I wanted to do on this version were to port two very different workspaces down. I must say porting is a bit slow at the moment but given that it is a job that is done once and once only, I can live with it. The first workspace was something I had knocked up quite quickly - a simple database with update/delete/edit facilities. It isn't particularly well written - the sort of thing that everyone writes for themselves and assumes that no-one will ever see. I had two minor problems with it under DOS. Firstly I had written it for 12 PFkeys - this particular set-up only had 10 of the twelve accessible. Moving 'quit' from PF12 to PF10 resolved that. Secondly, the line-drawing characters had been stripped out of the BOXTM function that decorates the help message. Again, only a minor irritation. Apart from that, no problems.

The second workspace was much larger (about 300 functions) and is one that had been moved from APL*Plus under VM to Dyalog about 18 months ago. It is a text editor with facilities for moving text around, inserting and deleting lines, formatting text and drawing graphics characters. The two problems mentioned above obviously occurred here but nothing else. I had effectively moved an application from a Unix machine to a DOS machine with only minor problems. Very impressive indeed.

There were certainly one or two bugs in it - this version is a couple of months away from being released yet - but nothing that appears dramatically difficult to solve. In fact, one of the bugs (excessive use of the cursor keys causing the APL to hang) generally manifests itself whilst playing the version of Othello that they have written for $\square$ SM (I was just about to win as well, honest).

Conclusion

This new version looks and feels just like the other versions of Dyalog I have used. I'm sorry to go on about it, but that is the over-riding impression that comes out. Personally, I like the multi-tasking features of running under Unix and the ability to be doing more than one thing at a time. However, there are many people and many companies who do not like Unix for one reason or another or who are bound by company loyalties and budgets to stay with DOS. This version could well be the answer to those people.

If you want to know what is happening to APL, get hold of a copy of this.

GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers

APL Matrix Printing with Stubs and Headings

Jerry M. Brennan, Jennifer E. Daly, and Lawrence H. Nitz

Introduction

Existing APL matrix printing capabilities are inadequate in terms of labelling and formatting for all but the simplest applications. Matrices that are larger than the page width of the printer or screen (typically about 80 columns) are wrapped, causing a loss of the original column structure and limiting the usefulness of many APL routines.

This formatting drawback coupled with the lack of column labels in default APL printing programs can result in confusing and aesthetically unsuitable numeric printouts. Therefore, the need exists for a matrix printing program that can present numeric matrices in labelled, columnar formats that are appropriate for business, financial, and statistical applications.

MPR is a matrix printer function that produces a compact, labelled printout of numeric matrices with user-specified row and column labels. (MPR was first used in the menu-driven statistical package 'Stat 1: A statistical toolbox' by Jerry M. Brennan and Lawrence H. Nitz, 1988, Kailua, Hawaii: Sugar Mill Software Corporation.) With MPR, matrices of any size can be printed in their entirety since values and labels are printed in segments as is common with other software. Also, certain values can be suppressed, allowing matrices such as ANOVA summary tables or financial reports to be printed with, for example, unnecessary zeros eliminated.

MPR is flexible, allowing considerable user customisation. Default settings include automatic determination of the minimum number of decimal places within each column, suppression of decimal points for integer columns, and default row and column labels. Settings can be changed with the global variables ANDEC, ALLEN, OPW, and APL. These environment variables are available as preset defaults or can be passed to the routine dyadically (see below for detailed instructions).

Such customisation represents a significant advantage of MPR and supports various applications of the program. For instance, MPR allows the user to print a series of different tables without explicitly formatting each combination of rows and columns, because the correct format for each matrix is automatically determined based on present default settings. This feature is currently accessible in spreadsheet programs such as Lotus 1-2-3, Excel and Quattro, but is not available as a system function in APL.

MPR in APL

MPR is a ambivalent function. The right argument consists of the data matrix to be printed. The optional left argument may contain:

- (1) row labels
- (2) column labels
- (3) maximum number of decimals
- (4) maximum label length
- (5) page width
- (6) maximum number of rows in each printed block.

MPR may be used monadically or dyadically. In the monadic instance, the program generates default row and column labels

AMPR MATRIX

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
R1	.00	.00	.34	.79	.47	.32	.25	.97	.77	.59
R2	.11	57.67	89.70	73.09	100.62	78.90	13.22	69.90	295.10	197.90
R3	114.28	223.70	95.93	271.23	291.53	229.78	245.23	.00	34.63	79.31
R4	100.94	108.00	206.75	385.47	196.00	.00	.00	.34	.79	.47

Monadically generated default row and column labels.

In the dyadic mode, row and column labels are passed to the program two ways:

- (a) through user-created variables
- (b) as a character string within quotation marks with rows first and columns second.

In the latter instance, the symbols ~ and ~ (i.e. left tack and right tack) are used to delimit row labels while < and > (i.e. implication and reverse implication) indicate column labels. The symbol / separates items within each of these delimiters:

'ROW COLUMN' AMPR MATRIX

	FIRST	SECOND	THIRD	FOURTH	FIFTH	SIXTH	SEVENTH	EIGHTH	NINTH
SALES	.00	.00	.34	.79	.47	.32	.25	.97	.77
INVENTORY	.11	57.67	89.70	73.09	100.62	78.90	13.22	69.90	295.10
SUPPLIES	114.28	223.70	95.93	271.23	291.53	229.78	245.23	.00	34.63
MISCELLANE	100.94	108.00	206.75	385.47	196.00	.00	.00	.34	.79
	TENTH								
SALES	.59								
INVENTORY	197.90								
SUPPLIES	79.31								
MISCELLANE	.47								

'T-/WEEK1/WEEK2/WEEK3/WEEK4-</JOE/MARY/SAM/HARRY/ETHEL/BERT/ERNIE/ANN/NANCY/TOM-
AMPR MATRIX

	JOE	MARY	SAM	HARRY	ETHEL	BERT	ERNIE	ANN	NANCY	TOM
WEEK1	.00	.00	.34	.79	.47	.32	.25	.97	.77	.59
WEEK2	.11	57.67	89.70	73.09	100.62	78.90	13.22	69.90	295.10	197.90
WEEK3	114.28	223.70	95.93	271.23	291.53	229.78	245.23	.00	34.63	79.31
WEEK4	100.94	108.00	206.75	385.47	196.00	.00	.00	.34	.79	.47

Dyadic use of MPR utilizing (a) user-defined variables and (b) a delimited, segmented character string to input row and column labels.

All values to be suppressed in the printed matrix are entered within quotation marks between the symbols `l` and `T`. However, MPR only suppresses true zeros. Thus, when decimal places less than the original value are selected, any non-zero decimal fraction that approaches zero (e.g. .00001) is printed as a non-zero number (i.e. .00...n). To suppress very small numbers, the original matrix should be modified with the operation `l` (i.e. floor):

'10T' ΔMPR MATRIX

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
R1		.00	.34	.79	.47	.32	.25	.97	.77	.59
R2	.11	57.67	89.70	73.09	100.62	78.90	13.22	69.90	295.10	197.90
R3	114.28	223.70	95.93	271.23	291.53	229.78	245.23		34.63	79.31
R4	100.94	108.00	206.75	385.47	196.00		.00	.34	.79	.47

'10T' ΔMPR I MATRIX

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
R1										
R2		57	89	73	100	78	13	69	295	197
R3	114	223	95	271	291	229	245		34	79
R4	100	108	206	385	196					

Value suppression via the left argument (a) without and (b) with use of the floor operator.

In addition to value suppression, the global variables ΔNDEC, ΔPL, ΔLLEN, and ΔPW can also be modified temporarily via the left argument. Here, the symbols n and u (i.e. cap and cup) are used to delimit environmental values from the row and column labels. The input syntax is ΔNDEC, ΔLLEN, ΔPW, ΔPL, with the sequence of these variables always preserved. Thus, ΔNDEC, which sets the maximum number of decimal places within the matrix and rounds data accordingly, occupies the first place in the vector. The default setting for this variable is 2. To change this setting, the user need only enter the desired number of maximum decimal places within the delimiting symbols.

The defaults for each of the remaining settings are as follows: ΔLLEN, which sets the length of the row and column labels, equals 10; ΔPW, which sets the page width, equals 80; and ΔPL, which sets the maximum number of rows in each formatted block, equals 50. For these settings, a negative number must be entered as a placeholder for each variable that will retain its default. For example, the vector -1 24 v-1 v-1 sets the maximum length for row and column labels for this print call to 24 without changing any of the other default settings; and 6 15 100 -1 changes the maximum number of decimal places to 6, label length value to 15, and the page width to 100 (the page width selected must correspond to the width of the printer. If the printer is only 80 columns wide and

100 columns are requested, the remaining 20 columns will be wrapped) without altering the default maximum number of rows per block:

'ROW COLUMN n6 15 "1 "10" AMPR MATRIX

	FIRST	SECOND	THIRD	FOURTH	FIFTH	SIXTH
SALES	.00000	.000015	.340660	.787984	.466541	.324566
INVENTORY	.10674	57.670103	89.701031	73.092784	100.618557	78.896907
SUPPLIES	114.27500	223.700000	95.925000	271.225000	291.525000	229.775000
MISCELLANEOUS	100.93750	108.000000	206.750000	385.470588	196.000000	.000000

	SEVENTH	EIGHTH	NINTH	TENTH
SALES	.252203	.970568	.772726	.587621
INVENTORY	13.216495	69.900000	295.100000	197.900000
SUPPLIES	245.225000	.000000	34.625000	79.312500
MISCELLANEOUS	.000015	.340660	.787984	.466541

Dyadic input of variables containing labels and default settings

MPR also accepts values to be suppressed through the left input string. In fact, the optional left argument can be used simultaneously to input row and column labels, default settings, and value suppression:

'1-DRUG/TEST/GENDER/DRUG x TEST-4<SS1/SS2/SS3/SS4/SS5/SS6/SS7/SS8/SS9/SS10>n4
"1 "1010T" AMPR MATRIX

	SS1	SS2	SS3	SS4	SS5	SS6	SS7
DRUG		.0000	.3407	.7880	.4665	.3246	.2522
TEST	.1067	57.6701	89.7010	73.0928	100.6186	78.8969	13.2165
GENDER	114.2750	223.7000	95.9250	271.2250	291.5250	229.7750	245.2250
DRUG x TEST	100.9375	108.0000	206.7500	385.4706	196.0000		.0000

	SS8	SS9	SS10
DRUG	.9706	.7727	.5876
TEST	69.9000	295.1000	197.9000
GENDER		34.6250	79.3125
DRUG x TEST	.3407	.7880	.4665

Explicit row and column labels, temporary modification of the global variables, and value suppression.

In this way, the settings for a particular project can be easily altered without modifying the existing defaults. Some examples of these changes are displayed below:

Option	Global	Default	Delimiters*	Example
Row Labels	--	R1,R2,...Rn	'<X>'	'</WEEK1/WEEK2>'
Column Labels	--	C1,C2,...Cn	'<X>'	'</JOE/MARY>'
Row and Column Labels	--	R1,Rn..C1,Cn	'<X><Y>'	'</1/2></A/B>'
Variable-Row Labels	--	R1,R2,...Rn	'X'	'ROWLABELS'
Variable-Column Labels	--	C1,C2,...Cn	'X'*	'COLUMNLABELS'
Variables-Row & Column	--	R1,Rn..C1,Cn	'X Y'	'ROW COLUMN'
Maximum decimal places	ANDEC	2	'nXu'	'n4u'
Maximum label length	ALLEN	10	'n-1 Xu'	'n-1 15u'
Maximum page width	OPW	80	'n-1 -1 Xu'	'n-1 -1 120u'
Maximum rows in block	APL	50	'n-1 -1 -1 Xu'	'n-1 -1 -1 10u'
Values suppressed	--	--	'1Xr'	'10 99r'

*X represents the changed default

< left tack
 > right tack
 < implication
 > reverse implication
 n cap
 u cup
 1 ascending tack
 r descending tack

*When a column variable is used without a row variable, the variable name must be preceded by a space.

Changing Defaults via the Optional Left Argument

Jerry M Brennan
 University of Hawaii
 Honolulu, Hawaii 96822

Appendix: The Function Listings

▽ A1 ΔMPR X:BLK;BLKRL;CBLK;CL;CLPS;D;DEC;F;FMT;I;II;K;L;LCOLL;LLEN;PL;PW;RB;
K;RL;RLPS;S;SEL;SUP;T;Z;b;c;r

```
[11] A Matrix print routine. Globals: ΔPL, ΔPW, ΔLLEN, ΔNDEC
[12] A Other FNS: MPRA L MPRAWCOLD MPRA CL CSEL BELIM
[13] A--- SYNTAX
[14] A 'RVAR S CVAR S' ΔMPR X where RVAR S & CVAR S are vars containing labels
[15] A 't/A/B/C<D/E/F>' ΔMPR X where rows are A B C and cols D E F
[16] A ' CVAR S' ΔMPR X or 'c/D/E/F>' ΔMPR X where RVAR S will be R1,R2...Rn
[17] A 'RVAR S' ΔMPR X or 't/A/B/C<' ΔMPR X where CVAR S will be C1,C2...Cn
[18] A ΔMPR X where RVAR S will be R1,R2,...Fn & CVAR S will be C1,C2...Cn
[19] A Uses global ΔNDEC ΔLLEN ΔPW ΔPL unless included within "n" and "u"
[10] A e.g. 'RVAR S CVAR S n2 10 132 60u' ΔMPR X uses as many as appear
[11] A length may be less than 4 use negative numbers for place holders.
[12] A Value suppression: any #'s placed between tτ will be blanked
[13] →LV×10#DNC 'Δ1' Δ Δ1<'
[14] LV:→LM×12=ρρX Δ X+(1,ρX)ρX
[15] LM:
[16] A--- DEFAULTS
[17] D←4↑Δ('nu' CSEL Δ1),12ρ' -9'
[18] D←(D×(D>0))+(D<0)×ΔNDEC, ΔLLEN, ΔPW, ΔPL
[19] DEC←D[1] Δ LLEN←D[2] Δ PW←D[3] Δ PL←D[4]
[20] SUP←, 'tτ' CSEL Δ1
[21] Δ1←(~Zv#Z+Δ1ε'nu1τ')/Δ1
[22] A--- LABELS
[23] →LA×10#,ρΔ1
[24] RL←'1 R' MPRA L X Δ CL←MPRA L X Δ →LX
[25] LA:r←~ 0 0 =ρRL←SStoMAT, 'tτ' CSEL Δ1 Δ C←~ 0 0 =ρCL←SStoMAT, 'C' CSEL Δ1
[26] →LX×1rΔC
[27] A elim between delims
[28] b←Δ1l' ' Δ →LB×1~'c'εΔ1 Δ Δ1←(~Zv#Z+Δ1ε'c')/Δ1
[29] LB:→LR×1~'t'εΔ1 Δ Δ1←' ', (~Zv#Z+Δ1ε't-')/Δ1 Δ b←1
[30] A ROW LABELS
[31] LR:→LD×1r Δ →LC×1l=b Δ RL←Δb↑Δ1 Δ Δ1←b↓Δ1 Δ →LD
[32] LC:RL←'1 R' MPRA L X Δ →LD
[33] A COL LABELS
[34] LD:→LX×1c Δ →LEX1Λ/Δ1=' ' Δ CL←ΔΔ1 Δ →LX
[35] LE:CL←'2 C' MPRA L X
[36] LX:
[37] RL←((1↑ρRL), LLEN(1↑ρRL))↑RL←TAB RL A row labels from vector to matrix
[38] CL←((1↑ρCL), LLEN(1↑ρCL))↑CL←TAB CL A col labels from vector to matrix
[39] LCOLL←1+~/~ΛφCL=' ' A finds length of each col label +1
[40] RLPS←~1↑RBLK←(1↑τρX)÷PL A RBLK is block # for each row. RLPS # blocks
[41] L←0
```

```

[42] A Loop for each block of ROWS
[43] L5:→PAU×1RLPS<L+L+1
[44] FMT←DEC MPRΔWCOLD(RBLK=L)/X A FMT[;2] find min # decimals for each col
[45] FMT[;1]←FMT[;1]LCOLL A finds min width for each col
[46] A BLKRL←row labs for col blk. CBLK←blk # for each col. CLPS←# col blks
[47] II←I+0 ◊ CBLK←0 ◊ BLKRL←BELIM(RBLK=L)/RL
[48] L54: A Loop to determine # COLS in each print block for this ROW block.
[49] CBLK←CBLK,(T←+/1>(+\I↓FMT[;1])÷PW-1↑ρBLKRL)/II←II+1
[50] I←I+T ◊ →L55×1I=1↑ρFMT ◊ →L54
[51] A Loop to print each sub block of ROWS & COLS
[52] L55:CLPS←1↑CBLK ◊ K←0
[53] L6:→L5×1CLPS<K+K+1
[54] SEL←(CBLK=K)/1ρCBLK A SEL is cols selected for this print
[55] DTCLN,((1↑ρBLKRL)ρ' '),FMT[SEL;1]MPRΔCL CL[SEL;1] A Print SEL col labels
[56] A Regular print without value suppression
[57] →SUP×10#ρSUP ◊ BLKRL,(, (CBLK=K)/FMT)* (RBLK=L)/(CBLK=K)/X ◊ →L6

58] A Print with value suppression
59] sUP:((1,ρBLKRL)ρBLKRL), 1 0 *(F[;1]/S←1SUP)φ[1]((,F←(CBLK=K)/FMT)*S←(RBLK=L
    )/(CBLK=K)/X),[0.5]' '
60] →L6
61] PAU:
    V

V C←BELIM C;A
[1] A elim mult lead & trail blanks for vecs elim trail blank cols for matrix
[2] →(1=ρρC)/L1 ◊ C←(φv\φv/' '#C)/C ◊ →0
[3] L1:C←((φv\φA)∧v\A←' '#C)/C A elim lead and trail delimiters
[4] C←(~C DSS 2ρ' ')/C A elim mult delimiters
    V

V R←F MPRΔCL L;K;T
[1] A Spaces col labels L using format F of col widths.
[2] R←' ' ◊ K←0
[3] L1:→0×1(1↑ρL)<K+K+1 ◊ R←R,(-+/\' '=φT)φT←F[K]↑,L[K;] ◊ →L1
    V

V R←Δ2 MPRΔL X;C;D
[1] A Labels for matrix X. Δ2 gives optional dim and root for labels
[2] A Default is last dim and root of 'C'.
[3] →L1×10#DNC 'Δ2' ◊ Δ2←'C'
[4] L1:→L2×1' '←Δ2 ◊ Δ2←(φρX), ' ',Δ2
[5] L2:D←(ρX)[Δ(Δ21' ')]↑Δ2] ◊ C←(Δ21' ')+Δ2
[6] R←(~A+R=' ')/R←((D,ρC)ρC),φ(D,1)ρ1D
[7] R←''' TAB(R#' ')/R←,R,'''
    V

```

```

▽ R←DEC MPRAWCOLD M:F:I;K;NC:T;V
[1]  A Determines min col widths for data matrix M
[2]  TOP:→ST×12=ρρM ◊ M+((ρM).1)ρM
[3]  ST:I+0↑[10*(10*-DEC)+I]IM
[4]  F←(NC+1↓ρM)ρ0 ◊ →PAU×1DEC=0
[5]  K←0 A Column counter loop.
[6]  L1:→PAU×1NC<K+K+1 ◊ V←M[K] ◊ →L1×1^/V=1V ◊ F[K]←1
[7]  L2:→L1×1(F[K]=DEC)
[8]  →L1×1(^/T=[T+V×10+F[K])
[9]  F[K]←F[K]+1
[10] →L2
[11] A 1=space I=integer part F=frac part (F>0)=decimal (1=×1/M)=neg sign
[12] PAU:R←(1+I+F+(F>0)+1=×1/M),[1.5]F
▽

▽ R←D CSEL C;M;S
[1]  A select text between delimiters D form into matrix R
[2]  S←ρR← 0 1 ↓(1↑D)tAB(Mv#M+C←D)/C
[3]  R←,R ◊ R[(R=1↓D)/1ρR]←' ' ◊ R←SρR
▽

▽ V←Δ2 tAB V:POS
[1]  A convert vector to matrix
[2]  →(1<ρρV)/0 A esc if already a matrix
[3]  →(0=UNC 'Δ2')/L2
[4]  →(1=ρ,Δ2)/L1 ◊ V[(V←Δ2)/1ρV]←1↑Δ2
[5]  L1:V←(1↑Δ2),V
[6]  L2:V←(1-(V=1↑V)11)↓V A remove trail delims
[7]  V←(~V DSS 2ρ1↑V)/V A elim mult delimiters
[8]  V←SStoMAT V ◊ →0
▽

▽ R←SStoMAT V:T
[1]  T←V←1↑V+V.1↑V ◊ V←(~T)/V ◊ T←T-1+0.1↑T+T/1ρT
[2]  T←T◊.≥1↑/0,T←(T≠0)/T ◊ R←(ρT)ρ(,T)\\V
▽

```

Personal Construct Analysis and Rule Formation in APL

by Mike Williams

About the Author

You ask for a very short potted biography of my career. Here goes...

Graduated from Newcastle University with an upper second in statistics. After a very brief summer job working for British Gas, returned to the university to work as a research assistant under G.B. Wetherill on a project to build an intelligent system for regression analysis.

Due to various monetary problems, project became a general investigation of statistical expert systems. It was in the course of investigating a possible system for data editing that I first learnt APL, by building a program to set up a system of edits. Since then, APL has been used by me to build ExpCheck (a system for spotting non-statistical faults in proposed experimental designs).

Currently, I am halfway through a Ph.D. on Aspects of model and rule elicitation for statistical expert systems, in which all of the algorithms and programs are been written in APL. This paper now forms part of that project.

Introduction

In a note that was published last year in Vector [7], I explained how APL could be used as the base language for a fault-spotting system for proposed experimental designs. The natural question to arise from that was whether APL could be used for other aspects of artificial intelligence, in particular the handling of rules and reasoning. Some aspects of these problems have been considered within the pages of this journal, for example by Christopher Harvey [1],[2], and most recently by Allan Prys-Williams [4]. However, they have mainly dealt with the manipulation of already existing rules, and also used either APL2 or Dyalog APL, both of which come with features not available currently in other common interpreters.

What I want to do in this note is to show that APL can also be useful in obtaining rules for expert systems, and that the technique can be used in any dialect of the language. For my purposes, the techniques will be illustrated by obtaining rules for picking from a set of very simple statistical tests. However, it is my belief that the algorithms and programs illustrated here can be used in any situation where selection from a list of choices is involved.

A Brief History

However, the formation of rules in a given situation is not possible without some form of 'picture' of the problem. A technique is needed that enables us to gain that picture - personal construct analysis is just such a technique. Personal construct analysis is a fairly well known technique, used in psychology to help understand a particular situation and in personal assessment. The proposer of the theory itself was George A. Kelly [3] in 1955. In the deceptively slim volume which presented the original ideas, he contended that a man reaches decisions about a situation by comparing it to similar problems that he has encountered in the past. These past events are used to form a type of scale in his mind, known as a construct, against which he measures each new situation and amends his decision accordingly.

Obviously, if these constructs could be obtained and used in some way, then the way in which a person makes decisions could be modelled. Mildred Shaw saw the possibilities in this, and devised a suite of programs that obtains these constructs by interactive questioning of a person.[5] This was made possible by starting with a basic set of things to decide about, and a small set of constructs that separate these objects. A construct can be thought of as a sort of scale, at one end of which is a description of some characteristic that distinguishes the objects or elements from each other, and at the other end a related but different aspect of the characteristic. Each end of the construct is known as a pole, the former been the left hand pole (LHP) and the latter the right hand pole (RHP). Then each of the objects or elements is scored against each construct. If the left hand pole is true, a score of 1 is allocated to that element for that construct.

Conversely, if the right hand pole is true, then a score of n is allocated, where n is some odd integer. For our example, n will be taken to be 5. If neither pole is strictly true, a score of between 1 and n is allocated. In this way, a matrix of scores for all elements against all constructs is formed. This is analysed, and new elements and/or constructs are added as a result of each stage of the analysis. In the end, a picture of the decision making process employed by the user is contained in the matrix of scores.

Although the actual process of forming the grid is simple, the original program that carried out the analysis required a very powerful micro in order to be fully and efficiently operational. Since then, scaled down versions have appeared in various languages, including BBC Basic. However, to the best of my knowledge, nobody has yet attempted to program a version in APL. This is mildly surprising, as the matrix structure used would make APL the ideal language to program in.

In addition, the possibility obviously existed of using the grid to obtain rules that model the way in which the user reached his selection or decision. This would be extremely valuable, as an effective portable method of gaining useful rules for any expert system. In the selection of tests or models in statistics, I have proposed that these rules be known

as Strategic Selection Rules, and have introduced them in a paper currently under preparation [8].

So how are the relevant techniques, algorithms, etc., prepared and programmed so that they can be executed? In the next section, we will look at the program that emulates the one prepared by Mildred Shaw for construct elicitation, PEGASUS. Then we will look at how a basic analysis of the grid can be done, again emulating one of Shaw's programs - FOCUS. Finally, the program for obtaining the rules will be introduced - SEARCH.

Pegasus

PEGASUS is actually a workspace, prepared in APL*PLUS, which contains a number of functions that are used for obtaining the matrix. First of all, a starter set of six elements to be separated is entered, and then four constructs are obtained that separate out those six elements.

The trick in obtaining the rest of the constructs and/or elements is to see how closely matched the elements have become, once the values against the first four constructs are obtained. If we have two elements, I & J, and the matrix containing the scores is A, then the similarity between the elements is measured by:

$$R = 100 - (A[I;K] - A[J;K]) \quad (4 \times CC)$$

where CC is the current number of constructs, and the summation is over all existing constructs.

The following APL function does this operation:

```

FUNCTION I;J;R;T;X;Y
-----
[1]      A
[2]      A This checks all possible pairs of elements, and finds that
[3]      A with the highest matching score. If that score is above 70,
[4]      A a new construct is suggested to separate them.

[5]      FLAG←0 ⋄ T←0
[6]      'STAGE 3 - ELEMENT MATCHING' ⋄ '' ⋄ 'Please wait ...' ⋄ I←1
[7]      L1:J←I+1
[8]      L2:R←100-100×(÷|A[I;]-A[J;])÷(4×CC)
[9]      →(R<T)/L3
[10]     T←R ⋄ X←I ⋄ Y←J
[11]     L3:→((J+J+1)≤EE)/L2
[12]     →((I+I+1)≤EE-1)/L1
[13]     →(T<70)/0
[14]     'The following are matched at ',(⌈T+0.5⌋), ' per cent :-'
[15]     ⌈←E[(X,Y);]
[16]     'You need to provide a new construct to separate these two'
[17]     'in some way that has not been used already.'
[18]     'Do you wish to do that?'
[19]     X←⌈
[20]     →((1+X)∈'YY')/L4
[21]     →0
[22]     L4:FLAG←1
      ▽

```

The largest value, and the elements that generate that value, are noted. If the value exceeds a given limit, then a new construct to separate these two elements is asked for. A similar situation allows us to add new elements by comparing constructs. However, because of the nature of personal constructs, we have to allow for the possibility of reverse matching.

The following function performs that task. Note especially that the formula for similarity has been subtly adapted to accommodate the reverse matching.

FUNCTION2;I;X;R;T

```

[1]      A
[2]      A This checks all existing constructs against the last to be
[3]      A entered, to see if any of them are too closely matched.
[4]      A If there are any, a new element is asked for.

[5]      FLAG+0
[6]      'STAGE 5 - CONSTRUCT MATCHING' ◊ '' ◊ 'Please wait ....'
[7]      T+0 ◊ I+1
[8]      L1:R+100-200*(+/( ((,A[I])- ,A[;CC]))÷(4×EE)
[9]      +(R≥0)/L2
[10]     'Reverse matching on constructs ',(I), ' and ',(I+CC)
[11]     R+100-200*(+/( ((,A[I])-(6-,A[;CC]))÷(4×EE)
[12]     L2:+(R<T)/L3
[13]     T+R ◊ X+I
[14]     L3:+( (I+I+1)≤CC-1)/L1
[15]     +(T<70)/0
[16]     'The following are matched at ',(I+T+0.5), ' per cent'
[17]     ◊+((C[X;]), ' - '),D[X;] ◊ ◊+((C[CC;]), ' - '),D[CC;]
[18]     'What is required is a new element to take different scores on'
[19]     'these scales. Can you think of such an element?'
[20]     X+◊
[21]     +((1+X)∈'YY')/L4
[22]     +0
[23]     L4:FLAG+1

```

▽

When the user is satisfied with his set of constructs, they are stored in a component file, and a report of his results is given. For example, the following was obtained while trying to find ways of selecting from a list of two sample analysis techniques.

```

Do you wish to save the results?
N
Please wait for a few seconds.
The purpose fo your grid was:-
Compare Significance Tests
The following is the current score matrix:-
  1 5 1 1 1 5 1 1
  1 5 1 1 5 5 5 1
  5 3 1 3 3 3 4 5
  1 1 3 5 1 1 1 5
  3 1 5 2 3 3 1 5
  3 1 5 5 3 1 1 1
  1 5 2 1 1 1 1 2
The elements you were examining were :-
T-Test
F-Test
Chi-Square
Median
Corr.Coeff
Spearman's
Aspin-Welch
and the constructs that you used were :-
Parameter estimate      - Relationships
Ranking                  - No Ranking
Normality Assumed       - No Normality Assumptions
Parametric               - Non-Parametric
Mean tests               - Variance tests
Robust                   - Non-Robust
Two Groups               - Many Groups
Tables for Results       - Formulae for Results
Thank you for running this program.

```

Having obtained the grid, the user can now go and see how all the elements and constructs relate to each other. To do this, we need the next workspace to be prepared - FOCUS.

Focus

Once again, FOCUS is a collection of programs that analyses the matrix of scores in order to see how all the elements and constructs relate to each other. The algorithm that is used is very simple:

1. Form a matrix G from the matrix of scores A . G is an upper triangular matrix, where $G[I,J]$ is the matching score for constructs I and J .

2. From G, we form a dendrogram or tree diagram by matching together the groups of constructs that have the highest matching score. This is repeated until all the elements are placed together in one group.

3. Repeat the process for the elements.

Obviously, the process is much simpler in APL, because the rather tiresome matrix comparisons required can be speedily programmed. Once the data has been read in from the component file, the results can be obtained within a minute or two. The following function is the one for grouping together the constructs once the matrix G has been obtained:

FUNCTION3;X;Y;I;J;R;S;T;L;X1

```

[1]      A
[2]      A Having formed the construct score grid, we now analyse the
[3]      A grid to find the similarities, and form the results
[4]      A required to draw the 'dendrogram'

[5]      'CONSTRUCT SCORE MATRIX ANALYSIS' ⍋ ''
[6]      L1:T+[(~.,G) 1 1 ⍋G)/.,G
[7]      R←X+1+[(T=G)/1CC×CC)÷CC ⍋ S←Y←(1+((T=G)/1CC×CC))-(R-1)×CC
[8]      +(G[X;X]=G[Y;Y])/L15
[9]      +(Z<G[X;X])/L13
[10]     R←G[X;X] ⍋ X1+((1 1 ⍋G)=R)/1CC ⍋ I+1
[11]     L5:G[(X1[I]);X1[I]]+Z
[12]     +((I+I+1)≤(ρX1))/L5
[13]     L13:G[X;X]←Z
[14]     +(Z<G[Y;Y])/L14
[15]     S←G[Y;Y] ⍋ X1+((1 1 ⍋G)=S)/1CC ⍋ I+1
[16]     L7:G[(X1[I]);X1[I]]+Z
[17]     +((I+I+1)≤(ρX1))/L7
[18]     L14:G[Y;Y]←Z
[19]     U[Z]←R ⍋ V[Z]←S ⍋ W[Z]←T ⍋ Z←Z+1
[20]     L15:G[X;Y]←1
[21]     +(Z/G[1;1]≠1+ 1 1 ⍋G)/L1
[22]     ⍋'GRID ANALYSIS FINISHED.'

```

▽

Once the grouping of the matrix has finished, a diagram can be produced showing the relationships.

The following is the function that produces the diagram, as well as a printout of the score matrix:

FUNCTION4;I;J;P;X

```

[1]      A
[2]      A We are now ready to print the results of the grid analysis
[3]      A for the construct matrix. The grid is printed out first,
[4]      A and then a dendrogram showing the relationships.

[5]      □CMD 'CLS'
[6]      'CONSTRUCT SCORE PRESNTN.' □ ' ' □ 'CONSTRUCT SCORE MATRIX'
[7]      I←1
[8]      L1:X←(I-1)+G1[I:]
[9]      X[I]+I □ □+(((I-1)×4)ρ' '),((2×(ρX))ρ 4 0)×X
[10]     →((I+I+1)≤CC)/L1
[11]     □←(4,1)ρ' '
[12]     'PRESS ANY KEY TO CONTINUE'
[13]     X←□INKEY □ P+0 □ I+Z-1
[14]     L11:→(U[I]>CC)/L16
[15]     □←C[(U[I]);],*U[I] □ P+P+1
[16]     L12:□←((1(30+(100-W[I])+2))ρ' '),*I
[17]     →(V[I]>CC)/L17
[18]     □←C[(V[I]);],*V[I] □ P+P+1
[19]     →(P=CC)/L18
[20]     J←CC+1
[21]     L13:→(U[J]=I)/L14
[22]     →((J+J+1)≤Z-1)/L13
[23]     J←CC+1
[24]     L20:→(V[J]=I)/L15
[25]     →((J+J+1)≤Z-1)/L20
[26]     L14:I+J □ →L12
[27]     L15:I+J □ →L13
[28]     L16:I+U[I] □ →L11
[29]     L17:I+V[I] □ →L11
[30]     L18:'PRESS ANY KEY TO CONTINUE'
[31]     X←□INKEY
[32]     □CMD 'CLS'

```

▽

Consider again the selection from tests for two samples. When our grid is analysed, the following output was obtained for the construct score matrix ...

FORMING CONSTRUCT SCORE MATRIX

Grid formation finished

CONSTRUCT SCORE MATRIX ANALYSIS

Grid analysis finished.

CONSTRUCT SCORE PRESENTATION.

Construct score matrix

1	29	21	36	57	14	36	21
2	64	79	14	57	36	64	
3	43	7	36	29	43		
4	21	64	14	43			
	5	57	64	21			
		6	50	36			
			7	14			
				8			

Press any key to continue

Parameter Estimate	1		14
Mean tests	5		13
Two Groups	7		15
Ranking	2		
Parametric	4	9	
Normality Assumed	3		10
Tables for Results	8		11
Robust	6		12
Press any key to continue			

It is a natural question to ask what the usefulness of this analysis is. As far as the element score matrix is concerned, it enables us to check that the elements we wish to select from have been clearly separated and distinguished from each other. However, more important is the fact that the construct score grid enables us to see exactly where any relationships between the poles of the constructs lie. Whilst they do not tell us exactly what the relationships are - the diagram is not detailed enough for that - it can show which constructs are closely related to each other. It was in realising this that the possibility of using this technique for rule elicitation first occurred. Mildred Shaw and

her husband, Brian Gaines, have shown that logical implications between the poles of the constructs can be obtained [6]. It was in adapting this work to APL that the final workspace was developed - one which extracted rules used by the user for selection from the matrix of scores.

Search

The derivation of the rules depends on examining a subset of all the elements or constructs and recalculating an intermediate similarity measure. Two types of rules can be obtained, one which lists the important characteristics of the elements, and another which gives the relationships between the characteristics. In the paper on strategic selection rules, these are referred to as model characteristic rules and characteristic implication rules respectively.

So how are these various rules obtained? We'll look at each type in turn, and examine how the basic algorithm is implemented in APL. Once again, the output from the program will be illustrated by the two-sample example.

The first type of rule mentioned is that which states the important characteristics of the elements. To obtain this, we take each element in turn and form a vector X which contains the score for that element over all the constructs. Now we consider each member of X , X_j .

If $X_j = 1$, then the left hand pole of construct j is a characteristic.

If $X_j = 5$, then the right hand pole of construct j is a characteristic.

All of the appropriate characteristics are then listed as the IF clauses for a rule to select that particular element. The truth of that particular deduction can be obtained as follows:

1. For each member of X , X_j , if $X_j = 5$ then let $X_j = 1$.
2. Calculate a truth value D , where $D = 1 - (\text{Sum } X - 1) \div (4 \times CC)$ where CC is the total number of constructs, i.e. X

This truth value is a little crude, and possible modifications are been considered. However, the results generated do appear to be reasonable.

The following function executes the algorithm:

RULEFORM;X;Y;Z;I

```

[1]      A
[2]      I+1
[3]      L1:=X+A[I;]
[4]      Q←'MODEL/TEST IS ',E[I;]
[5]      Q←'IF:-'
[6]      Y+(X=1)/\CC
[7]      Q←C[Y;]
[8]      Y+(X=5)/\CC
[9]      Q←D[Y;]
[10]     Q←'WITH TRUTH VALUE'
[11]     X[Y]+1
[12]     Z←1-(+/\|1-X)/(4×CC)  Q←*Z
[13]     ' '
[14]     +(I+I+1)≤EE)/L1
▽

```

When our grid was run through the analysis, the following rules were obtained:

Test characteristic rule formation

Model/test is T-test	Model/Test is Corr.Coeff
if:-	if:-
Parameter Estimate	Ranking
Normality Assumed	Two Groups
Parametric	No Normality Assumptions
Mean Tests	Non-Parametric
Two Groups	with Truth Value
Tables for Results	0.78125
No Ranking	
Non-Robust	
with Truth Value	
1	

Model/test is F-Test	Model/test is Spearmans
if:-	if:-
Parameter Estimate	Ranking
Normality Assumed	Robust
Parametric	Two Groups
Tables for Results	Tables for Results
No Ranking	No Normality Assumptions
Variance Tests	Non-Parametric
Non-Robust	with Truth Value
Many Groups	0.875
with Truth Value	
1	

Model/test is Median
 if:-
 Parameter Estimate
 Ranking
 Mean Tests
 Robust
 Two Groups
 Non-Parametric
 Formula for Results
 with Truth Value
 0.9375

Model/test is Aspin-Welch
 if:-
 Parameter Estimate
 Parametric
 Mean Tests
 Robust
 Two Groups
 No Ranking
 with Truth Value
 0.9375

Model/test is Chi-Square
 if:-
 Normality Assumed
 Relationships
 with Truth Value
 0.59375

As can be seen, the rules give the required characteristics for each test to be appropriate in a given situation.

The second type of rule that can be obtained is that which determines the relationships between the various characteristics that have been defined by the user. The algorithm is applied for all pairs of constructs, I and J, where:

$$1 \leq I, J \leq CC$$

1. Form a subset of the elements, where the members are those elements for which the score on construct I is 1. Call this subset X.
2. Calculate a distance score between constructs I and J for the subset of elements X, i.e. $R = 1 - ((\text{Sum } A[I;K] - A[J;K]) \div (4 \times X))$ This is essentially a reduced measure of similarity of constructs, where the summation, and hence the comparison, is over those elements for which the LHP of construct I is a characteristic.
3. Select a value T. T represents how true a particular deduction has to be before the user considers it to be of use.
4. If $R > T$, then we say that LHP of construct I LHP of construct J at truth level R. If $R < 1-T$, then we say that LHP of construct I RHP of construct J with truth value $1-R$.

The entire algorithm is repeated, but with two slight amendments:

- a) X is the set of elements for which the score on construct I is 5.
- b) The LHP and RHP in stage 5 are swapped.

The following function executes the algorithm as fully stated for all possible pairings.

```

SEARCH; I; J; X; R; Y; Z
-----

[1]  A FUNCTION 6 FOR MAIN BODY OF PAPER

[2]      I+1
[3]  L1: X+(A[;I]=1)/1EE
[4]      J+1
[5]  L2: +(J=I)/L3
[6]      R+1-(+|A[X;I]-A[X;J])÷(4×pX)
[7]      +((R>0.3)^(R<0.7))/L3
[8]      Y+I  Ø  Z+J
[9]      +(R≥0.7)/L4
[10]     Z←-Z
[11]     R+1-R
[12]  L4: SHOW Y,Z,R
[13]  L3: +((J+J+1)≤CC)/L2
[14]     +((I+I+1)≤CC)/L1
▽

```

The second function shown here (SHOW) merely gives the list of implications in a more readable format for use.

```

SHOW X; FLAG; STORE
-----

[1]  A SUPPLEMENTARY TO FUNCTION 6 FOR MAIN BODY OF PAPER

[2]      FLAG+1
[3]      +(X[2]>0)/L1
[4]      X[2]←-X[2]
[5]      FLAG+2
[6]  L1: STORE← 2 30 pC([X[2]];],D([X[2]]);]
[7]      [←C[X[1]];],STORE[FLAG;],*X[3]
▽

```

When the function is run, a list of useful implications is printed out. This list is then used to compile a map, from which the actual rules are read off. The following is one of the list of implications for the test selection problem:

Characteristic implication rule formation - Stage 1		
Parameter Estimate	No Ranking	0.75
Parameter Estimate	Normality Assumed	0.8125
Parameter Estimate	Parametric	0.75
Parameter Estimate	Mean Tests	0.75
Parameter Estimate	Two Groups	0.75
Ranking	No Normality Assumptions	0.833333
Ranking	Non-Parametric	0.75
Ranking	Robust	0.833333
Ranking	Two Groups	1

Normality Assumed	No Ranking	0.833333
Normality Assumed	Parametric	0.833333
Normality Assumed	Non-Robust	0.833333
Normality Assumed	Tables for Results	0.833333
Parametric	Parameter Estimate	1
Parametric	No Ranking	1
Parametric	Normality Assumed	0.916667
Parametric	Tables for Results	0.916667
Mean Tests	Parameter Estimate	1
Mean Tests	Normality Assumed	0.75
Mean Tests	Two Groups	1
Robust	Parameter Estimate	0.833333
Robust	Mean Tests	0.833333
Robust	Two Groups	1
Two Groups	Parameter Estimate	0.8
Two Groups	Mean Tests	0.8
Two Groups	Robust	0.7
Tables for Results	Parameter Estimate	0.833333
Press any key to continue		

As a result of the two lists of implications, a full set of rules are given. The following are some examples of the rules obtained:

IF Testing for means
THEN Parameter estimate required (with truth value 1)

IF Parameter estimate required
THEN Testing for means (with truth value .75)

IF Normality assumed
THEN Tables needed (with truth value .83)

Conclusions

The aim of this note was to show that APL could be used for effective rule generation in a given situation. As we have seen, by using well known techniques from psychology and adapting them slightly, a simple method for rule generation can be obtained and used.

There is still some work to be done on tidying up and improving the functions and algorithms used. However, as we have shown, the method has real potential for effective use, and once again shows that APL should not be ignored as a language for use in intelligent programming or expert system development.

Mike Williams,
Department of Mathematics and Statistics,
University of Newcastle upon Tyne,
NEWCASTLE UPON TYNE. NE1 7RU

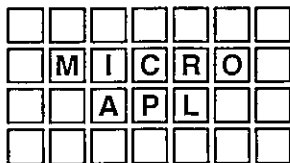
References

- [1] HARVEY, C. (1984a) A Practical Introduction to Expert Systems, Pt 1. Vector, Vol.1, No.3, pp101-111.
- [2] HARVEY, C. (1984b) A Practical Introduction to Expert Systems, Pt 2. Vector, Vol.1, No.4, pp98-102.
- [3] KELLY, G.A. (1955) The Psychology of Personal Constructs. Norton, N.Y.
- [4] PRYS-WILLIAMS, A. (1988) The Limits of Forward Chaining. Vector, Vol.5, No.2, pp88-103.
- [5] SHAW, M.L.G. (1979) On becoming a personal scientist. Academic Press, London.
- [6] SHAW, M.L.G. & GAINES, B.R. (1980) Fuzzy Semantics for Personal Constructs. in 'Systems Science and Science: Proceedings of 24th Annual North American Meeting.' Society of General Systems Resources, San Francisco, CA. pp 146-154.
- [7] WILLIAMS, M.K. (1988) ExpCheck - An Intelligent Diagnostic System in APL. Vector, Vol.4, No.3, pp83-88.
- [8] WILLIAMS, M.K. (1989) Strategic Selection Rules. In Preparation.

MicroAPL for ...

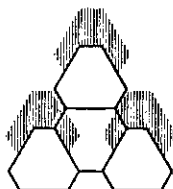
APL and networking consultancy

MicroAPL has 10 year's experience with small-computer APL systems. We have pioneered the usage of networked microcomputers and can plan, specify, install and maintain PC networks of all sizes. We have experience with both Ethernet and Token-ring networks and their mainframe gateways. Our programming skills cover all areas of APL and systems programming. MicroAPL can also train and supervise your APL staff on your behalf. If you would like to review your systems requirements, contact David Eastwood at:



MicroAPL Ltd.
South Bank Technopark
LONDON, SE1 6LN
Telephone: 01 922 8866

Access and Barclaycard
Orders accepted



APL PEOPLE

THE APL EMPLOYMENT AGENCY

- ★ Permanent and Contract Vacancies
- ★ UK and Overseas
- ★ Free Advice on career matters
- ★ Discreet, confidential service

Whether you are looking for staff or looking for
a job – we can help!

Contact: Jill Moss, APL People Ltd., The Old Malthouse
Clarence Street, Bath, England BA1 5NS

Tel: 0225 462602 during office hours
0225 333618 eves & w/ends

Or see her at the Conference!

TECHNICAL SECTION: Error Trapping

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

In this issue we concentrate on the error-trapping schemes used by three of the most popular APLs. These are presented as tutorials, and as far as possible, the authors have endeavoured to cover the material in a similar sequence. We hope that readers will find this presentation helpful when it comes to deciding which scheme most suits their particular requirements.

Contents

Hackers' Corner	Adrian Smith	90
Error Trapping Tutorials		
APL*PLUS	Dave Ziemann	91
APL.68000	David Eastwood	96
Dyalog APL	Pauline Brand	100
A Direct Definition Handler	Dave Ziemann	110
Semantic Class and Arrays of Functions	David Piper	118
APL Printing from APL.68000	Iain Hayward	127

Hacker's Corner: Where to Zap APL*PLUS Version-8

by Adrian Smith

I eventually became bored with the exercise of tracking down the banner in APLx.EXE, so I wrote a function to do it for me ...

```
R←STRING NFIND FI;LEN;OFF;IN
```

```
-----
[1]      A Returns position of <STRING> within DOS file (origin=0)
[2]      FI UNTIE -2 0 LEN←NUNSIZE -2 0 OFF←0 0 R←0
[3]      A Now check file contents in 4000-byte segments ...
[4]      LOOP:IN←NREAD -2 82 ,(4000[LEN-OFF],OFF
[5]      R←R,-1+OFF+(IN [SS STRING])/10IN
[6]      →(LEN≤OFF+4000)→DONE
[7]      OFF←OFF+4000 0 →LOOP
[8]      DONE:UNUNTIE -2
▽
```

Version-8 is a distinct improvement on its predecessors, as you only need to replace the banner. In earlier releases you had to obliterate the serial number and release information as well! It now has the good sense to detect that the banner has altered, and to suppress the version and serial information. This is probably just as well, as it checks for a valid number before it will load at all!

Here is the much simplified function to install your own signon banner:

```
BNRPOS PUTABAN BANNER
```

```
-----
[1]      A Patch in <BANNER> to file ΔEXE at <BNRPOS>.....
[2]      ΔEXE UNTIE -1
[3]      (631+BANNER,TCNL,6300TCNL)NREPLACE -1,BNRPOS
[4]      UNUNTIE -1
▽
```

```
ΔEXE←'ZAP6.EXE'      A .... set filename
140932 PUTABAN 'HELLO FROM APL6'  A Replace current banner
```

Maybe one day they will get the message and restore the B= option.

```
140932 '-----' NFIND 'ZAP6.EXE'  A Release 6
```

```
BNR←153429      A Vn8
BNR←102804      A Vn8 with SM
```

Error Trapping in APL*PLUS Systems A Tutorial Introduction

by Dave Ziemann (Cocking & Drury Ltd)

Background

The purpose of this tutorial is to review the error trapping tools available to users of APL*PLUS systems. The facilities described are standard throughout the whole family of APL*PLUS interpreters, and operate in the same way on all APL*PLUS systems.

What happens when an error occurs? Let us take as our example the following expression, which we know will result in an error:

```

      1 2 + 1 2 3
LENGTH ERROR
      1 2 + 1 2 3
      ^

```

Although it looks fairly straightforward, the system actually does quite a bit of work to get to that result. At this point we should distinguish between an error which is 'signalled' and an error which is 'reported'. When APL encounters an error, we say that an error is signalled. This does not necessarily mean that the error will be reported to the user, although the system's default behaviour is to do so, and in the above example this is exactly what happens.

In this tutorial we shall see how you can override the system's default behaviour by trapping signalled errors before they are reported. This is often called error trapping. What you do once you have trapped the error is entirely up to you of course; error handling is a much wider subject, beyond the scope of this article, as they say.

Checking the Diagnostic Message

If we are to stand any chance of handling an error we will need to know what error would have been reported by the system. This information can be produced by looking at the result of `□DM`, a niladic system function which records the last Diagnostic Message recorded in the workspace:

```

      □DM
LENGTH ERROR
      1 2 + 1 2 3
      ^

```

The diagnostic message produced above consists of three parts; the kind of error, the line that caused the error, and a caret that indicates the point in the line at which the error

was detected. You have to look quite carefully to see that typing `□DM` did not itself cause an error, but just displayed the diagnostic message. The diagnostic message is a newline delimited character vector; we can check this by looking at its shape:

```

    p□DM
42

```

Remember that the value produced by `□DM` will be different if a new error occurs.

Trapping the Original Error

So `□DM` can be used to handle our error, but how can we trap it in the first place? To do this we use the system variable `□ELX`, which contains the Error Latent Expression. As you might guess from its name, the error latent expression is a character vector which is executed by the system when an error is signalled. Because it is a system variable we can assign a value to it, for example:

```

    □ELX+'''YOU MADE AN ERROR'''
    □ELX
'YOU MADE AN ERROR'

```

Let us see what happens when we generate an error:

```

    1 2 + 1 2 3
YOU MADE AN ERROR

```

The system's default error trapping behaviour was overridden, and we caused our own message to be displayed. Not a very helpful error message you might think.

Of course, the system has a default value for `□ELX`, so that if you do not set it errors can still be reported. Can you see what that default value is?

Let us find out:

```

    □ELX
□DM

```

The default value for `□ELX` is the character vector `'□DM'`. So the system default behaviour is merely to display the diagnostic message. Notice that after the `□ELX` has been executed you are left in immediate execution mode.

We can embellish our error message by refining the value of `□ELX`, which can contain any APL expression. For example:

```

    □ELX+'''YOU MADE A ',□DM'
    □ELX
'YOU MADE A ',□DM

```

Let's try it out:

```

      1 2 + 1 2 3
YOU MADE A LENGTH ERROR
      1 2 + 1 2 3
      A

```

Remember that the diagnostic message contains the text of the signalled error which the system would have displayed:

```

      □DM
LENGTH ERROR
      1 2 + 1 2 3
      A

```

Another approach is to display the message and then execute an error processing routine. The purpose of this routine might be to log the error on a file somewhere, and inform the system steward. A suitable □ELX might be:

```
□ELX+ '□DM ◇ ERROR△PROCESS □DM'
```

What do you think would happen if an error occurred in the ERROR△PROCESS routine itself?

A good way of protecting against infinite loops in error handling code is always carefully to reset □ELX as soon as it is executed, for example:

```
□ELX+ '□ELX+ '□DM' ' ◇ □DM ◇ ERROR△PROCESS □DM'
```

Because □ELX is a system variable it can be included in the locals list of a defined function. In this way the scope of its effect can be controlled. When an error is signalled, it is the most local value of □ELX which is executed.

We have seen how □ELX and □DM can be used to control the trapping of APL errors in a general way.

Signalling Your Own Errors

Another useful tool is the system function □ERROR, which allows you to signal your own errors. Why should you want to signal your own errors, when you probably get enough already?

As an example consider the function `SQRT` which takes the square root of a number:

```

      ▽ Z+SQRT W
[1]  A <Z> is the square root of <W>
[2]
[3]  □ERROR(1≤W<0)/'ARGUMENT NEGATIVE'
[4]  Z+W*0.5
      ▽

```

If `<W>` is not negative, then the argument to `□ERROR` is an empty vector, and nothing happens; execution continues on line 4. If however, `<W>` is negative, then `□ERROR` will cause an error to be signalled. The error message signalled is our own tailored message 'ARGUMENT NEGATIVE'. Let's see it in action:

```

      SQRT 4
2
      SQRT -4
ARGUMENT NEGATIVE
      SQRT -4
      ^

```

Notice that the error was signalled in the environment that called the function `SQRT`, not within the function `SQRT` itself; we have made `SQRT` behave like a primitive APL function in this respect. We can confirm this by checking the `)SI` stack:

```
)SI
```

The `)SI` stack is still clear. The `□DM` contains our own error message, exactly as signalled:

```

      □DM
ARGUMENT NEGATIVE
      SQRT -4
      ^

```

Of course there are other errors which could occur with `SQRT`, for example we might pass it character data. Can you see how `□ERROR` can be used to pass any APL error back to the calling environment?

The following version of `SQRT` retains our own tailored message, and additionally passes all other errors to its caller:

```

      ▽ Z+SQRT W;□ELX
[1]  A <Z> is the square root of <W>
[2]
[3]  □ELX+'□ERROR ( ^\□DM≠□TCNL )/□DM'
[4]  □ERROR(1≤W<0)/'ARGUMENT NEGATIVE'
[5]  Z+W*0.5
      ▽

```

```

      SQR T 4
2    SQR T -4
ARGUMENT NEGATIVE
      SQR T -4
      ^

      SQR T 'HELLO'
DOMAIN ERROR
      SQR T 'HELLO'
      ^

```

Line 3 is a common APL*PLUS error trapping idiom; it causes any error which occurs within SQR T to be signalled in the calling environment. The expression (^\ODM#TCNL)/ODM extracts the name of the error from the diagnostic message which is produced when the error is signalled.

Notice that OELX has been localised in order to ensure that its effect is restricted to SQR T, and any of its sub-functions. The following question is left for the reader: What do you think happens if SQR T calls a sub-function in which OELX is not localised or assigned, and an error occurs within it?

To recap; the system names OELX, ODM and OERROR are all you need to successfully trap and handle APL errors in APL*PLUS systems. Error trapping is powerful, and open to misuse and abuse. A wide range of error handling tools and appropriate techniques can be developed from these primitives, topics which have not been covered here.

Error Trapping in APL.68000 A Tutorial Introduction

by David Eastwood

Background

When developing functions, or indeed running programs that one has written oneself, the ability to examine suspended functions and alter them before resuming execution is one of the major attractions of APL as a programming environment. When a system has been handed over to end-users (even other programmers) or indeed has been transformed into a packaged piece of software it is then unsatisfactory to report an APL error message and exit to calculator mode. It is almost certainly preferable to issue some helpful message and either attempt to correct or record the exact source of the error.

Trapping Simple Errors

APL.68000 has a simple but highly effective scheme to modify or control standard APL error reporting. A group of system functions are defined which allow the programmer to specify what actions to take in the event of an error, and which pass information to the programmer about the source and nature of an error. In addition the usual error messages may be altered or new ones introduced. The functions are:

Function	Operation
⌈ERX	Error trap
⌈LER	Last Error Number and source
⌈ERM	Last Error Message
⌈ERS	Error signal

The functions are complementary in the sense that a programmer will probably need to use them all to implement an effective scheme of error trapping.

The error trap is set via the system function ⌈ERX:

```
      VFOO  
[1] Z+⌈ERX LABEL      a Z will contain the previous value of ⌈ERX
```

Control will pass to label when an error occurs in this function, or a called function which does not have error trapping set, and the function returns the previous trap value. When an error occurs, the normal error display of error message and line number is suppressed. A line number of 0 suppresses the trap.

A non-error trapped function, called by an error trapped function, will behave as if it is locked. A branch will again take place to the designated line in the calling function.

Having transferred execution to an error handling routine, it is important to know the type of error that has occurred and also where it occurred. Sometimes the APL function can attempt some sort of corrective action, but often the error is logged and some message passed to the user.

The function `⌈LER` returns the error code number (see appendix) and the line where the error took place, as a two element vector. If, when error trapping is active, an error occurs, the line number will refer to the most recent function to which the error has been propagated (i.e. the error trapped function).

To read the error message, the system function `⌈ERM` shows the character vector that APL68000 would normally print with a Carriage Return (`␣`) between lines. `⌈ERM` only works if error trapping is set. If error trapping is not set, `⌈ERM` returns an empty vector. Inside a locked function, `⌈ERM` will show the error message that would be displayed if the function were unlocked. Outside a locked function, however, `⌈ERM` is set to be an empty vector for security reasons.

```

▽ FOO;Z:ER
[1]  Z+⌈ERX ERR
[2]  100×'A'
[3]  'THIS LINE WILL NOT BE REACHED'
[4]  ERR:'INTERNAL PROGRAM ERROR'
[5]  ER+␣R ⌈BOX ⌈ERM a form the ⌈ERM vector into a matrix
[6]  'ERROR MESSAGE: ',(ER[1;]),' ON FUNCTION LINE ',▽⌈LER[2]
▽

```

Example of Error Trapping

It is quite common to encounter the error message `WS FULL` if you try to carry out operations using large arrays. Rather than have your function stop, you might like to check for this error state and undertake corrective action.

```

▽ ADDUP;X;DATA
[1]  A Adds up all the numbers up to that entered
[2]  X←□ERX ERR A X is used to hide the return from □ERX
[3]  START:'ENTER A NUMBER'
[4]  DATA+L□ A Make it the next lowest integer
[5]  'THE SUM OF THE FIRST',(▼DATA), ' NUMBERS IS: '
[6]  +/1DATA
[7]  →0 A End of the function
[8]  ERR:→(1≠1+□LER)/REALERR A err code 1 is ws full
[9]  'THE NUMBER YOU ENTERED WAS TOO BIG TO USE, TRY AGAIN'
[10] →START
[11] REALERR: A An error has occurred which is not ws full
[12] 'ERROR TYPE',(▼1+□LER), ' ON LINE ',▼1+□LER
[13] 'MESSAGE IS:'
[14] □ERM
▽

```

WARNING

Make sure that you have some escape route from the error trap routine, otherwise any error within that section of the function will cause an uninteruptible loop. (The Interrupt key, CTRL-C, also causes an error - type 13!)

Error Signalling

If some error report is to be made to the user of the system, it is useful to be able to modify the usual APL error messages, which may not be very meaningful to the end user. This can be carried out by the function □ERS. □ERS can be used to force an standard APL error report, or, if used with a character left argument, it will display those characters and assign the error code in its right argument to □LER.

Here is an example where □ERS is used to make sure that the user hasn't hit the interrupt key accidentally:

```

▽ FOO;DATA
[1]  □ERX ERR A set error trap
[2]  L:'ENTER YOUR EXPRESSION'
[3]  DATA+□
[4]  'THE RESULT IS:'
[5]  *DATA
[6]  →L
[7]  ERR:→(13≠1+□LER)/NOTINT A not interrupt
[8]  'DID YOU MEAN TO HIT INTERRUPT? (Y/N)'
[9]  →('Y'≠1+□)/L □ □ERS 13 A signal if confirmed
[10] NOTINT:□ERS 1+□LER A signal other errors
▽

```

If the right argument is a number in the range 1 to 51, □ERS will display the standard APL.68000 error message. As you will see later in the chapter, some error numbers are

undefined, and in these cases `□ERS` will display unknown error type signalled. If the right argument is an empty vector, no error is signalled. Used with a character left argument, the error message may be altered. An empty vector left argument (") will suppress the error message.

```

      ▽ R←AV B
[1]   'NUMERIC ARGUMENT PLEASE' □ERS (4=□DR B)/8
[2]   R←(+/B)÷ρB
      ▽

      AV 1 2 3
2      AV 'ABC'
NUMERIC ARGUMENT PLEASE
      AV
      ^

      □LER
8 0    □SI          .... no response (the function has been halted)

```

Appendix: Error Code Numbers

Errors within APL68000 are allocated codes in the range 1 to 51. Some are undefined, but may still be used by `□ERS`.

1	ws full	20	component not in file
2	syntax error	21	file allocation exceeded
3	index error	23	file maintenance in progress
4	rank error	24	file or component held
5	length error	25	incorrect command, limit error
6	value error	26	data damaged
11	domain error	27	user not in system
13	interrupt	28	user allocation exceeded
16	nonce error	29	file in existence
17	file i/o error	40	disc full
18	file not in system	41	file locked
19	unauthorised file access	42	logical unit not found

Error Trapping in Dyalog APL A Tutorial Introduction

by Pauline Brand (Dyadic Systems Ltd)

Background

We often give courses to people who taught themselves APL and now have several years' programming experience to their credit, and we have found that a surprising number of them have not discovered some of the "non-language" features available in most APLs, in particular "error trapping" or "event handling".

The purpose of this article is to show some of the ways in which the ideas of error trapping can be used to great effect to change the flow of control in a system.

Introduction to Error Trapping

Most APLs have error trapping facilities in one form or another, but this article discusses the facilities available to a Dyalog APL programmer.

First, we must have an idea of what is meant by error trapping. We are all used to entering some duff APL code and seeing a (sometimes) rather obscure, esoteric error message echoed back:

```
10÷0
DOMAIN ERROR
10÷0
A
```

Now, these sorts of error messages are fine for us clever APL programmers, but meaningless to most of our users. We need to find a way to bypass the default action of APL, so that we can take an action of our own.

Every error message reported by Dyalog APL has a corresponding error number (see the Appendix). Many of these error numbers plus messages are common across all versions of APL. The code for DOMAIN ERROR is 11, whilst LENGTH ERROR has code 5.

Dyalog APL provides system functions and variables that allow you to find out this information without looking it up in the table:

```

10÷0
DOMAIN ERROR
10÷0
A

    ⍋EN                                A ERROR NO FOR LAST ERROR
11

    ⍋EM 11                             A ERROR MESSAGE FOR CODE 11
DOMAIN ERROR

    ⍋DM                                A DIAGNOSTIC MESSAGE
DOMAIN ERROR 10÷0 A

                                A 3-ELEMENT NESTED VECTOR OF
                                A MESSAGE, NUMBER AND CARAT

    DISPLAY ⍋DM                       A DISPLAY TO SHOW STRUCTURE

+-----+
| DOMAIN ERROR | 10÷0 | A |
+-----+

    ⍋⍋DM                                A MIX (†) OF VECTOR GIVES A
DOMAIN ERROR                          A SIMPLE MATRIX, WHICH IS
10÷0                                  A THE SAME AS THE ERROR MSG
A                                     A DISPLAYED BY APL

```

We also need a facility to tell APL "whenever error 11 occurs, instead of doing what you normally do, take this action instead". In Dyalog APL, we use the system variable `⍋TRAP`, which can be assigned a nested vector of trap specifications. Each trap specification is itself a nested vector, of length 3, with each element defined as:

- list of error number(s): the error numbers we are interested in.
- action code: Either 'E' (Execute) or 'C' (Cut Back). There are others, but they are seldom used.
- action to be taken: APL expression, usually a branch statement or a call to an APL function.

Setting a Simple Trap

So a single trap specification may be set up as:

```
⌈TRAP+5 'E' 'ACTION1'
```

and a multiple trap specification may be set up as:

```
⌈TRAP+(5 'E' 'ACTION1')((1 2 3 4) 'C' 'ACTION2')
```

The action code **E** tells APL that you want your action to be taken in the function in which the error occurred, whereas the code **C** indicates that you want your action to be taken in the function where the $\lceil$ TRAP was set; if necessary, APL must first travel back up the SI stack (cut-back) until it reaches that function. These action codes are best illustrated by example.

Let's try setting a $\lceil$ TRAP on DOMAIN ERROR:

```
⌈TRAP+11 'E' ' ' 'PLEASE GIVE A NON-ZERO RIGHT ARGUMENT' ' '
```

When we enter:

```
10÷0
```

APL executes the expression, and notes that it causes an error number 11. Before issuing the standard error, it scans its $\lceil$ TRAP table, to see if you were interested enough in that error to set a trap; you were, so APL executes the action specified by you:

```
10÷0
PLEASE GIVE A NON-ZERO RIGHT ARGUMENT
```

Let's reset our $\lceil$ TRAP:

```
⌈TRAP+0⌈⌈TRAP                                A NO TRAPS NOW SET
```

and write a function to take the place of the intrinsic $\div$...

```

V R←A DIV B
[1] R←A÷B
[2] V
```

Then run it:

```

10 DIV 0
DOMAIN ERROR
DIV[1] R←A÷B
^
```

Let's edit our function and include a localised `□TRAP`:

```

      ▽ R←A DIV B;□TRAP
[1]  A SET THE TRAP
[2]  □TRAP←11 'E' '→ERR1'
[3]  A DO THE WORK; IF IT RESULTS IN ERROR 11, EXECUTE THE TRAP
[4]  R←A÷B
[5]  A ALL OK IF WE GOT TO HERE, SO EXIT
[6]  →0
[7]  A WILL GET HERE ONLY IF ERROR 11 OCCURRED
[8]  ERR1:'PLEASE GIVE A NON-ZERO RIGHT ARGUMENT'
      ▽

```

Running the function with good and bad arguments has the desired effect:

```

      10 DIV 2
5
      10 DIV 0
PLEASE GIVE A NON-ZERO RIGHT ARGUMENT

```

`□TRAP` is a variable like any other, and since it is localised in `DIV`, it is only effective in `DIV` and any other functions that may be called by `DIV`. So

```

      10 DIV 0
DOMAIN ERROR
      10÷0
      A

```

still gives an error, since there is no trap set in the global environment.

What happens to our function if we run it with other duff arguments:

```

      1 2 3 DIV 4 5
LENGTH ERROR
DIV[4] R←A÷B
      A

```

Here is an error that we have taken no account of. Change `DIV` to take this new error into account:

```

      ▽ R←A DIV B;□TRAP
[1]  A SET THE TRAP
[2]  □TRAP←(11 'E' '→ERR1')(5 'E' '→ERR2')
[3]  A DO THE WORK; IF IT RESULTS IN ERROR 11, EXECUTE THE TRAP
[4]  R←A÷B
[5]  A ALL OK IF WE GOT TO HERE, SO EXIT
[6]  →0
[7]  A WILL GET HERE ONLY IF ERROR 11 OCCURRED
[8]  ERR1:'PLEASE GIVE A NON-ZERO RIGHT ARGUMENT' ◇ →0
[9]  A WILL GET TO HERE ONLY IF ERROR 5 OCCURRED
[10] ERR2:'ARGUMENTS MUST BE SAME LENGTH'
      ▽

```

```

)RESET
1 2 3 DIV 4 5
ARGUMENTS MUST BE THE SAME LENGTH

```

But here's another problem that we didn't think of:

```

(2 3p16) DIV (2 3 4p124)
RANK ERROR
DIV[4] R+A÷B
  ^

```

Often when we're writing a system, we can't think of everything that may go wrong ahead of time; so we need a way of catching "everything else that I may not have thought of". The error number used for "everything else" is zero:

```

)RESET
[TRAP + 0 'E' ' ' 'INVALID ARGUMENTS' ' ' ^ SET GLOBAL TRAP
(2 3p16) DIV (2 3 4p124) ^ AND RUN FUNCTION
INVALID ARGUMENTS

```

In this case, when APL executed line 4 of our function DIV, it encountered an error number 4 (RANK ERROR). It searched the local trap table, found nothing relating to error 4, so searched further up the stack to see if the error was mentioned anywhere else. It found an entry with an associated Execute code, so executed the appropriate action AT THE POINT THAT THE ERROR OCCURRED. Let's see what's in the stack:

```

)SI
DIV[4]*
  ^
  ^DM
RANK ERROR
DIV[4] R+A÷B
  ^

```

So although our action has been taken, execution has stopped where it normally would after a RANK ERROR.

Beware of Global Traps!

We must be careful when we set global traps; let's call the non-existent function BUG whenever we get an unexpected error:

```

)RESET
[TRAP + 0 'E' 'BUG'
(2 3p16) DIV (2 3 4p124)

```

Nothing happens, since APL traps a RANK ERROR on line 4 of DIV, so executes the trap statement, which causes a VALUE ERROR, which activates the trap action, which causes a VALUE ERROR, which etc., etc. If we had also chosen to trap on 1000 (ALL INTERRUPTS), then we'd be in trouble!

Let's define a function BUG:

```

V BUG
[1] A CALLED WHENEVER THERE IS AN UNEXPECTED ERROR
[2] '*** UNEXPECTED ERROR OCCURRED IN FUNCTION',␣1+␣SI
[3] '*** PLEASE CALL YOUR SYSTEM ADMINISTRATION'
[4] '*** WORKSPACE SAVED AS BUG.',␣1+␣SI
[5] A TIDY UP ... RESET ␣LX, UNTIE FILES ... ETC
[6] ␣SAVE 'BUG.',␣1+␣SI
[7] '*** LOGGING YOU OFF THE SYSTEM'
[8] ␣OFF
V

```

Now, whenever we run our system and an unexpected error occurs, our BUG function will be called.

```

10 DIV 0
PLEASE GIVE NON-ZERO RIGHT ARGUMENT

(2 3p16) DIV (2 3 4p112)

*** UNEXPECTED ERROR OCCURRED IN FUNCTION DIV
*** PLEASE CALL YOUR SYSTEM ADMINISTRATOR
*** WORKSPACE SAVED AS BUG.DIV
*** LOGGING YOU OFF THE SYSTEM

```

The system administrator can then load BUG.DIV, look at the SI stack, discover the problem, and fix it.

In many cases, you can of course achieve the same effect of a trap by using APL code to detect the problem before it happens. Consider the function TIEΔFILE, which checks to see if a file already exists before it tries to access it:

```

V R←TIEΔFILE FILE;FILES
[1] A TIE FILE FILE WITH NEXT AVAILABLE TIE NUMBER
[2] FILES←␣FLIB 'MYDIR' A ALL FILES IN MY DIRECTORY
[3] FILES←DBR"␣FILES A REMOVE TRAILING BLANKS
[4] →ERRX1~(␣FILE)␣FILES A REQD FILE IN LIST?
[5] FILE ␣TIE R+1+␣/0,␣FNOMS A TIE FILE WITH NEXT NUMBER
[6] →0 A ... AND EXIT
[7] ERR:R+'FILE DOES NOT EXIST' A ERROR MESSAGE
V

```

This function executes the same code whether the file name is right or wrong, and it could take a while to get all the file names in your directory. It would be neater and more efficient to take action ONLY when the file name is wrong:

```

VR+TIEΔFILE FILE;□TRAP
[1]  A TIE FILE FILE WITH NEXT AVAILABLE TIE NUMBER
[2]  □TRAP+22 'E' 'ERR'          A SET TRAP
[3]  FILE □TIE R+1+[/O,FNUMS      A TIE FILE WITH NEXT NUMBER
[4]  +0                          A ... AND EXIT IF OK
[5]  ERR:R+'FILE DOES NOT EXIST'  A ERROR MESSAGE
V

```

Cutting Back the Stack

Let us consider the effect of using Cut-Back instead of Execute. Consider the system illustrated below, in which the function REPORT gives the user the option of 4 reports to be generated:

```

REPORT

REP1      REP2      REP3      REP4

... DIV ...

```

where REPORT looks something like this:

```

V REPORT;OPTIONS;OPTION;□TRAP
[1]  A DRIVER FUNCTIONS FOR REPORT SUB-SYSTEM. IF AN
[2]  A UNEXPECTED ERROR OCCURS, TAKE ACTION IN FUNCTION WHERE
[3]  A ERROR OCCURRED
[4]  □TRAP+0 'E' 'BUG'          A SET GLOBAL TRAP
[5]  OPTIONS+'REP1' 'REP2' 'REP3' 'REP4' A AVAILABLE OPTIONS
[6]  LOOP:→ENDX10=pOPTION+MENU OPTIONS A ASK USER TO CHOOSE
[7]  →OPTION                    A EXECUTE RELEVANT FUNCTION
[8]  →LOOP                      A REPEAT UNTIL EXIT
[9]  END:                        A NOW END
V

```

Suppose the user chooses REP3, and an unexpected error occurs in DIV.

The good news is that the system administrator gets a snapshot copy of the workspace that he can play with:

```

        )LOAD BUG.DIV                A LOAD WORKSPACE
SAVED .....

        )SI                          A WHERE DID ERROR OCCUR?
DIV[4]*
REP3[6]
↑
REPORT[7]

        ↑DM                          A WHAT HAPPENED?
RANK ERROR
DIV[4] R+A÷B
        A

        V                            A EDIT FUNCTION ON TOP OF
STACK
[0] R+A DIV B
.....

```

The bad news is, our user is locked out of the whole system, even though it may only be REP3 that has a problem. We can get around this by making use of the CUT-BACK action code:

```

V REPORT;OPTIONS;OPTION;[]TRAP
[1] A DRIVER FUNCTIONS FOR REPORT SUB-SYSTEM. IF AN
[2] A UNEXPECTED ERROR OCCURS, CUT THE STACK BACK TO THIS
[3] A FUNCTION, THEN TAKE ACTION
[4] []TRAP+0 'C' '↑ERR'                A SET GLOBAL TRAP
[5] OPTIONS+ 'REP1' 'REP2' 'REP3' 'REP4' A AVAILABLE OPTIONS
[6] LOOP:→ENDX10=POPTION+MENU OPTIONS A ASK USER TO CHOOSE
[7] ↑OPTION                A EXECUTE RELEVANT FUNCTION
[8] →LOOP                A REPEAT UNTIL EXIT
[9] ERR:MESSAGE'UNEXPECTED ERROR IN',OPTION A TELL USER
[10] MESSAGE:'REMOVING FROM LIST' A ... WHAT'S HAPPENING
[11] OPTIONS+ OPTIONS ~<OPTION A REMOVE OPTION FROM LIST
[12] →LOOP                A ... AND REPEAT
[13] END:                A END
V

```

Suppose the user runs this version of REPORT and chooses REP3. When the unexpected error occurs in DIV, APL will check its trap specifications, and see that the relevant trap was set in REPORT with a cut-back code. APL therefore cuts back the stack to the function in which the trap was set, THEN takes the specified action. Looking at the SI stack above, we can see that APL must jump out of DIV, then REP3, then ↑, to return to line 7 of REPORT; THEN it takes the specified action.

User-defined Events

It would be useful to be able to employ this idea of cutting back the stack and taking an alternative route through the code, when a condition other than an APL error occurs. To achieve this, we must be able to trap on events other than APL errors, and we must be able to define these events to APL. We do the former by using event codes in the range 500 to 999, and the latter by using `DSIGNAL`.

Consider our system; ideally, when an unexpected error occurs, we want to save a snapshot copy of our workspace (execute `BUG` in place), then immediately jump back to `REPORT` (cutback on an event, not an APL error) and reduce our options. We can achieve this by changing our functions a little, and using `DSIGNAL`:

```

V REPORT;OPTIONS;OPTION;DTRAP
[1]  A DRIVER FUNCTIONS FOR REPORT SUB-SYSTEM. IF AN
[2]  A UNEXPECTED ERROR OCCURS, MAKE A SNAPSHOT COPY OF THE
[3]  A WORKSPACE THEN CUTBACK THE STACK TO THIS FUNCTION
[4]  DTRAP+(0 'E' 'BUG')(500 'C' '+ERR' A SET GLOBAL TRAP
[5]  OPTIONS+'REP1' 'REP2' 'REP3' 'REP4' A AVAILABLE OPTIONS
[6]  LOOP;+ENDX10=POPTION+MENU OPTIONS A ASK USER TO CHOOSE
[7]  DOPTION A EXECUTE RELEVANT FUNCTION
[8]  +LOOP A REPEAT UNTIL EXIT
[9]  ERR:MESSAGE'UNEXPECTED ERROR IN',OPTION A TELL USER
[10] MESSAGE:'REMOVING FROM LIST' A ... WHAT'S HAPPENING
[11] OPTIONS* OPTIONS ~DOPTION A REMOVE OPTION FROM LIST
[12] +LOOP A AND REPEAT
[13] END; A END
V
V BUG
[1]  A CALLED WHENEVER THERE IS AN UNEXPECTED ERROR
[2]  '*** UNEXPECTED ERROR OCCURRED IN FUNCTION',D1+DSI
[3]  '*** PLEASE CALL YOUR SYSTEM ADMINISTRATOR'
[4]  '*** WORKSPACE SAVED AS BUG,',D1+DSI
[5]  A TIDY UP ..., RESET DLX, UNTIE FILES ... ETC
[6]  DSAVE 'BUG,',D1+DSI
[7]  '*** RETURNING YOU TO DRIVER FUNCTION FOR RESELECTION'
[8]  DSIGNAL 500
V

```

Now when the unexpected error occurs, the first trap specification catches it, and the `BUG` function is executed in place. Instead of logging the user off as before, an event 500 is signalled to APL. APL checks its trap specifications, sees that 500 has been set in `REPORT` as a cut-back, so cuts back to `REPORT` before branching to `ERR`.

Event handling, which employs a combination of all the system functions and variables described, allows us dynamically to alter the flow of control through our system, as well as to handle errors gracefully. It is a very powerful facility, which is simple to use but is often neglected.

Appendix: The Full Error-code Table

Code	Event
0	Any event in range 1-999
1	WS FULL
2	SYNTAX ERROR
3	INDEX ERROR
4	RANK ERROR
5	LENGTH ERROR
6	VALUE ERROR
7	FORMAT ERROR
10	LIMIT ERROR
11	DOMAIN ERROR
12	HOLD ERROR
15	LST FULL
16	NONCE ERROR
18	FILE TIE ERROR
19	FILE ACCESS ERROR
20	FILE INDEX ERROR
22	FILE NAME ERROR
23	FILE DAMAGED
24	FILE TIED
26	FILE SYSTEM ERROR
28	FILE SYSTEM NOT AVAILABLE
30	FILE SYSTEM TIES USED UP
31	FILE TIE QUOTA USED UP
32	FILE NAME QUOTA USED UP
34	FILE SYSTEM NO SPACE
38	FILE COMPONENT DAMAGED
72	NO PIPES
73	LOST SLAVE
76	PROCESSOR TABLE FULL
84	TRAP ERROR
200 - 499	Reserved for distributed auxiliary processors
500 - 999	User-defined events
1000	Any event in range 1001-1006
1001	Stop vector
1002	Weak interrupt
1003	INTERRUPT
1004	INPUT INTERRUPT
1005	EOF INTERRUPT
1006	TIMEOUT

An Unusual Error Handling Application: A Direct Definition Handler

by Dave Ziemann (Cocking & Drury Ltd)

Abstract

This article describes an unusual application for error handling in the APL*PLUS II system for 386 based machines, namely a facility for interpreting direct definition statements in immediate execution mode. The direct definition facility is described first, followed by an exposition of its implementation using the available error handling techniques.

Introducing Direct Definition

Direct definition is a notation for associating an APL statement with a name, for example:

```
TWICE: 2×ω
```

A function TWICE is thereby defined, which multiplies its argument by two. The colon separates the function name from its definition, and the ω (omega) represents the argument. The function can then be used:

```
TWICE 1 2 3
2 4 6
```

If an α (alpha) appears in the definition the function can be called dyadically, the α representing the function left argument. A niladic function is defined when neither α nor ω appear. For example:

```
PLUS: α+ω
PI: 01
1 PLUS TWICE PI
7.283185307
```

A three-segment form of definition is used to allow one of two alternative statements to be conditionally executed:

```
FACT: ω : ω>1 : ω×FACT ω-1
FACT 10
3628800
```

The middle segment, the condition segment, is executed first. It must evaluate to a 0 or 1. If its value is 0 then the left segment is executed. If the value of the condition segment is

1 then the statement in the right segment is executed instead. The three-segment form is often used with recursion, or to determine if a left argument has been supplied.

A definition can be displayed by typing its name followed by a colon, for example:

```
PLUS:
PLUS:  $\alpha + \omega$ 
```

Essentially, that's all there is to direct definition. A future issue of Vector may well contain an article describing the APL*PLUS II Direct Definition Handler facility, its uses and benefits in detail. The remainder of this article is concerned with how the APL*PLUS II error trapping tools are used to make the facility work.

Why is Direct Definition so Scarce?

Although various forms of direct definition have been used as a notation for a long time, its use in programming commercial APL applications has been very limited. There are two main reasons for this.

Firstly, direct definition has only traditionally been made available through a set of cover functions which perform the appropriate operations, for example:

```
DD 'PLUS:  $\alpha + \omega$ '
PLUS
DD 'PLUS: '
PLUS:  $\alpha + \omega$ 
```

The use of direct definition cover functions makes the tool unwieldy; you have to type the cover function name followed by the definition inside quotes. If your definition contains quotes then you have to double them up.

Secondly the commercial APL vendors have not included direct definition as a primitive facility in their interpreters. The public domain interpreter I-APL, which was written by Paul Chapman, is the only APL system which does do this, but I-APL is not appropriate for commercial work.

What is needed is a fast, commercial APL which either provides direct definition as a primitive facility, or allows for its provision in a natural way.

A single source for APL*PLUS

APL*PLUS
PC



for PCs
and the PS2

APL*PLUS
UNIX



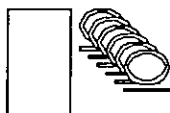
for UNIX
systems

APL*PLUS
PRO



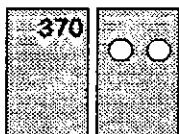
for the PS2
model 80

APL*PLUS
VMS



for the
VAX range

APL*PLUS
VM & MVS



for IBM
mainframes

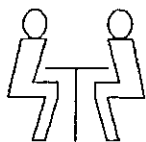


COCKING & DRURY LTD.

THE APL PROFESSIONALS

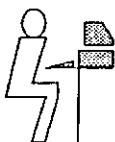
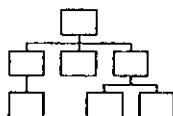
180 Tottenham Court Road,
London W1P 9LE
Tel: 01 - 436 9481

A single source for APL



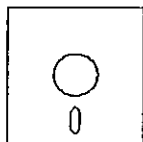
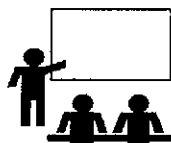
Advice

Design



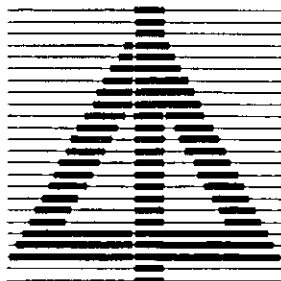
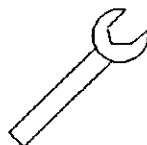
Application
Development

Education



APL software

Support



COCKING & DRURY LTD.

THE APL PROFESSIONALS

180 Tottenham Court Road,
London W1P 9LE

Tel: 01 - 436 9481

Direct Definition in APL*PLUS II

The APL*PLUS II system for 386 machines satisfies the latter condition, and for an interesting reason, which we now explore.

In APL defined functions, the form

```
label:line
```

is understood to be a labelled line, and we certainly do not want to change that. What is the meaning of this form in immediate execution mode?

Most APL systems actually tolerate this form in immediate execution mode, without reporting an error of any kind. What happens is that the line is executed and the label ignored. The draft APL standard (hopefully soon to be published by the International Standards Organisation) reports an error for this construct, so the observed behaviour (allowing a label) is a perfectly legal consistent extension, but of course a pointless one.

The APL*PLUS II system however, is close to the formal model in the standard in that it signals an error when a labelled line is typed in immediate execution mode, as follows:

```
LAB: 1+2
SYNTAX ERROR
LAB: 1+2
^
```

Here comes the good bit - this error can be trapped, analysed and then reinterpreted as a potential direct definition statement. All that is needed is a suitable global `⍉ELX` and an appropriate APL function to do the analysis. This function is called `DDHANDLE`, and it needs to be in the workspace. Here is what the `⍉ELX` looks like after the Direct Definition Handler has been enabled:

```
DDHANDLE 'ON'
The Direct Definition Handler is now on.

⍉ELX
⍺{~DDHANDLE ⍵DM}/'⍵DM'
```

Let's consider what happens with this error latent expression when an error occurs. When the error is signalled by the system, the diagnostic message is passed into the analysis function `DDHANDLE`. Then one of two things can happen, dependent upon whether the diagnostic message contains a potential direct definition statement or not. The diagnostic message is considered a potential direct definition statement if it contains a caret that points to a colon, as above.

Let us first suppose that this condition is not met; in this case DDHANDLE will return a 0, $\square$ DM is executed, and the diagnostic message is reported to the user in the normal way:

```

      1 2 + 1 2 3
LENGTH ERROR
      1 2 + 1 2 3
      ^

```

If the diagnostic message does correspond to an attempted direct definition statement then there are again two possibilities; if the statement is not a valid direct definition request, an appropriate error message (DDEFN ERROR) is signalled in the calling environment by DDHANDLE, using the $\square$ ERROR system function. For example:

```

      A B: 1+2
DDEFN ERROR
      A B: 1+2
      ^

```

If the diagnostic message contains a valid direct definition statement however, then DDHANDLE returns 1, an empty vector is executed, and no error report is made. In this case DDHANDLE also quietly defines an APL function whose name and definition are determined by the definition request entered, e.g.:

```

      DAB: w~' ' ^ Delete All Blanks
      )FNS
      DAB DDHANDLE FACT PI PLUS TWICE

```

In this way error-signalling APL statements which are not considered to be potential direct definition requests are transparent to the Direct Definition Handler, and are reported by APL in the way that the user expects.

In fact, thanks to a suggestion by James A Brown, the Direct Definition Handler does not even prevent you from having your own specialised error handling routine active at the same time. Suppose we want to embellish the system's usual error reporting with a bell sound and a more friendly message. We might use a $\square$ ELX like this:

```

       $\square$ ELX+ $\square$ 
       $\square$ SOUND 1 2p200  $\diamond$  'YOU MADE A ', $\square$ DM

```

It works like this:

```

      1 2 + 1 2 3
YOU MADE A LENGTH ERROR
      1 2 + 1 2 3
      ^

```

You probably didn't hear the bell ring, but you can see the message.

But now we also want to use direct definition. Lets enable the handler and take a look at the `DELX`:

```
DDHANDLE 'ON'
The Direct Definition Handler is now on.

DELX
⚡(~DDHANDLE DDM)/'SOUND 1 2p200 ⋄ 'YOU MADE A ',DDM'
```

Your own `DELX` got imbedded in the new one, but it will only be executed if a non direct definition error occurs, as shown here:

```
DAB:
DAB: ω~' ' A Delete All Blanks

A B: 1+2
DDEFN ERROR
A B: 1+2
A

1 2 + 1 2 3
YOU MADE A LENGTH ERROR
1 2 + 1 2 3
A
```

The handler can be turned off to restore the prior value of `DELX` as follows:

```
DDHANDLE 'OFF'
The Direct Definition Handler is now off.

DELX
SOUND 1 2p200 ⋄ 'YOU MADE A ',DDM
```

Using Directly Defined Functions Inside Other Programs

Perhaps you want to use direct definition inside APL programs. Clearly it's not possible to type a definition directly because your entry will be interpreted as a labelled line, as explained at the start of this article. To do this we must use a support function for defining functions on the fly.

It's very useful to be able to define a function directly when you want to apply it to a nested array with the 'each' operator. As a trivial example, consider the problem of finding the rank of each item in a nested array. We could write:

```
RE←"p"p"ARRAY
```

but this requires three eaches (loops) and is not in general easy to read.

The alternative is to define a function RANK and to apply it to each item of the argument, as follows:

```
DDdef 'RANK: ρρω' ⋄ RE←RANK"ARRAY
```

The function DDdef establishes the function defined in its argument; you can then use it immediately. Uses of this technique will be covered in a future article; for now we will stick to error handling. Can you see how DDdef might work?

Here is its definition:

```
DDdef: ⚡ Δ □ELX←□ELX, ' ⋄ →0' Δ DDHANDLE 'ON' ⚡/NR
```

```
Δ:α
```

The function Δ merely returns its left argument and so acts as a right-to-left operating statement separator.

Firstly the handler is enabled by running DDHANDLE 'ON'. This establishes the appropriate □ELX, which is then modified so that DDdef is not left on the SI stack when the execute completes. Finally, the direct definition statement itself, is executed. This of course signals an error which is picked up by the handler, and the function is defined.

Don't worry, the handler automatically localises any variable names that appear directly to the left of the assignment arrow, so setting □ELX within DDdef does not modify its global value.

The ⚡/NR tells the handler not to extract an explicit result from the execute primitive, which in this case will not produce one.

Summary

The APL*PLUS II Direct Definition Handler has proved to be a powerful and versatile tool. The two factors that made it possible are the APL*PLUS error trapping facilities and the interpreter's behaviour for the label form in immediate execution mode. The three factors that make it a joy to use are the absence of direct definition cover-functions, the fact that only a single support function is required in the workspace, and the speed of the 386 hardware.

Semantic Class and Arrays of Functions

by D. B. Piper

Abstract

Many papers have already been published which discuss arrays of functions in APL. Some of the most significant are listed in the references at the end of this paper. Landaeta (LA86, LA88) and Bernecky (BE84) have made fairly rigorous proposals for APL2 and Sharp APL type interpreters respectively.

The basic concepts behind arrays of functions are discussed first. Landaeta's proposal (LA86) is re-presented and criticised. Finally, the concept of semantic class is introduced, with an appropriate mechanism for the manipulation of arrays of functions.

Introduction

APL supports operations on arrays of data:

```

      1 2 3 × 4 5 6
4 10 18

```

Scalar extension allows a constant to be specified as a scalar:

```

      2 × 4 5 6
8 10 12

```

An implementation that supports strand notation allows an expression of the sort:

```

      2 3 4 + 5 6 7          ..(1)
7 9 11

```

to be re-written as:

```

      (2+5)(3+6)(4+7)          ..(2)

```

At first glance, the expression

```

      (2=5)(3-6)(4×7)          ..(3)

```

is very similar, but has no "in-stranded" version such as (1) above. An APL supporting arrays of functions would allow such a short form:

```

      2 3 4 (+ - ×) 5 6 7          ..(4)

```

The sub-expression (+ - ×) in (4) above is a vector of functions

Operations on Arrays of Functions

If a vector of functions can be created in a way analogous to a vector of data, it should be possible to manipulate function arrays in similar ways. For assignment:

$$F \leftarrow + - \times \quad \dots (5)$$

we can then write

$$2 \ 3 \ 4 \ F \ 5 \ 6 \ 7 \quad \dots (6)$$

As Brown (BR84) points out, function assignment of this sort is a conforming extension of the syntax of APL2, since it currently produces a SYNTAX ERROR.

Once the concepts of functional arrays and assignment are adopted, it is a short step to manipulating arrays of functions. It is at this point that difficulties begin to arise.

1. Syntax Class.

APL embodies the concept of syntax class for each object (primitive or defined) in a workspace. Variables, functions and operators are each given a different class. Name class (as returned by the `⌈NC` system function) is a concept similar to that of syntax class. Variables are given syntax class 0, functions class 1, and operators class 2.

An object of a given syntax class operates on objects only of a lower syntax class. Functions operate on variables, creating new variables. Operators operate on functions (or variables) and return derived functions as a result.

This restriction on the syntax class of arguments and operands is the basis of the problem. For example, if we wish to alter the shape of a function array, the re-shape function cannot be used since a function array has syntax class 1.

2. Possible Solutions.

The concept of a whole new set of operators to manipulate function arrays is impractical (BR84). Presumably it would also be desirable to manipulate arrays of operators, requiring objects of a new syntax class (called Transforms (BK84)). Arrays of transforms then come into question, and so on in an infinite regression.

Landaeta (LA86, LA88) proposes the concept of an elevator object with a new syntax class. The elevator takes an object of syntax class N and returns a derived object of syntax class $N + 1$. Landaeta (LA86) uses the symbol $\rho\$$:

$\rho\$$

returns an operator which can be used to manipulate a function array. Repeated use of the elevator can produce an object of arbitrary syntax class, to operate on objects of the correct associated syntax class.

3. Problems with the elevator.

A number of problems can be identified with the use of the elevator. Firstly what is the syntax class of the elevator itself? Since it can produce an object of arbitrary syntax class, it can only be assigned an "infinite" syntax class. The problem of an infinite regression of syntax classes still exists.

The derived object:

$\rho \$ \quad \dots (7)$

is an operator (with syntax class 2), but obeys the syntactic rules of a function. Landaeta argued that since (7) is an ambivalent operator, which does not exist in APL2, a modified syntax was acceptable. This argument seems somewhat contrived and rather violates the idea of keeping APL syntax as simple as possible.

Semantic Class

The approach of changing the syntax class of an object gives rise to the problem mentioned above. Note, however, the concern is not with the syntax class of the operation in question, but rather with the syntax class of the arguments. The concept of changing the syntax class of an object without changing its syntactic properties is fundamentally flawed. Since the syntactic properties are not being changed, the syntax class should not be changed. Instead the semantic properties of the operation should be modified.

In order to define the semantic properties of an object, a three-item vector is defined for each object in the workspace. This defines the maximum syntax class of the left and right arguments and explicit result. Each item can take values 0, 1 or 2. The semantic property vector for a variable is empty. By default the semantic property vector for a function is 0 0 0, for an operator it is 1 1 1.

A new operator is defined to allow the semantic property vector to be manipulated. The CONTEXT operator is represented by \$ (to maintain consistency with (LA86)). The left operand of CONTEXT is the object whose semantic properties are to be changed. The right operand is the new semantic property vector. The only unusual characteristic of the CONTEXT operator is its default semantic property vector - 2 1 2. The value of 2 for the left operand and explicit result semantic classes allows operators to be manipulated and returned.

1. Use of CONTEXT.

To reshape a function array:

```
F ← + - ×
G ← 2 2(ρ$ 0 1 1)F      ..(8)
```

In (8) above, the CONTEXT operator is used to change the semantic properties of the re-shape function, so that:

The left argument is a variable.
The right argument is a function.
The result is a function.

The array G will be a 2-by-2 function array.

Catenation of function arrays:

```
F ← + -
G ← × ÷
H ← F(,$ 1 1 1)G
```

The derived catenate function has semantic class 1 for both arguments and its explicit result. A function vector is passed for both arguments. A new function vector is returned as the explicit result.

2. Other Functions

```
F ← × ÷ ρ v ^
G ← + ρ ε
G(ε$ 1 1 0)F
0 1 0
```

Membership is being used with both arguments of syntax class 1. The explicit result is of syntax class 0. Note that the symbols in a function vector are not the same as the equivalent symbols in a character vector.

```
F ← × ÷ ρ v ^
G ← '+ρε'
G(ε$ 0 1 0)F
0 0 0
```

3. Missing Syntax Classes

The use of the CONTEXT operator allows objects of mixed syntax class to be created. These have special properties, which will be illustrated below:

```
F ← + - ÷
V ← 1 2 3
X ← F(,$ 1 0 0)V
```

X is a vector containing objects of both variable and function syntax class. An object of mixed syntax class inherits the highest syntax class of any of its constituents. The object X is therefore a function; it is a niladic function since a right argument is already present (and no function takes a left argument only). These properties can be illustrated:

```

      X
-1 - .5 - .33333          ... (9)

```

Note that X is evaluated as if the expression

```

+ - ÷ 1 2 3

```

had been typed, rather than:

```

      (+ - ÷) 1 2 3          ... (10)
1 -2 .33333

```

Other properties:

```

      ρX                      A SHAPE OF RESULT OF X
3
(ρ$ 0 1 0)X                  A SHAPE OF VECTOR X
6

```

Since X is evaluated, a problem arises over displaying the contents of X. In APL2, this can be achieved using the disclose function:

```

      (⊃$ 0 1 0)X
+ - ÷ 1 2 3

```

4. Other mixed syntax-class objects

The niladic function X is not very useful, however if a single argument is supplied on the left, a named monadic function can be created:

```

      INCREMENT+1 (, $ 0 1 1) +
      INCREMENT 3
4
      INCREMENT INCREMENT 4 5 6
6 7 8
      ρINCREMENT
SYNTAX ERROR
      ρINCREMENT
      A A
      (ρ$ 0 1 0) INCREMENT
2
      (⊃$ 0 1 0) INCREMENT
1+

```

5. Properties of mixed syntax-class objects

From the above examples, (especially (9) and (10)), it is easy to see that a reference to an object containing mixed syntax classes causes an attempt at evaluation. In other words, many variables, functions or operators can be stranded together without any evaluation taking place. As soon as an attempt is made to mix syntax classes via stranding, evaluation is triggered. Objects of mixed syntax class have to be created by the use of catenation or some other function.

User Defined Functions

The creation of arrays of functions is not limited to primitive functions. Defined functions may also be used, with beneficial effects on several "traditional" APL problems. For example, in a menu-driven system, one technique for executing a function selected from a menu is to pick the function name from an array and then execute it. The problems associated with the use of execute are discussed elsewhere (P186,P188); its use should be avoided.

If function arrays are permitted, execute can be avoided:

```
OPTS←OPT_1 OPT_2 OPT_3 OPT_4
RC←(OPT_NO(⊃$ 0 1 1)OPTS) PARMS
```

The option number (OPT_NO) is used to pick the correct function from the list. The reference to this function with the argument PARMS causes its evaluation.

1. ⍎CR and ⍎FX Redefined

The system functions ⍎CR and ⍎FX are often criticised. ⍎FX is particularly criticised since the function is created as a side effect, it is not the explicit result of the function. ⍎FX could be redefined to use function assignment. The semantic vector of the new function is 0 0 2, that is it takes two data arguments and returns a function:

```
MYFN←⍎FX ARRAY
```

The right argument contains the code of the function (or defined operator). Note the function is assigned as the explicit result of the system function. The ⍎CR function is defined as the inverse of ⍎FX, its semantic property vector is 0 2 0.

```
ARRAY←⍎CR MYFN
```

Note the function is passed explicitly as the right argument (rather than simply being named as the right argument). The explicit result is the character representation of the function.

2. The DEL Editor

In the same way that `□FX` was redefined, the DEL editor can be defined as a primitive function with semantic property vector 0 2 2:

```
NEWFN+ VOLDFN
```

The facilities provided by DEL are of course implementation dependent. For editing a new function, the jot can be used to represent a null function:

```
NEWFN+ ∇ °
```

The left argument of `∇` could be implemented either to specify the execution properties of the defined object (as with `□FX` in APL2) or to pass an opening command to the editor (as a character vector).

Function Headers

Since the function is being named as the explicit result of both `□FX` and the DEL editor, a question can be raised about the meaning of the function name specified in the function header line. The function assignment capability allows the meaning of names in the header to be rationalised.

Currently, all names in the header of a defined function are local, except for the function name. By explicitly assigning the function to a name, the function name specified in the header can be considered the local function name. This is particularly useful for recursive functions - consider the following definition of a recursive factorial function:

```
CODE
R+ ΔFACT NUM
A RECURSIVE FACTORIAL FN.
R+1
→(NUM≤1)/0
R+NUM×ΔFACT NUM-1

FACTORIAL+□FX CODE
FACTORIAL 4

24
ΔFACT 4
VALUE ERROR
ΔFACT 4
A
```

The benefit of this approach to recursive functions is that the interpreter "knows" it is re-invoking the same function all the time. Because of this knowledge, the local copy of the function re-invoked each time can be a "pre-parsed" version - all the work involving

syntactic analysis is performed only once. The impact on performance of recursive code could be considerable.

Conclusions

The inclusion of arrays of functions into APL syntax seems at first glance to be a natural extension, relatively simple to implement. The number of papers written on the subject is mute testament to the difficulties that are encountered in practice.

The distinction drawn between syntactic and semantic properties appears to solve many of the problems. Functions remain functions but are semantically transformed to operate on arrays of functions rather than data. Contrast this to the approach of Landaeta where functions are transformed into operators, but retain the syntactic properties of functions.

The three-item semantic property vector allows the semantic properties of the left and right arguments and the explicit result to be specified independently. This independence can be used to rationalise the definitions of $\square CR$ and $\square FX$ as well as the DEL editor.

The assignment of functions as the explicit result of the editor or $\square FX$ clarifies the meaning of the function name in the function header. This development presents the opportunity for a considerable improvement in the performance of recursive functions by removing one of the major performance overheads - the continual re-invocation of the same unit of code (note that some implementations have already by-passed this problem in other ways).

The introduction of arrays of functions to APL is probably the next major enhancement to the language following nested arrays. The implications for the power, flexibility and usability of the language are probably as great as they have been with nested arrays.

References

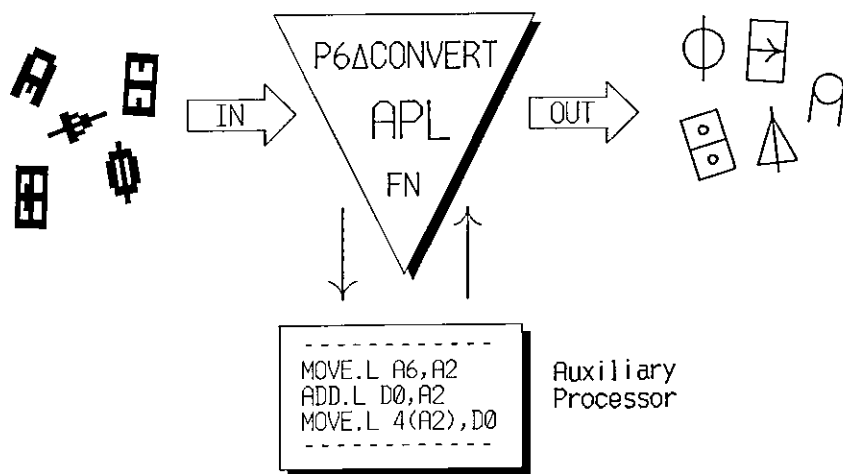
- LA86 Landaeta D.J. A Notation for Manipulating Arrays of Operations
APL86 proceedings APL Quote Quad Vol.16 No.4
- LA88 Landaeta D.J. The Theory of Function Arrays
APL88 proceedings APL Quote Quad Vol.18
- BE84 Bernecky R. Function Arrays
APL84 proceedings APL Quote Quad Vol.14 No.4
- BR84 Brown J.A. Function Assignment and Array of Functions
APL84 proceedings APL Quote Quad Vol.14 No.4
- BK84 Benkard J.P.
Syntactic Experiments with Arrays of Functions and Operators
APL84 proceedings APL Quote Quad Vol.14 No.4
- P186 Piper D.B. Using Quad-FX to Facilitate the use of Auxiliary Processors
Technical Articles Section Vector Vol.3 No.2
- P188 Piper D.B. APL88 Code Quality
Letter in Technical Correspondence Section. Vector Vol.4 No.4

Other Articles of Interest

- Hodgkinson R. APL Procedures
APL86 proceedings APL Quote Quad Vol.16 No.4
- Werder A. Arrays of Objects in Rationalised APL
APL88 proceedings APL Quote Quad Vol.18
- Gfeller M. A Framework for Extensions to APL
APL88 proceedings APL Quote Quad Vol.18

A System For Printing From APL.68000

by Iain Hayward



Abstract

This article describes an APL font, an auxiliary processor, and associated functions for printing from within APL.68000 to a 24-pin dot matrix printer.

Introduction

APL is an intellectual tool. Until something better comes along it is the only computer language which lets one program almost fast enough to avoid hindering one's thought processes, and still allow unlimited flexibility. The perfect development environment for APL is undoubtedly a WIMPS system, and a few enlightened hardware manufacturers (such as Commodore, Atari and Apple) produce such systems at affordable prices. Thanks to MicroAPL we now have an excellent APL for the Amiga, Atari ST and the Macintosh, lacking only nested arrays (which remain controversial) and good printer support. The latter I have attempted to put right myself, by developing a set of utilities on the Amiga which are also portable to other systems. I am donating the results of my work to the BAA Software Library on a disk to which this article can be considered the documentation.

Method

The first step was to get an APL font for my printer. Since I was unable to find one (well, not a free one anyway!) I created my own using a font designer which I had written in APL. The program was inspired by an article in MicroAPL News on editing bit maps on the Macintosh. With the font installed and selected, the printer could be made to emulate a normal APL device, provided that only standard APL characters were printed (including backspace) and that no attempt was made to switch character sets. I was now able to use the `⌈HC` system function to obtain a continuous hard copy of the screen.

What I really wanted though, was an APL function which could print any APL text variable directly to the printer, including all characters in the atomic vector. This is a feature missing from the current version (6.04C) of APL68000 for the Amiga, regardless of the type of printer you have. The function would have to do two things; first translate the character codes and resolve overstrikes, then send the output to the printer via DOS. The second part of the operation is system dependent, so in order to increase portability, and following the ideal of one job per function, I decided to separate the two processes. The first part was written as an auxiliary processor in assembly language (for speed) and should work on any 68000 based system. The second part was written in APL using DOS functions provided by MicroAPL in a utilities workspace. Since these functions appear to have their counterparts on other systems conversion should be easy.

All that remained then, was to write a few simple functions for initializing and selecting options on the printer, and one to return the vector representation of a function to be printed. The latter was needed because in APL68000 there is no `⌈VR` system function.

The Font

A sample display of the font was published in Vol.5, No.3 of Vector (page 110), however the non-APL characters shown there were actually from the resident character sets of my printer; the NEC P6. For the font I have redefined all characters with ASCII codes in the range 32 to 127, with the exception of space and the digits 0 to 9 which occupy the same position as their non-APL counterparts. Overstrike characters are obtained in the traditional way by back-spacing, and don't seem to inconvenience the printer at all.

I have included a listing of the dot patterns for each character in the font in a file on the BAASL disk so that anyone whose printer is incompatible with the NEC P6 can easily edit and adapt the font. It will be necessary to write a small program to do this of course, and the format of the file for each character is as follows:

- Line 1: decimal ASCII code of the character (32 to 127).
- Line 2: LEFTSPACE, WIDTH, RIGHTSPACE (measured in dots).
- Lines 3 to 26: dot pattern (24 lines by WIDTH columns).

The sum of the values in line 2 is always the same in order to ensure that overstrikes are properly aligned, even if proportional spacing has been selected. The font data resides in the workspace in a global integer vector called P6ΔAPLΔFONTΔDATA.

The P6ΔCONVERT function

This is a cover function for an auxiliary processor which takes any character variable, including high order arrays, and returns a character vector of ASCII characters to be sent to the printer in order to print that variable as it would appear on the display. For the sake of flexibility, P6ΔCONVERT ignores the print width setting and does not append a terminating carriage return.

P6ΔCONVERT uses the resident character sets in the printer as far as possible, inserting control sequences to select national character sets where necessary. The assembly code for the auxiliary processor resides in a global integer vector called P6ΔCONVERTΔAPΔCODE and is referenced by the cover function.

The WRITEDOS function

This function sends character data directly to a DOS file or device without any conversion taking place.

```

▽ R←DOSFILE WRITEDOS DATA;TIENR
[1]  A▽ Writes character data to an AmigaDOS file or device.
[2]  A Right argument is data (any format).
[3]  A Left argument is AmigaDOS file spec (char vector or scalar).
[4]  A Returns AmigaDOS error code (integer scalar).
[5]  A Uses AMCREATE, AMTIE, AMUNTIE & AMWRITE from workspace AMFILE.
[6]  TIENR+1 A File tie number.
[7]  ERS(4*□DR DOSFILE)/11 A File spec must be characters.
[8]  A(0=ρρDOSFILE)/'DOSFILE+',DOSFILE' A Deal with scalar.
[9]  ERS(1*ρρDOSFILE)/4 A No arrays.
[10] DATA←,DATA A Make data a vector.
[11] A(4*□DR DATA)/'DATA+4 □DR DATA' A Make it characters.
[12] A(0=R+TIENR AMTIE DOSFILE)/GOTFILE A Tie AmigaDOS file.
[13] A(205*R)/0 A Quit unless error = FILE NOT FOUND.
[14] A(0*R+AMCREATE DOSFILE)/0 A Create AmigaDOS file.
[15] A(0*R+TIENR AMTIE DOSFILE)/0 A Tie AmigaDOS file.
[16] GOTFILE:
[17] A(0*R+DATA AMWRITE TIENR)/0 A Write out data.
[18] R←AMUNTIE TIENR A Untie file.
[19] A▽ <AMUTILS> by Iain Hayward, 22Apr89.
▽

```

WRITEDOS is used for downloading the font data and for sending control codes to the printer. It could be used to write out the font data to a DOS file which is then copied to the printer as part of the startup sequence.

The left argument to WRITEDOS is a character vector containing the AmigaDOS output device or file specification. In my case this is the parallel port device "PAR:". I use it in preference to the printer device "PRT:" because the data may contain characters which are interpreted by the printer driver as a control sequence. "SER:" should be used by anyone whose printer is connected to the serial port. The right argument to WRITEDOS is the data to be written and would normally be a character vector, although any other type of variable is accepted and converted to bytes.

WRITEDOS returns an integer scalar containing an AmigaDOS error number. Normally this is zero indicating no error, but if any other value is returned then something is wrong, and an explanation can be obtained either by calling the AFE function in workspace AMFILE, or else by looking up the error number in the AmigaDOS manual.

The PRINT function

This is really just a cover function for printing in APL which can be customized to suit your own environment and preferences. It calls P6ΔCONVERT to convert the data and then WRITEDOS to send it to the printer. The example below specifies the printer device, formats numeric data, and appends a terminating newline character.

```

▽ R←PRINT INDATA;OUTDATA;MYDEVICE
[1]  A▽ Prints APL data on a NEC P6 printer with downloaded font.
[2]  A Right argument is data to be printed (any format).
[3]  A Returns AmigaDOS error code (integer scalar).
[4]  MYDEVICE←'PAR:' A Printer device specification.
[5]  ⍠(4#⊞DR INDATA)/'INDATA+▽INDATA' A Convert numeric to text.
[6]  OUTDATA←P6ΔCONVERT INDATA A Translate codes for the printer.
[7]  ⍠(⊞L*~1↑,INDATA)/'OUTDATA+OUTDATA,P6ΔCONVERT ⊞L' A Append a newline?
[8]  R←MYDEVICE WRITEDOS OUTDATA A Send it to the printer.
[9]  A▽ <P6UTILS> by Iain Hayward, 22Apr89.

```

▽

Initialising the Printer

I have created a function called `INITPRINTER` to download the APL character font and to modify some of the printer settings. It takes one argument which is an integer vector of print options. The vector currently contains only two elements, the first being the width of the left margin in columns, and the second a boolean flag to indicate whether to select or cancel proportional spacing. The function can easily be extended to take other options, but it is wise to select all options before downloading the font. I have found that function listings look best with proportional spacing on, especially if they contain a lot of comments, but for printing arrays of data it is best to turn this off so that the columns line up. `INITPRINTER` reports its progress as in the following example:

```
INITPRINTER 5 1
    Left margin 5 columns,
    Proportional spacing on,
    Downloading font data...
    Done!
```

One should allow a few seconds for the font data to be downloaded, but make sure that the printer is on-line and has paper loaded, or `INITPRINTER` may hang.

Writing the AP routines

There is a small section at the back of the APL.68000 Language Manual which describes the auxiliary processor mechanism. This gives most of the information needed to write a machine code routine to be accessed via AP 1, but it still leaves a few questions unanswered. In particular there are no examples, and no description of the format of a symbol table entry. MicroAPL do not provide a toolbox of low-level routines either such as is available for Dyalog APL, but I suspect that all of their functions are custom-coded for speed. Fortunately MicroAPL were very helpful here, promptly answering my queries, and even sending me a listing of the source code for an example auxiliary processor.

The code for an auxiliary processor is stored in an APL variable and must be position-independent so that it can be moved anywhere in memory and still work. For development of the routines I used the excellent German "Profimat" assembler which has an option to convert instructions into their PC-relative form as far as possible, and to report an error where this is not possible. The object code was loaded into a variable using DOS utility functions from the AMFILE workspace, and then converted to integer format so that it does not corrupt the screen if displayed.

For anyone interested in writing auxiliary processors I am planning an article on the subject for a future edition of MicroAPL News.

Portability

The APL functions and the auxiliary processor were written for APL68000 and should work on any system based on the 68000 processor. It will be necessary to modify the WRITEDOS function to use the DOS utility functions (listed in line 5) which MicroAPL has provided for that system and to change the DOS error return code in line 13 (I believe it is 33 for the Atari ST).

To use the software with printers incompatible with the NEC P6 it will be necessary to modify the font and/or the auxiliary processor. For the benefit of anyone wishing to do this I have included a machine readable version of the font (see above) and the source code for the auxiliary processor on the BAASL disk.

Copyright Note

One or two colleagues have drawn my attention to the commercial potential of this software, and indeed I can see that it might be of some value to a business using APL. However it is my wish that it reaches the hands of people who can't or wouldn't pay for it; typically home computer users and people in education. I am giving it away in order to help and encourage amateur users of APL.

I would like to make it quite clear that these utilities and the font are going into the public domain, that they are available free, and that I retain all rights. They may not be distributed as, or as part of, a commercial product.

Iain Hayward

280A rue de Neudorf
L-2222 Luxembourg
G. D. of Luxembourg

Order Form: I-APL Ltd February 1989

The address for queries is:
 2 Blenheim Road
 St Albans
 Herts AL1 4NR

The address for orders is:
 56 The Crescent, Milton
 Weston-super-Mare
 Avon BS22 8DU

Please supply:

Part	Item	(pounds sterling)		
		No copies	Price	Total
PC12	PC disk & manual (counts 1 item)	_____	£4	_____
BBC12	B/Master disk & manual (counts 1 item)	_____	£4	_____
A12	Archimedes disk & manual (counts 1 item)	_____	£4	_____
T3	Tutorial by Thomson & Alvord (1 item) (Paper 45pp)	_____	£2	_____
M2	I-APL PC BBC & Archimedes manual (1) (Paper about 64pp)	_____	£2	_____
E4	Encyclopedia by Helzer (1 item) (Paper vii+299pp)	_____	£5	_____
S5	APL in Social Studies by Traberman (1) (Paper 36pp)	_____	£2	_____
PLEASE	Postage 50 pence per item _____ items		x 0.50	_____
PAY BOTH				
POST AND	Packing charge per order		£1	1.00
PACKING				

(Optional donation _____) I enclose cheque to I-APL Ltd for: _____

We take no credit cards.

Name: _____

Full address: _____

Please send me a VAT invoice (tick) ☐ Vat is only chargeable at 15% on the disk (27p).
 The above prices include VAT. Regd No 479 1634 13

Office use only

British APL Association Software Library: Order Form

To: Alison Chatterton
 9 Oak Grove,
 Hertford,
 Herts SG13 8AT, ENGLAND

Please supply the disks circled below:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53
54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71
72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89
90	91	92	93	94	95	96	97	98	99	100							

Total number of disks (please check): _____

at 2 Pounds per disk: BAA Member's rate _____
 OR at 3 Pounds per disk: Non-member's rate _____

I want to join the BAA, and enclose
 the 10 Pounds annual membership fee: _____

Postage and handling: 1.00

Add 2 Pounds for orders outside the UK: _____

Total order - remittance included: _____

Your signature: _____

Your name: _____

Full address: _____

Please make cheques payable to the British APL Association. Payment can be made in \$ - pay 1.5\$ (US) for each £. All orders must include payment - allow 30 days for delivery.

Software is accepted by the British APL Association on good faith and we do not vouch for or make any claims regarding donated software. The BAA shall not be held responsible for damage caused by the use of library software, and shall not be liable in any way in the event that submitted material proves to be the subject of third party copyright. Your signature above accordingly indemnifies the BAA from all liability.

Index to Advertisers

APL People	88
British Airways	16,17
British APL Association	6
Cocking and Drury	112,113
Dyadic Systems Ltd	2,50
HMW Computing	4
MicroAPL	22,88
APL Booklist (Renaissance Data Systems)	12

All queries regarding advertising in VECTOR should be made to Alison Chatterton, at the address on the inside back cover.

BRITISH APL ASSOCIATION

Membership Application/Renewal Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____
 Department: _____
 Organisation: _____
 Address Line 1: _____
 Address Line 2: _____
 Address Line 3: _____
 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category (please circle):89/90

Non-voting student membership (UK only)£5
 UK private membership£10
 Overseas private membership£18
 Airmail supplement (not needed for Europe)£8
 Corporate membership£85
 Corporate membership overseas£140
 Sustaining membership£360

For student applicants:

Name of course: _____
 Name and title of supervisor: _____
 Signature of supervisor: _____

If you would like to pay by credit card ...

Card no (Access/Visa): _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque "The British APL Association", or you may quote your Access or Visa number. Please send the completed form to:

BAA Administration,
 Alison Chatterton, 9 Oak Grove,
 HERTFORD SG13 8AT
 England

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1988/89 Committee

Chairman:	Peter Donnelly 0420-87024	Dyadic Systems Ltd., Park House, The High Street, ALTON, Hants GU34 1EN
Secretary:	Graham Parkhouse 0483-571281 (x2379)	Dept. of Mech. Eng., University of Surrey, GUILDFORD GU2 5XH.
Treasurer:	John Sullivan 01 462-3628	24 Mounthurst Road, Hayes, Kent BR2 7QN
Journal Editor:	Adrian Smith 04393-385	Brook House, Gilling East, YORK YO6 4JJ
Education:	Dr Alan Sykes 0792-295296	Dept of Management Science University College of Swansea, Singleton Park, SWANSEA SA2 8PP
Publicity:	Jonathan Martin 01 562-5697	(S499) British Airways, PO Box 10, Heathrow Airport, HOUNSLOW, Middlesex TW6 2JA
Technical:	David Eastwood 01-922 8866	MicroAPL Ltd South Bank Technopark, 90 London Road, LONDON SE1 6LN
Recruitment:	Jill Moss 0225-462602	APL People Ltd, The Old Malthouse Clarence St., BATH, Avon BA1 5NS
Projects:	John Searle 01 948-6737	13A Mount Ararat Road, Richmond, Surrey TW10 6PQ
Administration:	Alison Chatterton 0992-552489	9, Oak Grove, HERTFORD, SG13 8AT

Journal Working Group

Jonathan Barman	048839-575
Anthony Camacho	0727-60130
Cathy Dargue	0895-31313
Val Lusmore	0225-448501
Adrian Smith	04393-385
David Ziemann	01-436 9481

Activities Working Group

Phil Goucher	03727-21282
Maurice Jordan	01-562 3090
David Parker	01-834 2333

Typeset using MS Word 4.0 on Xerox 3700 (Covers on Apple Laserwriter)

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People
The Old Malthouse
Clarence St, BATH, BA1 5NS
Tel 0225 452602

Cocking & Drury Ltd
180 Tottenham Court Rd
LONDON, W1P 9LE
Tel 01-436 9431

HMW Computing Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel 01-853 4242
Fax 01-853 8825

Inner Product Ltd
Eagle House
78 Clapham Common Southside
LONDON SW4 9DG
Tel 01-873 8354

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel 01-922 8866

Peter Gyrfax Systems
213 Goldhurst Terrace
LONDON NW3 8ER
Tel 01-624 7016
Mobile 0860-877963

APL Software Tech
14 Rosewood Avenue
Avonson, BRISTOL BS12 2PP
Tel 0454 415747

Dyadic Systems Ltd
Park House, The High Street
ALTON, Hants
Tel 0420 37024

APL Impetus Ltd
Rusper, Sandy Lane
Ivy Hatch, SEVENOAKS
Kent TN13 0PD
Tel 0732 335126

Morotechnics Systems Ltd
356 West Barnes Lane
Molesey Park, Surrey
Tel 01-949 6288
Fax 01-336 1062

H.P. Sharp Associates
10 Dean Parry St
LONDON SW1A 1QB
Tel 01-222 7033

